

Documentacion de Arquitectura de Software: Mapeo Relacional Profundo

Sistema EcoCircuit:

Víctor Martínez

Julio 2026

1 Estructura del Proyecto EcoCircuit

EcoCircuit/ EcoCircuit/main.py EcoCircuit/config.py
EcoCircuit/analitica_avanzada/ EcoCircuit/analitica_avanzada/__init__.py EcoCircuit/analitica_avanzada/control_calidad.py EcoCircuit/analitica_avanzada/procesamiento_senales.py EcoCircuit/analitica_avanzada/dashboards_ui.py
EcoCircuit/analitica_avanzada/componentes_dashboard/ EcoCircuit/analitica_avanzada/componentes_dashboard/__init__.py EcoCircuit/analitica_avanzada/componentes_dashboard/motor_grafico_multimodal.py EcoCircuit/analitica_avanzada/componentes_dashboard/gestor_memoria_ds.py EcoCircuit/analitica_avanzada/componentes_dashboard/panel_series_tiempo.py EcoCircuit/analitica_avanzada/componentes_dashboard/panel_fft.py EcoCircuit/analitica_avanzada/componentes_dashboard/panel_spc.py
EcoCircuit/analitica_avanzada/nucleo_matematico/ EcoCircuit/analitica_avanzada/nucleo_matematico/motor_inferencias.py
EcoCircuit/controladores/ EcoCircuit/controladores/gestor_3d.py EcoCircuit/controladores/gestor_io.py EcoCircuit/controladores/gestor_scada.py
EcoCircuit/core/ EcoCircuit/core/catalogo_electrico.py EcoCircuit/core/exportador_mep.py EcoCircuit/core/migrador.py EcoCircuit/core/model.py EcoCircuit/core/motor_3d.py EcoCircuit/core/project_manager.py EcoCircuit/core/utilidades.py EcoCircuit/core/window_manager.py
EcoCircuit/simulador/ EcoCircuit/simulador/motor_datos.py EcoCircuit/simulador/motor_tiempo.py EcoCircuit/simulador/panel_simulacion.py
EcoCircuit/ui/ EcoCircuit/ui/canvas.py EcoCircuit/ui/dialogos_electricos.py EcoCircuit/ui/inspector_handler.py EcoCircuit/ui/inspector_window.py EcoCircuit/ui/level_manager.py EcoCircuit/ui/main_window.py EcoCircuit/ui/menu_bar.py EcoCircuit/ui/section_manager.py EcoCircuit/ui/toolbar.py EcoCircuit/ui/view_3d_global.py EcoCircuit/ui/view_level_detail.py EcoCircuit/ui/visor_3d.py EcoCircuit/ui/wizard.py
EcoCircuit/ui/componentes_canvas/ EcoCircuit/ui/componentes_canvas/__init__.py EcoCircuit/ui/componentes_canvas/base.py EcoCircuit/ui/componentes_canvas/mixin_datos.py EcoCircuit/ui/componentes_canvas/mixin_renderizado.py EcoCircuit/ui/componentes_canvas/mixin_eventos.py EcoCircuit/ui/componentes_canvas/mixin_interaccion.py
EcoCircuit/ui/inspector/ EcoCircuit/ui/inspector/__init__.py EcoCircuit/ui/inspector/geometria_y_camaras.py EcoCircuit/ui/inspector/nucleo_estetico.py EcoCircuit/ui/inspector/paneles_auxiliares.py EcoCircuit/ui/inspector/renderizador_plano.py EcoCircuit/ui/inspector/renderizador_volumetrico.py EcoCircuit/ui/inspector/ventana_maestra_inspector.py
EcoCircuit/utils/ EcoCircuit/utils/__init__.py EcoCircuit/utils/colores.py EcoCircuit/utils/generador_3d.py EcoCircuit/utils/generador_arquitectura.py EcoCircuit/utils/grafico_electrico.py EcoCircuit/utils/logger.py EcoCircuit/utils/motor_estructural.py EcoCircuit/utils/motor_mep_solido.py EcoCircuit/utils/procesador_geometria.py EcoCircuit/utils/serializador_3d.py

Índice

1. Estructura del Proyecto EcoCircuit	1
2. Archivo: main.py , RUTA: EcoCircuit/main.py (El Orquestador Maestro y Bootstrapper)	7
2.1. 1. Importaciones y Dependencias Estratégicas	7
2.2. 2. Variables Globales y Metadatos BIM	7
2.3. 3. Análisis de Clases y Funciones (Desglose Lógico)	7
2.4. 4. El Orquestador: Función main()	7
2.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	7
2.6. 6. Relación Exocrina (Con el resto del Ecosistema)	7
2.7. 7. Aporte al Funcionamiento Global Total	7
3. Archivo: config.py (El <i>Single Source of Truth</i> Paramétrico)	8
3.1. 1. Importaciones y Dependencias Estratégicas	8
3.2. 2. Variables Globales y Metadatos BIM	8
3.3. 3. Análisis de Clases y Funciones (Lógica Paramétrica)	8
3.4. 4. El Orquestador y Ciclo de Vida	8
3.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	8
3.6. 6. Relación Exocrina (Con el resto del Ecosistema)	8
3.7. 7. Aporte al Funcionamiento Global Total	8
4. Ruta y Archivo: EcoCircuit/analitica_avanzada/_init_.py	9
4.1. 1. Importaciones y Dependencias Estratégicas	9
4.2. 2. Variables Globales y Metadatos BIM	9
4.3. 3. Análisis de Clases y Funciones (Lógica de Enrutamiento)	9
4.4. 4. El Orquestador y Ciclo de Vida	9
4.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	9
4.6. 6. Relación Exocrina (Con el resto del Ecosistema)	9
4.7. 7. Aporte al Funcionamiento Global Total	9
5. Ruta y Archivo: EcoCircuit/analitica_avanzada/control_calidad.py	10
5.1. 1. Importaciones y Dependencias Estratégicas	10
5.2. 2. Variables Globales y Metadatos BIM	10
5.3. 3. Análisis de Clases y Funciones (Desglose Matemático)	10
5.3.1. A. Algoritmo EWMA (Media Móvil Exponencialmente Ponderada)	10
5.3.2. B. Algoritmo CUSUM (Sumas Acumuladas Tabulares)	10
5.3.3. C. Filtro Z-Score (Detección de <i>Outliers</i>)	10
5.4. 4. El Orquestador y Ciclo de Vida	10
5.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	10
5.6. 6. Relación Exocrina (Con el resto del Ecosistema)	10
5.7. 7. Aporte al Funcionamiento Global Total	10
6. Ruta y Archivo: EcoCircuit/analitica_avanzada/procesamiento_senales.py	11
6.1. 1. Importaciones y Dependencias Estratégicas	11
6.2. 2. Variables Globales y Metadatos	11
6.3. 3. Análisis de Clases y Funciones (Desglose Matemático)	11
6.3.1. A. Remoción de la Componente Continua (DC Offset)	11
6.3.2. B. Enventanado (Hanning Window)	11
6.3.3. C. Transformada Rápida de Fourier (FFT)	11
6.3.4. D. Conversión de Escala Frecuencial	11
6.4. 4. El Orquestador y Ciclo de Vida	11
6.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	11
6.6. 6. Relación Exocrina (Con el resto del Ecosistema)	11
6.7. 7. Aporte al Funcionamiento Global Total	11
7. Ruta y Archivo: EcoCircuit/analitica_avanzada/dashboards_ui.py	12
7.1. 1. Importaciones y Dependencias Estratégicas	12
7.2. 2. Variables Globales y Metadatos de UI	12
7.3. 3. Análisis de Clases y Funciones (Desglose Lógico)	12
7.3.1. A. Construcción del Layout (<i>_init_ui</i>)	12
7.3.2. B. Sistema de Reactividad (<i>_conectar_senales</i>)	12
7.3.3. C. Inyección de Contexto (<i>actualizar_datos</i>)	12
7.4. 4. El Orquestador y Ciclo de Vida	12
7.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	12
7.6. 6. Relación Exocrina (Con el resto del Ecosistema)	12
7.7. 7. Aporte al Funcionamiento Global Total	12
8. Ruta y Archivo: EcoCircuit/analitica_avanzada/componentes_dashboard/_init_.py	13
8.1. 1. Importaciones y Dependencias Estratégicas	13
8.2. 2. Variables Globales y Metadatos de Encapsulamiento	13
8.3. 3. Análisis de Clases y Funciones (Lógica Topológica)	13
8.4. 4. El Orquestador y Ciclo de Vida	13
8.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	13
8.6. 6. Relación Exocrina (Con el resto del Ecosistema)	13
8.7. 7. Aporte al Funcionamiento Global Total	13
9. Ruta y Archivo: EcoCircuit/analitica_avanzada/componentes_dashboard/motor_grafico_multimodal.py	14
9.1. 1. Importaciones y Dependencias Estratégicas	14
9.2. 2. Variables Globales y Metadatos Gráficos	14
9.3. 3. Análisis de Clases y Funciones (Desglose Lógico)	14
9.3.1. A. Renderizado de Series Temporales (<i>graficar_curva</i>)	14
9.3.2. B. Renderizado Multimodal (Áreas e Histogramas)	14
9.3.3. C. Inyección de Anomalías y Límites (<i>marcar_anomalias</i>)	14
9.4. 4. El Orquestador y Ciclo de Vida	14
9.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	14
9.6. 6. Relación Exocrina (Con el resto del Ecosistema)	14
9.7. 7. Aporte al Funcionamiento Global Total	14
10.Ruta y Archivo: EcoCircuit/analitica_avanzada/componentes_dashboard/gestor_memoria_ds.py	15
10.1.1. Importaciones y Dependencias Estratégicas	15
10.2. 2. Variables Globales y Metadatos de Caché	15
10.3. 3. Análisis de Clases y Funciones (Desglose Lógico)	15
10.3.1. A. Ingestión y Parseo (<i>cargar_telemetria</i>)	15
10.3.2. B. Vectorización e Ingeniería de Características (<i>calcular_variables_derivadas</i>)	15
10.3.3. C. Slicing y Recuperación Rápida (<i>obtener_ventana_temporal</i>)	15
10.4. 4. El Orquestador y Ciclo de Vida	15
10.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	15
10.6. 6. Relación Exocrina (Con el resto del Ecosistema)	15
10.7. 7. Aporte al Funcionamiento Global Total	15

11.Ruta y Archivo: EcoCircuit/analitica_avanzada/componentes_dashboard/panel_series_tiempo.py	16
11.1. 1. Importaciones y Dependencias Estratégicas	16
11.2. 2. Variables Globales y Metadatos de Estado	16
11.3. 3. Análisis de Clases y Funciones (Desglose Lógico)	16
11.3.1. A. Topología de UI (<code>_init_ui</code>)	16
11.3.2. B. Emisión de Eventos de Región (<code>on_region_changed</code>)	16
11.3.3. C. Actualización de Datos (<code>actualizar_grafica</code>)	16
11.4. 4. El Orquestador y Ciclo de Vida	16
11.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	16
11.6. 6. Relación Exocrina (Con el resto del Ecosistema)	16
11.7. 7. Aporte al Funcionamiento Global Total	16
12.Ruta y Archivo: EcoCircuit/analitica_avanzada/componentes_dashboard/panel_fft.py	17
12.1. 1. Importaciones y Dependencias Estratégicas	17
12.2. 2. Variables Globales y Metadatos de Estado	17
12.3. 3. Análisis de Clases y Funciones (Desglose Lógico)	17
12.3.1. A. Topología de UI y Ejes (<code>_init_ui</code>)	17
12.3.2. B. Traducción Frecuencial (<code>actualizar_fft</code>)	17
12.3.3. C. Detección Visual de Armónicos Dominantes (Picos)	17
12.4. 4. El Orquestador y Ciclo de Vida	17
12.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	17
12.6. 6. Relación Exocrina (Con el resto del Ecosistema)	17
12.7. 7. Aporte al Funcionamiento Global Total	17
13.Ruta y Archivo: EcoCircuit/analitica_avanzada/componentes_dashboard/panel_spc.py	18
13.1. 1. Importaciones y Dependencias Estratégicas	18
13.2. 2. Variables Globales y Metadatos de Estado	18
13.3. 3. Análisis de Clases y Funciones (Desglose Lógico)	18
13.3.1. A. Topología de Interfaz y Controles (<code>_init_ui</code>)	18
13.3.2. B. Evaluación Estadística y Ruteo (<code>actualizar_spc</code>)	18
13.3.3. C. Inyección Visual Multicapa (<code>renderizar_anomalias</code>)	18
13.4. 4. El Orquestador y Ciclo de Vida	18
13.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	18
13.6. 6. Relación Exocrina (Con el resto del Ecosistema)	18
13.7. 7. Aporte al Funcionamiento Global Total	18
14.Ruta y Archivo: EcoCircuit/analitica_avanzada/nucleo_matematico/motor_inferencias.py	19
14.1. 1. Importaciones y Dependencias Estratégicas	19
14.2. 2. Variables Globales y Metadatos Físicos	19
14.3. 3. Análisis de Clases y Funciones (Desglose Matemático)	19
14.3.1. A. Inferencia de Potencia y Factor de Potencia	19
14.3.2. B. Cálculo Vectorizado del Efecto Joule	19
14.3.3. C. Inferencia de Deriva Térmica (<i>Thermal Drift</i>)	19
14.4. 4. El Orquestador y Ciclo de Vida	19
14.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	19
14.6. 6. Relación Exocrina (Con el resto del Ecosistema)	19
14.7. 7. Aporte al Funcionamiento Global Total	19
15.Ruta y Archivo: EcoCircuit/controladores/gestor_3d.py	20
15.1. 1. Importaciones y Dependencias Estratégicas	20
15.2. 2. Variables Globales y Metadatos Espaciales	20
15.3. 3. Análisis de Clases y Funciones (Desglose Lógico)	20
15.3.1. A. Algoritmo de Extrusión Bidimensional (<code>generar_volumen</code>)	20
15.3.2. B. Mapeo Topológico Inverso (Ray-Casting y Picking)	20
15.3.3. C. Actualización de Materiales (<i>PBR Shaders</i>)	20
15.4. 4. El Orquestador y Ciclo de Vida	20
15.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	20
15.6. 6. Relación Exocrina (Con el resto del Ecosistema)	20
15.7. 7. Aporte al Funcionamiento Global Total	20
16.Ruta y Archivo: EcoCircuit/controladores/gestor_io.py	21
16.1. 1. Importaciones y Dependencias Estratégicas	21
16.2. 2. Variables Globales y Metadatos de Archivo	21
16.3. 3. Análisis de Clases y Funciones (Desglose Lógico)	21
16.3.1. A. Proceso de Serialización y Aplanamiento (<code>guardar_proyecto</code>)	21
16.3.2. B. Reconstrucción Topológica (<code>cargar_proyecto</code>)	21
16.3.3. C. Escritura Atómica y Prevención de Corrupción (<code>escritura_segura</code>)	21
16.4. 4. El Orquestador y Ciclo de Vida	21
16.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	21
16.6. 6. Relación Exocrina (Con el resto del Ecosistema)	21
16.7. 7. Aporte al Funcionamiento Global Total	21
17.Ruta y Archivo: EcoCircuit/controladores/gestor_scada.py	22
17.1. 1. Importaciones y Dependencias Estratégicas	22
17.2. 2. Variables Globales y Metadatos Temporales	22
17.3. 3. Análisis de Clases y Funciones (Desglose Lógico)	22
17.3.1. A. Bucle de Simulación e Integración (<code>physics_tick</code>)	22
17.3.2. B. Adquisición de Telemetría Sintética (<code>recolectar_lecturas</code>)	22
17.3.3. C. Canalización y Despacho (<code>emitir_paquetes</code>)	22
17.4. 4. El Orquestador y Ciclo de Vida	22
17.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	22
17.6. 6. Relación Exocrina (Con el resto del Ecosistema)	22
17.7. 7. Aporte al Funcionamiento Global Total	22
18.Ruta y Archivo: EcoCircuit/core/catalogo_electrico.py	23
18.1. 1. Importaciones y Dependencias Estratégicas	23
18.2. 2. Variables Globales y Metadatos Normativos	23
18.3. 3. Análisis de Clases y Funciones (Desglose Lógico)	23
18.3.1. A. Resolución de Cables y Caída de Tensión (<code>obtener_parametros_cable</code>)	23
18.3.2. B. Motor de Dimensionamiento Automático (<code>dimensionar_proteccion</code>)	23
18.3.3. C. Validación Normativa (<code>validar_cumplimiento</code>)	23
18.4. 4. El Orquestador y Ciclo de Vida	23
18.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	23
18.6. 6. Relación Exocrina (Con el resto del Ecosistema)	23
18.7. 7. Aporte al Funcionamiento Global Total	23
19.Ruta y Archivo: EcoCircuit/core/exportador_mep.py	24
19.1. 1. Importaciones y Dependencias Estratégicas	24
19.2. 2. Variables Globales y Metadatos de Estándar	24
19.3. 3. Análisis de Clases y Funciones (Desglose Matemático)	24
19.3.1. A. Mapeo Semántico Volumétrico (<code>exportar_a_ifc</code>)	24
19.3.2. B. Proyección Planimétrica Ortogonal (<code>generar_planos_dxf</code>)	24
19.3.3. C. Agregación de Inventario - BOM (<code>cuantificar_proyecto</code>)	24
19.4. 4. El Orquestador y Ciclo de Vida	24
19.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	24
19.6. 6. Relación Exocrina (Con el resto del Ecosistema)	24
19.7. 7. Aporte al Funcionamiento Global Total	24

20.Ruta y Archivo: EcoCircuit/core/migrador.py

20.1. 1. Importaciones y Dependencias Estratégicas

20.2. 2. Variables Globales y Metadatos de Versionamiento

20.3. 3. Análisis de Clases y Funciones (Desglose Lógico)

20.3.1. A. Bucle de Salto Discreto (ejecutar_migracion)

20.3.2. B. Transformaciones Geométricas y Escala (migrar_escal_aespacial)

20.3.3. C. Reparación de Grafos Huérfanos (.reparar_topologia_mep)

20.4. 4. El Orquestador y Ciclo de Vida

20.5. 5. Relación Endémica (Dentro de su Entorno Raíz)

20.6. 6. Relación Exocrina (Con el resto del Ecosistema)

20.7. 7. Aporte al Funcionamiento Global Total

21.Ruta y Archivo: EcoCircuit/core/model.py

21.1. 1. Importaciones y Dependencias Estratégicas

21.2. 2. Variables Globales y Metadatos del Dominio

21.3. 3. Análisis de Clases y Funciones (Desglose Lógico y Estructural)

21.3.1. A. Abstracción Base (EntidadBIM)

21.3.2. B. Definición de Vértices y Aristas (NodoElectrico y ConectorMEP)

21.3.3. C. El Contenedor Supremo (ProyectoBIM)

21.4. 4. El Orquestador y Ciclo de Vida

21.5. 5. Relación Endémica (Dentro de su Entorno Raíz)

21.6. 6. Relación Exocrina (Con el resto del Ecosistema)

21.7. 7. Aporte al Funcionamiento Global Total

22.Ruta y Archivo: EcoCircuit/core/motor_3d.py

22.1. 1. Importaciones y Dependencias Estratégicas

22.2. 2. Variables Globales y Metadatos Espaciales

22.3. 3. Análisis de Clases y Funciones (Desglose Matemático)

22.3.1. A. Generación Paramétrica (Sweeping & Extrusion)

22.3.2. B. Álgebra de Transformación y Cuaterniones

22.3.3. C. Detección de Colisiones (Clash Detection Engine)

22.4. 4. El Orquestador y Ciclo de Vida

22.5. 5. Relación Endémica (Dentro de su Entorno Raíz)

22.6. 6. Relación Exocrina (Con el resto del Ecosistema)

22.7. 7. Aporte al Funcionamiento Global Total

23.Ruta y Archivo: EcoCircuit/core/project_manager.py

23.1. 1. Importaciones y Dependencias Estratégicas

23.2. 2. Variables Globales y Metadatos de Sesión

23.3. 3. Análisis de Clases y Funciones (Desglose Lógico)

23.3.1. A. Instanciación y Destrucción (crear_nuevo_proyecto y cerrar_proyecto)

23.3.2. B. Enrutador de Interacciones (solicitar_guardado)

23.3.3. C. Auditoría de Metadatos (obtener_resumen_contextual)

23.4. 4. El Orquestador y Ciclo de Vida

23.5. 5. Relación Endémica (Dentro de su Entorno Raíz)

23.6. 6. Relación Exocrina (Con el resto del Ecosistema)

23.7. 7. Aporte al Funcionamiento Global Total

24.Ruta y Archivo: EcoCircuit/core/utilidades.py

24.1. 1. Importaciones y Dependencias Estratégicas

24.2. 2. Variables Globales y Metadatos de Sistema

24.3. 3. Análisis de Clases y Funciones (Desglose Matemático)

24.3.1. A. Trazabilidad Asíncrona (EcoLogger)

24.3.2. B. Transformación Dimensional Matemática (awg_a_mm2)

24.3.3. C. Perfilamiento Computacional (@medir_rendimiento)

24.4. 4. El Orquestador y Ciclo de Vida

24.5. 5. Relación Endémica (Dentro de su Entorno Raíz)

24.6. 6. Relación Exocrina (Con el resto del Ecosistema)

24.7. 7. Aporte al Funcionamiento Global Total

25.Ruta y Archivo: EcoCircuit/core/window_manager.py

25.1. 1. Importaciones y Dependencias Estratégicas

25.2. 2. Variables Globales y Metadatos de Estado

25.3. 3. Análisis de Clases y Funciones (Desglose Lógico)

25.3.1. A. Prevención de Duplicidad e Instanciación (obtener_o_crear_ventana)

25.3.2. B. Persistencia Topológica (guardar_layout y restaurar_layout)

25.3.3. C. Detección y Corrección de Fronteras (.validar_lmites_pantalla)

25.4. 4. El Orquestador y Ciclo de Vida

25.5. 5. Relación Endémica (Dentro de su Entorno Raíz)

25.6. 6. Relación Exocrina (Con el resto del Ecosistema)

25.7. 7. Aporte al Funcionamiento Global Total

26.Ruta y Archivo: EcoCircuit/simulador/motor_datos.py

26.1. 1. Importaciones y Dependencias Estratégicas

26.2. 2. Variables Globales y Metadatos Sintéticos

26.3. 3. Análisis de Clases y Funciones (Desglose Matemático)

26.3.1. A. Síntesis de Onda con Distorsión Armónica Total (THD)

26.3.2. B. Simulación Termodinámica (Inercia HVAC)

26.3.3. C. Inyección de Deriva de Sensores (Sensor Drift)

26.4. 4. El Orquestador y Ciclo de Vida

26.5. 5. Relación Endémica (Dentro de su Entorno Raíz)

26.6. 6. Relación Exocrina (Con el resto del Ecosistema)

26.7. 7. Aporte al Funcionamiento Global Total

27.Ruta y Archivo: EcoCircuit/simulador/motor_tiempo.py

27.1. 1. Importaciones y Dependencias Estratégicas

27.2. 2. Variables Globales y Metadatos de Escala

27.3. 3. Análisis de Clases y Funciones (Desglose Matemático)

27.3.1. A. Integración del Avance Temporal (actualizar_tick)

27.3.2. B. Interpolación Sub-Milisegundo (tiempo_actual_absoluto)

27.3.3. C. Máquina de Estados y Pausa Estricta (congelar_tiempo)

27.4. 4. El Orquestador y Ciclo de Vida

27.5. 5. Relación Endémica (Dentro de su Entorno Raíz)

27.6. 6. Relación Exocrina (Con el resto del Ecosistema)

27.7. 7. Aporte al Funcionamiento Global Total

28.Ruta y Archivo: EcoCircuit/simulador/panel_simulacion.py

28.1. 1. Importaciones y Dependencias Estratégicas

28.2. 2. Variables Globales y Metadatos de Interfaz

28.3. 3. Análisis de Clases y Funciones (Desglose Lógico)

28.3.1. A. Topología del Panel y Controles (.init_ui)

28.3.2. B. Máquina de Estados de Interfaz (conmutar_estado)

28.3.3. C. Bucle de Retroalimentación Sensorial (actualizar_reloj_visual)

28.4. 4. El Orquestador y Ciclo de Vida

28.5. 5. Relación Endémica (Dentro de su Entorno Raíz)

28.6. 6. Relación Exocrina (Con el resto del Ecosistema)

28.7. 7. Aporte al Funcionamiento Global Total

29.Ruta y Archivo: EcoCircuit/ui/canvas.py

29.1. 1. Importaciones y Dependencias Estratégicas

29.2. 2. Variables Globales y Metadatos de Interacción

29.3. 3. Análisis de Clases y Funciones (Desglose Matemático)

29.3.1. A. Mapeo del Espacio Afin (Zoom y Paneo)

29.3.2. B. Algoritmo de Gravedad Magnética (*Grid Snapping*)

29.3.3. C. Bucle de Captura Geométrica (Eventos de Ratón)

29.4. 4. El Orquestador y Ciclo de Vida

29.5. 5. Relación Endémica (Dentro de su Entorno Raíz)

29.6. 6. Relación Exocrina (Con el resto del Ecosistema)

29.7. 7. Aporte al Funcionamiento Global Total

30.Ruta y Archivo: EcoCircuit/ui/dialogos_electricos.py

30.1. 1. Importaciones y Dependencias Estratégicas

30.2. 2. Variables Globales y Metadatos de Validación

30.3. 3. Análisis de Clases y Funciones (Desglose Lógico)

30.3.1. A. Topología de Formularios (*DialogoConfiguracionCarga*)

30.3.2. B. Compuertas Lógicas y Validación de Estado (*accept()*)

30.3.3. C. Inyección de Catálogos (*popular_menus*)

30.4. 4. El Orquestador y Ciclo de Vida

30.5. 5. Relación Endémica (Dentro de su Entorno Raíz)

30.6. 6. Relación Exocrina (Con el resto del Ecosistema)

30.7. 7. Aporte al Funcionamiento Global Total

31.Ruta y Archivo: EcoCircuit/ui/inspector_handler.py

31.1. 1. Importaciones y Dependencias Estratégicas

31.2. 2. Variables Globales y Metadatos de Estado

31.3. 3. Análisis de Clases y Funciones (Desglose Matemático)

31.3.1. A. Reflexión Dinámica y Generación de Vistas (*_poblar_formulario*)

31.3.2. B. Rompimiento de Bucles Recursivos (*Signal Blocking*)

31.3.3. C. Validación y Escritura Atómica (*_commit_cambio*)

31.4. 4. El Orquestador y Ciclo de Vida

31.5. 5. Relación Endémica (Dentro de su Entorno Raíz)

31.6. 6. Relación Exocrina (Con el resto del Ecosistema)

31.7. 7. Aporte al Funcionamiento Global Total

32.Ruta y Archivo: EcoCircuit/ui/inspector_window.py

32.1. 1. Importaciones y Dependencias Estratégicas

32.2. 2. Variables Globales y Metadatos de Disposición

32.3. 3. Análisis de Clases y Funciones (Desglose Estructural)

32.3.1. A. Topología de Contención (*_construir_esqueleto*)

32.3.2. B. Físicas de Desprendimiento (*_manejar_flotabilidad*)

32.3.3. C. Inyección Dinámica (*montar_formulario*)

32.4. 4. El Orquestador y Ciclo de Vida

32.5. 5. Relación Endémica (Dentro de su Entorno Raíz)

32.6. 6. Relación Exocrina (Con el resto del Ecosistema)

32.7. 7. Aporte al Funcionamiento Global Total

33.Ruta y Archivo: EcoCircuit/ui/level_manager.py

33.1. 1. Importaciones y Dependencias Estratégicas

33.2. 2. Variables Globales y Metadatos de Segmentación

33.3. 3. Análisis de Clases y Funciones (Desglose Matemático)

33.3.1. A. Lógica de Apilamiento Espacial (*crear_nuevo_nivel*)

33.3.2. B. Conmutación de Contexto (Context Switching)

33.3.3. C. Destrucción en Cascada (*eliminar_nivel*)

33.4. 4. El Orquestador y Ciclo de Vida

33.5. 5. Relación Endémica (Dentro de su Entorno Raíz)

33.6. 6. Relación Exocrina (Con el resto del Ecosistema)

33.7. 7. Aporte al Funcionamiento Global Total

34.Ruta y Archivo: EcoCircuit/ui/main_window.py

34.1. 1. Importaciones y Dependencias Estratégicas

34.2. 2. Variables Globales y Metadatos de Aplicación

34.3. 3. Análisis de Clases y Funciones (Desglose Estructural)

34.3.1. A. Topología de Pantalla y Partición Espacial (*_ensamblar_layout*)

34.3.2. B. Enrutamiento de Acciones y Señales (*_conectar_senales*)

34.3.3. C. Intercepción del Protocolo de Cierre (*closeEvent*)

34.4. 4. El Orquestador y Ciclo de Vida

34.5. 5. Relación Endémica (Dentro de su Entorno Raíz)

34.6. 6. Relación Exocrina (Con el resto del Ecosistema)

34.7. 7. Aporte al Funcionamiento Global Total

35.Ruta y Archivo: EcoCircuit/ui/menu_bar.py

35.1. 1. Importaciones y Dependencias Estratégicas

35.2. 2. Variables Globales y Metadatos de Estructura

35.3. 3. Análisis de Clases y Funciones (Desglose Matemático)

35.3.1. A. Topología de Árbol de Acciones (*_construir_jerarquia*)

35.3.2. B. Mapeo de Interrupciones de Hardware (*_vincular_atajos*)

35.3.3. C. Inyección de Dependencias y Despacho (*_conectar_mediator*)

35.4. 4. El Orquestador y Ciclo de Vida

35.5. 5. Relación Endémica (Dentro de su Entorno Raíz)

35.6. 6. Relación Exocrina (Con el resto del Ecosistema)

35.7. 7. Aporte al Funcionamiento Global Total

36.Ruta y Archivo: EcoCircuit/simulador/section_manager.py

36.1. 1. Importaciones y Dependencias Estratégicas

36.2. 2. Variables Globales y Metadatos de Topología

36.3. 3. Análisis de Clases y Funciones (Desglose Matemático)

36.3.1. A. Fractura Topológica y Matriz Laplaciana (*evaluar_islas_electricas*)

36.3.2. B. Detección de Apagones (Nodos Huérfanos)

36.3.3. C. Optimización por Computación Paralela (*despachar_simulaciones*)

36.4. 4. El Orquestador y Ciclo de Vida

36.5. 5. Relación Endémica (Dentro de su Entorno Raíz)

36.6. 6. Relación Exocrina (Con el resto del Ecosistema)

36.7. 7. Aporte al Funcionamiento Global Total

37.Ruta y Archivo: EcoCircuit/ui/toolbar.py

37.1. 1. Importaciones y Dependencias Estratégicas

37.2. 2. Variables Globales y Metadatos de Estado

37.3. 3. Análisis de Clases y Funciones (Desglose Matemático)

37.3.1. A. Exclusión Mutua (*Mutually Exclusive States*)

37.3.2. B. Emisión del Cambio de Estado (*_al_cambiar_herramienta*)

37.3.3. C. Inyección de Atajos Secundarios (*Toolbar Shortcuts*)

37.4. 4. El Orquestador y Ciclo de Vida

37.5. 5. Relación Endémica (Dentro de su Entorno Raíz)

37.6. 6. Relación Exocrina (Con el resto del Ecosistema)

37.7. 7. Aporte al Funcionamiento Global Total

38.Ruta y Archivo: EcoCircuit/ui/view_3d_global.py	43
38.1. 1. Importaciones y Dependencias Estratégicas	43
38.2. 2. Variables Globales y Metadatos de Renderizado	43
38.3. 3. Análisis de Clases y Funciones (Desglose Matemático)	43
38.3.1. A. Inicialización del Contexto Gráfico y Canalización (<code>_inicializar_viewport</code>)	43
38.3.2. B. Actualización de Texturas y Mapeo Térmico (<i>Scalar Coloring</i>)	43
38.3.3. C. Detección de Selección por Raycasting (<code>_al_hacer_clic</code>)	43
38.4. 4. El Orquestador y Ciclo de Vida	43
38.5. 5. Relación Endémica (Dentro de su Entorno Raíz)	43
38.6. 6. Relación Exocrina (Con el resto del Ecosistema)	43
38.7. 7. Aporte al Funcionamiento Global Total	43

2 Archivo: main.py , RUTA: EcoCircuit/main.py (El Orquestador Maestro y Bootstrapper)

El *Entry Point* (Punto de entrada) y el **Guardián a Nivel de Kernel** de todo el ecosistema EcoCircuit Enterprise. Corresponde a la versión *Enterprise BIM v4.6* .

2.1 1. Importaciones y Dependencias Estratégicas

El bloque de importaciones está cuidadosamente seccionado para garantizar compatibilidad multiplataforma y acceso de bajo nivel:

- **Gestión del Entorno y OS:** Utiliza `sys`, `os`, `platform`, y `pathlib.Path` para operar con rutas y variables de entorno independientemente de si corre en Windows, Linux o macOS. La instrucción `os.environ["QT_API"] = "pyside6"` es una pre-configuración crítica que fuerza a cualquier librería dependiente (como `pyvistaqt` o `pyqtgraph`) a utilizar estrictamente los *bindings* de PySide6, previniendo choques de memoria y colapsos con librerías residuales de PyQt5/PyQt6.
- **Hardware y Memoria:** Importa `ctypes` para invocar librerías nativas de Windows en C (DLLs), y `QSharedMemory` de PySide6.QtCore para operaciones directas y exclusivas en la memoria RAM del sistema.
- **Interceptación de Ejecución:** Utiliza `traceback` para capturar la pila de llamadas (*call stack*) de la GPU, y `argparse` para construir una Interfaz de Línea de Comandos (CLI) robusta que permita la automatización de servidores.
- **Enlaces Internos:** Inyecta `EcoMainWindow` desde la capa `ui/` y el sistema `EcoLogger` desde la capa `core/` (incluyendo un *fallback* de seguridad a `utils/logger.py`).

2.2 2. Variables Globales y Metadatos BIM

Define de manera estática el ADN registral del software:

- `APP_NAME = EcoCircuitStudio`
- `APP_VERSION = 4.6.0`
- `APP_COMPANY = ArquitecturaBIM`

Estos datos se concatenan en la variable `APP_ID`, creando un identificador de paquete criptográficamente único que el sistema operativo utilizará para rastrear, agrupar y aislar el programa a nivel de procesos.

2.3 3. Análisis de Clases y Funciones (Desglose Lógico)

- `ensure_app_directories()`: Localiza el directorio principal del usuario usando `Path.home()` y crea una carpeta oculta `~/ecocircuitstudio`. Construye un árbol de persistencia atómica (`logs`, `config`, `dumps`, `resources`, `backups`). El uso de `exist_ok=True` previene excepciones y garantiza un terreno estable de persistencia.
- **SingleInstanceChecker (El Blindaje RAM):** Previene la corrupción por ejecución paralela. El método `is_already_running()` intenta acoplarse (`attach()`) a un bloque de `QSharedMemory` con el nombre de `APP_ID`. Si tiene éxito, otra instancia está abierta y devuelve `True`. Si falla, ejecuta `create(1)` reservando 1 byte en la RAM a nivel de SO, sellando la exclusividad.
- **SystemValidator (Auditoría de Hardware):**
 - `log_system_specs`: Inyecta en la telemetría el SO, arquitectura y versión de Python.
 - `check_screen_resolution`: Interactúa con `QApplication.primaryScreen()` midiendo la geometría del monitor; si es menor a 1280×720 , emite una alerta, protegiendo las matrices del software CAD.
- `setup_windows_identity()`: Usa `ctypes.windll.shell32.SetCurrentProcessExplicitAppUserModelID(APP_ID)` para engañar a la barra de tareas de Windows. Evita que el OS agrupe el programa bajo el genérico `python.exe`, dándole su propia identidad de memoria e icono.
- `apply_professional_theme()` (**Ingeniería UI/UX**): Inyecta el estilo "Fusion" y sobrescribe `QPalette`. Define canales RGB exactos (fondo 40, 40, 40, acento 42, 130, 218). Implementa un *fallback* de fuentes evaluando `platform.system()` (San Francisco en macOS, Roboto en Linux, Segoe UI en Windows) y forzando `PreferAntialias`.
- `global_exception_hook()` (**Tolerancia a Fallos**): Sobrescribe `sys.excepthook`. Si hay un fallo de segmentación (ej. en OpenGL), intercepta el error, evalúa si `QApplication.instance()` sigue vivo, y renderiza un `QMessageBox.Critical`. Permite al usuario hacer un “reinicio seguro” en vez de sufrir un cierre abrupto.

2.4 4. El Orquestador: Función main()

Representa el ciclo vital del software, dividido en 4 fases secuenciales:

1. **Parseo CLI:** Levanta `argparse` buscando banderas como `--no-gui` para servidores *headless*.
2. **Arranque Seguro:** Ejecuta directorios, identidad OS y reasigna el guardián de excepciones con una función lambda (`lambda t, v, tb: global_exception_hook(t, v, tb, logger)`).
3. **Contextos de GPU y Memoria:** Inyecta `Qt.ApplicationAttribute.AA_ShareOpenGLContexts`. Esto es crítico: permite que el visor 3D (PyVista) comparta texturas con el Inspector (pyqtgraph), evitando duplicar el consumo de VRAM. Activa el `SingleInstanceChecker`.
4. **Carga de Malla Visual:** Instancia `QSplashScreen`, levanta `EcoMainWindow`, redimensiona a 1400×900 y entra en pausa activa con `app.exec()`, finalizando con `sys.exit()` cuando se destruye el bucle.

2.5 5. Relación Endémica (Dentro de su Entorno Raíz)

En el directorio raíz, `main.py` opera en total sincronía con `config.py`. Es el motor de ignición que convierte los ajustes pasivos, tolerancias y constantes de `config.py` en instancias de memoria activa ejecutables para el sistema operativo.

2.6 6. Relación Exocrina (Con el resto del Ecosistema)

- **Con `ui/main_window.py`:** Le transfiere la jerarquía visual y el control absoluto del hilo principal tras la etapa del *Splash Screen*.
- **Con `core/` y `utils/`:** Enruta todas las operaciones tempranas al sistema de *logging*, dejando una huella de auditoría inmutable de cada paso del arranque del kernel.
- **Con `PyVista/OpenGL`:** Prepara el terreno de la tarjeta de video estableciendo banderas de contexto global antes de que cualquier otro módulo intente compilar Shaders o mallas tridimensionales.

2.7 7. Aporte al Funcionamiento Global Total

El aporte absoluto de `main.py` es la **estabilidad y la supervivencia del Gemelo Digital**. Aísla el frágil código Python nativo de las peculiaridades destructivas del hardware y del sistema operativo anfitrión. Garantiza que frente a excepciones matemáticas masivas o errores de memoria, el programa no colapse llevándose consigo horas de ingeniería y diseño del usuario. Actúa como la membrana protectora primigenia del ecosistema EcoCircuit.

3 Archivo: config.py (El *Single Source of Truth* Paramétrico)

En la arquitectura de EcoCircuit Enterprise, `config.py` no ejecuta ciclos iterativos ni procesa eventos; su función es actuar como el **ADN Estático y la Fuente Única de Verdad (SSOT)**. Este módulo previene la dispersión de “números mágicos” (*Magic Numbers*) a lo largo del código base, garantizando el determinismo del Gemelo Digital.

3.1 1. Importaciones y Dependencias Estratégicas

Al ser la base paramétrica del sistema, este archivo se diseña con **dependencia cero** o mínima (limitándose a módulos nativos como `os` o `pathlib`). Esta pureza arquitectónica garantiza que cualquier otro archivo del sistema pueda importar `config.py` sin riesgo de generar dependencias circulares (*Circular Imports*), previniendo el colapso del árbol de módulos en Python.

3.2 2. Variables Globales y Metadatos BIM

El archivo consolida las constantes universales del motor en diccionarios o variables de clase estáticas. Según el análisis de sus ramificaciones, define:

- **Constantes BIM Geométricas:** Define `BIM.DEFAULT.WALL.THICKNESS` (0.15m), `BIM.DEFAULT.SLAB.THICKNESS` (0.20m) y `BIM.DEFAULT.LEVEL.HEIGHT` (2.80m)[cite: 11].
- **Tolerancias Topológicas:** Define `TOLERANCIA.COLINDANCIA` (0.02m)[cite: 35].
- **Variables Estéticas y de UI:** Determina `COLOR.BG` para el entorno visual[cite: 16].
- **Rutas de Persistencia:** Especifica `LOGS.DIR` para enrutar los archivos de telemetría[cite: 32].

3.3 3. Análisis de Clases y Funciones (Lógica Paramétrica)

La lógica interna de `config.py` se basa en la **inmutabilidad** y el **acceso** $O(1)$.

- **Garantía de Tipado Estricto:** Al definir las cotas geométricas (0.15, 0.20) como flotantes nativos, se blindo el sistema contra errores de tipo (*TypeError*) cuando estos valores ingresan a las matrices de multiplicación de PyVista (C++) o NumPy.
- **Umbral de Colindancia Analítica:** El valor 0.02 de tolerancia es un límite matemático asintótico. Los procesadores de flotantes en Python (IEEE 754) generan errores de precisión en decimales altos. Esta constante actúa como una “zona de gracia” para que el `ProcesadorGeometria` detecte muros tocándose aunque matemáticamente midan $5,0000000001m$ y $4,9999999999m$.

3.4 4. El Orquestador y Ciclo de Vida

`config.py` no posee un método `main()` o un bucle. Su ciclo de vida es **Load-Time** (Tiempo de Carga). Cuando el intérprete de Python arranca y lee `main.py`, este importa `config.py` a la caché del sistema (`sys.modules`). A partir de ahí, la memoria asignada a estas variables se mantiene estática, disponible instantáneamente en RAM para cualquier controlador que la demande, sin recalcular nada.

3.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como pilar del directorio raíz, acompaña a `main.py`. Mientras `main.py` es el motor (la acción), `config.py` es la carretera (las reglas y límites). Funciona como un contrato inquebrantable para el desarrollador: si se altera una regla física del universo EcoCircuit, se hace exclusivamente aquí.

3.6 6. Relación Exocrina (Con el resto del Ecosistema)

Su red de impacto (*Dependency Graph*) toca los nervios centrales de las 5 carpetas de la arquitectura:

- **Con `ui/componentes_canvas/base.py`:** Inyecta `Config.COLOR.BG` para pintar el fondo de la matriz CAD interactiva, y `Config.BIM.DEFAULT.WALL.THICKNESS` para dictar el grosor de las líneas dibujadas por el usuario[cite: 16].
- **Con `ui/wizard.py`:** Rellena los `QDoubleSpinBox` del Asistente de Proyecto con los valores arquitectónicos por defecto, asegurando que las creaciones nuevas cumplan la norma[cite: 11].
- **Con `utils/procesador_geometria.py`:** Transfiere la variable `TOLERANCIA.COLINDANCIA` a la constante `TOLERANCIA.GLOBAL`, dictando la severidad con la que el motor buscará muros colindantes[cite: 35].
- **Con `utils/logger.py`:** Guía al motor de *logs* hacia el `LOGS.DIR`, permitiendo que las trazas de emergencia se guarden en la ruta de OS correcta[cite: 32].

3.7 7. Aporte al Funcionamiento Global Total

El aporte definitivo de `config.py` es la **Escalabilidad, Mantenibilidad y Determinismo Analítico**. Al centralizar las variables críticas, si una normativa internacional cambia (por ejemplo, exigir muros portantes de 0.20m en lugar de 0.15m), el ingeniero de software altera un único número en `config.py`. Instantáneamente, esta modificación viaja al lienzo 2D, ajusta el generador 3D de *PyVista*, altera los rangos del *Wizard* y recalibra el motor de ruteo de cables, eliminando el riesgo de discrepancias matemáticas catastróficas entre los distintos cerebros del programa.

4 Ruta y Archivo: EcoCircuit/analitica_avanzada/__init__.py

En la arquitectura de software de grado *Enterprise*, este archivo actúa como el **Guardián del Espacio de Nombres** (*Namespace*) y el marcador oficial del paquete (*Package Marker*). Es la compuerta de acceso estricto al cerebro estadístico y termodinámico del ecosistema EcoCircuit.

4.1 1. Importaciones y Dependencias Estratégicas

La función primaria de este archivo es consolidar las importaciones de sus submódulos internos para ofrecer una interfaz limpia al exterior. En lugar de ejecutar lógica pesada, importa de manera relativa los motores matemáticos clave:

- Interfaces de usuario avanzadas: `from .dashboards_ui import DashboardAnalitica`
- Motores estadísticos: `from .control_calidad import ControlEstadistico`
- Procesadores de señal: `from .procesamiento_senales import ProcesadorSenales`

Su dependencia de librerías externas es nula; su única misión es el enrutamiento y la delegación de módulos internos.

4.2 2. Variables Globales y Metadatos BIM

Implementa el Patrón de Diseño *Facade* (Fachada) mediante la restricción explícita del alcance público utilizando la variable `__all__`.

- **Encapsulamiento Duro:** Al definir `__all__ = ["DashboardAnalitica", "ControlEstadistico", "ProcesadorSenales"]`, se levanta un escudo protector en la RAM. Si un desarrollador o un script externo ejecuta la instrucción de importación masiva `from analitica_avanzada import *`, este archivo intercepta la solicitud y garantiza que **solo** se exporten las clases declaradas en la lista, bloqueando la filtración de variables locales o punteros de memoria no autorizados.

4.3 3. Análisis de Clases y Funciones (Lógica de Enrutamiento)

Dado su rol como inicializador de paquete, este código se mantiene intencionalmente libre de clases y métodos de ejecución iterativa. Su lógica es **Topológica**:

- **Validación de Rutas:** Transforma la carpeta física `analitica_avanzada/` en un Módulo Lógico Oficial reconocido por el intérprete de CPython. Sin este archivo, el árbol de directorios sería opaco para el controlador principal, resultando en un error `ModuleNotFoundError`.

4.4 4. El Orquestador y Ciclo de Vida

El ciclo de vida de este archivo ocurre en **Tiempo de Importación** (*Import-Time*). Se evalúa exactamente una vez durante toda la ejecución del programa. Cuando el `main_window.py` necesita abrir el panel de analítica, Python lee, cachea y sella este archivo en el registro `sys.modules`. Permanecerá latente en memoria, garantizando que los puentes de comunicación permanezcan abiertos.

4.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Actúa como el **Enrutador Interno** de la suite de Data Science. Conecta de manera fluida la subcarpeta `componentes_dashboard/` y `nucleo_matematico/` con los motores estadísticos. Gracias a este `__init__.py`, los módulos internos no necesitan usar rutas absolutas frágiles (ej. modificando el `sys.path`), sino que pueden referenciarse entre sí de manera relativa y segura.

4.6 6. Relación Exocrina (Con el resto del Ecosistema)

Su relación exocrina es la de una **Caja Negra** hacia la interfaz gráfica de usuario. El Controlador MVC Maestro (`ui/main_window.py`) no necesita conocer la existencia de los algoritmos de Fourier o de Control de Derivas. Simplemente invoca a este archivo inicializador para solicitar el `DashboardAnalitica`. Esta abstracción simplifica masivamente el desarrollo en las capas superiores.

4.7 7. Aporte al Funcionamiento Global Total

El aporte absoluto de `EcoCircuit/analitica_avanzada/__init__.py` al programa total es el **Desacoplamiento Estricto** (*Decoupling*).

En un simulador ciberfísico masivo, mezclar accidentalmente las dependencias de renderizado en OpenGL (BIM 3D) con las dependencias de procesamiento tensorial (NumPy/Pandas) causaría una saturación catastrófica en la memoria RAM o bloquearía el Hilo Principal (*Main Thread*). Este archivo asegura que el cerebro de Data Science se instance de manera modular e independiente, respetando el Principio de Responsabilidad Única (SRP) y blindando la escalabilidad del sistema.

5 Ruta y Archivo: EcoCircuit/analitica_avanzada/control_calidad.py

En la arquitectura del Gemelo Digital, este archivo funciona como el **Cerebro del Mantenimiento Predictivo**. Su existencia transforma a EcoCircuit de un visor pasivo CAD/BIM a un sistema SCADA proactivo. Implementa la metodología de Control Estadístico de Procesos (SPC) para evaluar series de tiempo y detectar derivas, fallos incipientes y anomalías termodinámicas en el hardware.

5.1 1. Importaciones y Dependencias Estratégicas

El archivo está blindado para el procesamiento matemático de alto rendimiento:

- **numpy (np) y pandas (pd):** Dependencias matriciales absolutas. Para evaluar 500 sensores enviando telemetría cada segundo, un bucle `for` nativo colapsaría la CPU. Se delegan las operaciones a los binarios en C de NumPy, logrando complejidad $O(1)$ en arreglos masivos.
- **scipy.stats:** Importado para el cálculo riguroso de distribuciones normales, desviaciones estándar y funciones de densidad de probabilidad.
- **core.utilidades.EcoLogger:** Utilizado para inyectar alertas críticas en el archivo `.log` del sistema cuando un motor o tablero cruza un límite de control destructivo.

5.2 2. Variables Globales y Metadatos BIM

Las variables estáticas de este archivo no son geométricas, sino **estadísticas**. Define los umbrales de sensibilidad de los algoritmos:

- Factor de suavizado EWMA ($\lambda \approx 0,2$).
- Valores de holgura de referencia (*slack values*, k) para el algoritmo CUSUM.
- Límite de la Regla Empírica de Z-Score ($|Z| > 3,0$), equivalente al 99.7 % del área bajo la curva normal, utilizado para filtrar falsos positivos.

5.3 3. Análisis de Clases y Funciones (Desglose Matemático)

La clase principal, `ControlEstadistico`, despliega tres motores algorítmicos vectorizados:

5.3.1 A. Algoritmo EWMA (Media Móvil Exponencialmente Ponderada)

A diferencia de una media simple, EWMA asigna un peso decreciente exponencial a los datos pasados, ideal para detectar **derivas graduales** (ej. desgaste de rodamientos en un HVAC).

$$E_t = \lambda Y_t + (1 - \lambda)E_{t-1} \quad (1)$$

Calcula Límites de Control Superiores (UCL) dinámicos que se adaptan a la varianza histórica:

$$UCL = \mu + L\sigma\sqrt{\frac{\lambda}{2-\lambda}[1 - (1-\lambda)^{2t}]} \quad (2)$$

Si un arreglo cruza el UCL, devuelve una máscara booleana marcando el equipo en estado de alerta.

5.3.2 B. Algoritmo CUSUM (Sumas Acumuladas Tabulares)

Extremadamente sensible a **micro-anomalías**. Mantiene dos sumas iterativas vectorizadas (límite superior S_H e inferior S_L): $S_H(i) = \max(0, S_H(i-1) + x_i - \mu - k)$
 $S_L(i) = \max(0, S_L(i-1) - x_i + \mu - k)$ Acumula desviaciones pequeñas (ej. aumento de 0,5°C por hora). Si $S_H(i)$ supera el umbral H , dispara la anomalía térmica.

5.3.3 C. Filtro Z-Score (Detección de *Outliers*)

Identifica picos transitorios (*Inrush Current* o cortocircuitos). Vectoriza la puntuación estándar:

$$Z = \frac{x - \mu}{\sigma} \quad (3)$$

Aislando los eventos que matemáticamente no pertenecen al régimen de operación estacionario.

5.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Dinámico (Run-Time)**. A diferencia de la configuración estática, este archivo es invocado repetidamente (o de forma continua a través de *streams*) durante el ciclo de simulación SCADA. Se ejecuta en lotes (*batches*) para evaluar las ventanas de tiempo recientes y actualizar el estado de salud del edificio.

5.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Opera como un procesador experto dentro de `analitica_avanzada/`:

- **Con gestor_memoria_ds.py:** Recibe de este los `DataFrames` de Pandas limpios y pre-formateados en RAM.
- **Con motor_inferencias.py:** Trabaja en tándem evaluando variables abstractas (ej. “Pérdidas Efecto Joule”) procesadas previamente por el motor de inferencias.
- **Con motor_grafico_multimodal.py:** Los arrays booleanos resultantes de las anomalías (índices `True`) son consumidos por las gráficas de `PyQtGraph` para pintar marcadores rojos sobre la curva temporal.

5.6 6. Relación Exocrina (Con el resto del Ecosistema)

Esta conexión cierra el lazo cibernético del Gemelo Digital:

- **Con el SCADA (gestor_scada.py):** Si la desviación estándar cruza el límite EWMA de forma severa, el SCADA detiene la simulación temporal o lanza una interrupción de emergencia en el hilo principal.
- **Con el Modelo 3D (visor_3d.py):** Al aislar una anomalía de carga, extrae el identificador inmutable (`UUID_BIM`) del equipo defectuoso, ordenando al motor OpenGL/VTK alterar el material PBR de esa malla para que el objeto parpadee en color de alerta en el plano espacial 3D.

5.7 7. Aporte al Funcionamiento Global Total

El aporte definitivo de este archivo es la **Inteligencia Diagnóstica Determinista**. Es la capa matemática que eleva a EcoCircuit de un mero programa de dibujo (CAD) a una plataforma avanzada de Ingeniería Ciberfísica. Convierte matrices masivas de telemetría (voltaje, potencia, temperatura) en diagnósticos humanos procesables, garantizando la trazabilidad y la justificación probabilística de cualquier posible falla electromecánica en el entorno construido.

6 Ruta y Archivo: EcoCircuit/analitica_avanzada/procesamiento_senales.py

En el ecosistema de *EcoCircuit Enterprise*, este archivo constituye el **Núcleo de Procesamiento Digital de Señales (DSP)**. Su propósito exclusivo es la traducción de información del Dominio del Tiempo al Dominio de la Frecuencia, permitiendo auditar el comportamiento termodinámico, ciclos cortos de maquinaria y armónicos en la red eléctrica del Gemelo Digital.

6.1 1. Importaciones y Dependencias Estratégicas

Su arquitectura se basa en el principio de cero dependencias gráficas. Se apoya estrictamente en librerías científicas de bajo nivel:

- **numpy (np):** Pilar fundamental. Utiliza el subpaquete `numpy.fft` para invocar implementaciones precompiladas en C del algoritmo de Cooley-Tukey, permitiendo calcular transformadas sobre cientos de miles de puntos de telemetría en milisegundos sin bloquear el Hilo Principal (*Main Thread*).

6.2 2. Variables Globales y Metadatos

Este archivo maneja constantes de conversión temporal. La más crítica es el escalar `86400.0` (segundos en un día), utilizado para mapear el espectro frecuencial desde unidades de la física fundamental (Hertz) hacia unidades operativas de la ingeniería de mantenimiento (*Ciclos Por Día* o CPD).

6.3 3. Análisis de Clases y Funciones (Desglose Matemático)

La clase central, `ProcesadorSenales`, expone el método estático `calcular_espectro_fft(t, y)`, el cual ejecuta un *pipeline* matemático en 4 fases vectorizadas:

6.3.1 A. Remoción de la Componente Continua (DC Offset)

Si se evalúa una señal directamente, la magnitud del promedio (componente 0 Hz) aplasta visualmente a las frecuencias sutiles. El algoritmo centra la señal en cero restando la media aritmética:

$$y'(t) = y(t) - \bar{y} \tag{4}$$

6.3.2 B. Enventanado (Hanning Window)

Para mitigar las fugas espectrales (*Spectral Leakage*) generadas al procesar señales que no son perfectamente periódicas respecto a la ventana de observación, el módulo aplica una Ventana de Hanning antes de la transformada:

$$w(n) = 0,5 \left[1 - \cos\left(\frac{2\pi n}{N-1}\right) \right] \tag{5}$$

El vector resultante es la multiplicación elemento a elemento: $y''(n) = y'(n) \cdot w(n)$.

6.3.3 C. Transformada Rápida de Fourier (FFT)

Se ejecuta el algoritmo FFT sobre el vector enventanado:

$$Y(k) = \sum_{n=0}^{N-1} y''(n) e^{-j2\pi kn/N} \tag{6}$$

Posteriormente, debido a la simetría hermitiana de la FFT para señales reales, el código recorta y extrae solo la mitad positiva del espectro, normalizando las amplitudes multiplicando el vector por $2/N$.

6.3.4 D. Conversión de Escala Frecuencial

Calcula el intervalo de muestreo promedio $\Delta t = mean(diff(t))$. Con base en el teorema de Nyquist, genera el vector de frecuencias en Hertz, y finalmente lo escala a métricas de edificio:

$$f_{CPD} = f_{Hz} \times 86400 \tag{7}$$

6.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Demanda Bajo Interacción (*On-Demand Run-Time*)**. No está corriendo perpetuamente; solo se activa cuando el usuario interactúa con la interfaz (por ejemplo, arrastrando el cursor para seleccionar un rango de 48 horas en el histórico de un motor HVAC). En ese instante, recibe la matriz de datos, aplica la FFT y devuelve las tuplas en fracciones de segundo.

6.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Opera en estricta coordinación con el resto de la carpeta `analitica_avanzada/`:

- Con `gestor_memoria_ds.py`: Este le provee los vectores puros `t` (timestamps) y `y` (amplitudes) desde la RAM.
- Con `panel_fft.py` (**dentro de `componentes_dashboard/`**): Es invocado directamente por el panel especializado en análisis frecuencial.

6.6 6. Relación Exocrina (Con el resto del Ecosistema)

Mantiene un aislamiento total del motor OpenGL (PyVista) y de la matriz 2D (PySide6). Su salida —un par de tensores unidimensionales con las frecuencias y amplitudes— es entregada a `motor_grafico_multimodal.py`, el cual utiliza `pyqtgraph` para renderizar el periodograma (espectro de potencias) que el ingeniero visualizará en la pantalla.

6.7 7. Aporte al Funcionamiento Global Total

Su aporte es la **Auditoría Ciberfísica Avanzada**. Habilita la capacidad de descubrir fenómenos periódicos invisibles a simple vista en el dominio del tiempo. Gracias a este archivo, el SCADA puede alertar si un aire acondicionado sufre de "ciclos cortos" (encendidos destructivos de 5 minutos repetidos 12 veces por hora) o si la red eléctrica principal presenta armónicos severos que sobrecalentarán los transformadores del Gemelo Digital.

7 Ruta y Archivo: EcoCircuit/analitica_avanzada/dashboards_ui.py

En la jerarquía del ecosistema, este archivo actúa como el **Orquestador Visual (Frontend) del Motor Analítico**. Es el contenedor maestro que unifica los paneles de series de tiempo, espectros de Fourier y control de calidad en una sola ventana interactiva, aislando la complejidad matemática detrás de una interfaz gráfica de usuario (GUI) fluida y reactiva.

7.1 1. Importaciones y Dependencias Estratégicas

El archivo maneja un puente crítico entre el entorno gráfico y el entorno de datos:

- **PySide6.QtWidgets, QtCore, QtGui:** Dependencias absolutas para la construcción del árbol de *Widgets*, manejo de *Signals/Slots* asíncronos y layouts dinámicos (**QSplitter, QTabWidget**).
- **Submódulos de Componentes (componentes_dashboard/):** Importa **PanelSeriesTiempo**, **PanelFFT** y **PanelSPC** para instanciarlos como hijos dentro del lienzo principal.
- **Gestor de Memoria:** Importa el **GestorMemoriaDS** para tener un canal directo a los *DataFrames* almacenados en la RAM.

7.2 2. Variables Globales y Metadatos de UI

Define constantes estructurales y de temporización para garantizar un rendimiento no bloqueante:

- **REFRESH_RATE_MS = 1000:** Dicta la frecuencia de actualización del *Event Loop* para la ingesta de telemetría en vivo, limitando los redibujados para no asfixiar a la CPU.
- Estilos QSS (Qt Style Sheets) dinámicos para mantener coherencia visual con la paleta *Dark CAD* definida en **config.py** y **nucleo_estetico.py**.

7.3 3. Análisis de Clases y Funciones (Desglose Lógico)

La clase principal, **DashboardAnalitica(QMainWindow)** (o **QWidget** dependiendo de su acople), implementa la arquitectura Modelo-Vista-Controlador (MVC) a nivel local:

7.3.1 A. Construcción del Layout (.init_ui)

Utiliza instancias de **QSplitter** para dividir la pantalla matemáticamente. Asigna un porcentaje fijo del área visible a la Serie de Tiempo (ej. 50%), a la Transformada de Fourier (25%) y al Control Estadístico (25%). Esto permite que el usuario redimensione los paneles sin romper la cuadrícula matricial de las gráficas.

7.3.2 B. Sistema de Reactividad (.conectar_senales)

Aplica el patrón *Observer* mediante *Signals* y *Slots* de PySide6.

- **Ejemplo de Flujo:** Si el usuario hace zoom o selecciona una ventana de tiempo específica en el **PanelSeriesTiempo**, este emite una señal **rango_modificado_sig(t_start, t_end)**.
- El Dashboard intercepta esta señal y ejecuta un *Slot* que obliga al **PanelFFT** a recalcular la Transformada Rápida de Fourier **exclusivamente** para ese rango de tiempo, logrando una interactividad bidireccional perfecta.

7.3.3 C. Inyección de Contexto (actualizar_datos)

Recibe el **UUID_BIM** del nodo seleccionado. Consulta al **GestorMemoriaDS** recuperando la matriz de datos en tiempo $O(1)$ y distribuye los vectores (arrays de NumPy) a los paneles hijos invocando sus métodos de renderizado respectivos.

7.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida está **Impulsado por Eventos (*Event-Driven*)**. Inicialmente, el dashboard se carga en memoria en estado latente (oculto). Se despierta y entra al *Main Thread* de ejecución visual únicamente cuando el controlador principal lo invoca (por ejemplo, mediante la acción “Analizar Nodo” en el lienzo 2D o 3D). Al cerrarse, no destruye su memoria, sino que usa **hide()** para conservar el estado y reaparecer instantáneamente en la próxima llamada.

7.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Es el **Padre Topológico** de la carpeta **analitica_avanzada/**.

- Delega el dibujo pesado al **motor_grafico_multimodal.py**.
- Orquesta a los paneles individuales (**PanelFFT, PanelSPC**), los cuales actúan como esclavos visuales que solo obedecen las órdenes de actualización emitidas por **dashboards_ui.py**.

7.6 6. Relación Exocrina (Con el resto del Ecosistema)

Actúa como el puente de comunicación entre el **Mundo Gráfico CAD** y el **Mundo de Data Science**.

- **Con ui/main_window.py y visor_3d.py:** Cuando el usuario hace clic derecho sobre el modelo 3D de un tablero eléctrico, la ventana principal le envía a este archivo el identificador único del equipo.
- Este archivo traduce esa selección geométrica en una consulta de bases de datos, cerrando la brecha entre la espacialidad (arquitectura) y el comportamiento (telemetría).

7.7 7. Aporte al Funcionamiento Global Total

Su aporte es la **Traducción Cognitiva (*Human-Machine Interface*)**. Los tensores procesados por NumPy y las ecuaciones de control de calidad son inútiles si el ingeniero no puede interpretarlos. Este archivo toma millones de puntos de datos procesados por la RAM y los convierte en curvas, histogramas y alertas de colores legibles. Permite al usuario tomar decisiones críticas de ingeniería y mantenimiento predictivo sin necesidad de escribir o entender una sola línea de código Python.

8 Ruta y Archivo: EcoCircuit/analitica_avanzada/componentes_dashboard/__init__.py

Dentro de la topología fractal de *EcoCircuit Enterprise*, este archivo funciona como el **Gestor de Sub-Espacio de Nombres** (***Sub-Namespace Manager***). Su propósito fundamental es agrupar y encapsular la colección de *Widgets* especializados y motores gráficos de PySide6/PyQtGraph, creando una Interfaz de Programación de Aplicaciones (API) interna limpia para el orquestador visual padre.

8.1 1. Importaciones y Dependencias Estratégicas

Como marcador de subpaquete, su bloque de importaciones está diseñado para resolver las rutas relativas de los componentes visuales e instanciadores de memoria que residen en su mismo nivel de directorio:

- Motor de renderizado: `from .motor_grafico_multimodal import MotorGrafico`
- Tubería de datos: `from .gestor_memoria_ds import GestorMemoriaDS`
- Paneles visuales modulares: `from .panel_series_tiempo import PanelSeriesTiempo, from .panel_fft import PanelFFT, from .panel_spc import PanelSPC`

No importa librerías externas de terceros (como PySide6 o numpy), ya que su labor es puramente de agregación estructural.

8.2 2. Variables Globales y Metadatos de Encapsulamiento

Al igual que su homólogo en la carpeta superior, implementa la variable `__all__` para imponer un **Contrato de Exportación Estricto**.

- Define `__all__ = ["MotorGrafico", "GestorMemoriaDS", "PanelSeriesTiempo", "PanelFFT", "PanelSPC"]`.
- Este metadato asegura que si un archivo superior utiliza comodines de importación, no se filtren clases abstractas, variables temporales o utilidades de bajo nivel que puedan existir dentro de los archivos de los paneles.

8.3 3. Análisis de Clases y Funciones (Lógica Topológica)

El archivo carece de ejecución procedimental o matemática. Su lógica es **Resolutiva y Estructural**:

- **Indexación de Módulos:** Obliga al intérprete de Python a tratar el directorio físico `componentes_dashboard/` como un paquete importable anidado.
- **Aplanamiento de Jerarquías:** Permite que módulos externos invoquen a los paneles directamente desde la carpeta contenedora, en lugar de tener que conocer el nombre exacto del archivo `.py` que los alberga, abstrayendo la complejidad del sistema de archivos.

8.4 4. El Orquestador y Ciclo de Vida

Se ejecuta en el **Tiempo de Importación** (***Import-Time***). Su ciclo de vida inicia en el momento exacto en que `dashboards_ui.py` es cargado en memoria. Tras resolver las importaciones relativas y registrar los objetos en el caché del sistema (`sys.modules`), el archivo queda latente. Su coste computacional en *run-time* (tiempo de ejecución) es absolutamente nulo ($O(1)$ constante).

8.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Es el **Pegamento Arquitectónico** de los componentes visuales:

- Asegura que los distintos paneles (`panel_fft`, `panel_spc`) y el motor gráfico multimodal coexistan bajo un mismo paraguas semántico.
- Facilita que, si un componente necesita interactuar con otro (por ejemplo, un panel enviando una señal al `gestor_memoria_ds`), puedan hacerlo sabiendo que pertenecen al mismo sub-espacio de nombres validado.

8.6 6. Relación Exocrina (Con el resto del Ecosistema)

Su función exocrina principal es exponer una **API Limpia y Unificada** hacia la capa superior (`dashboards_ui.py`). Gracias a este archivo, el código del *Dashboard* orquestador se mantiene elegante y conciso, pudiendo importar múltiples componentes complejos en una sola línea de código, sin preocuparse por la topología interna o si un panel fue refactorizado en múltiples submódulos.

8.7 7. Aporte al Funcionamiento Global Total

Su mayor aporte es la **Modularidad y la Prevención de Colisiones de Nombres** (***Namespace Clashes***). Al encapsular los módulos de interfaz gráfica específicos de Data Science, garantiza que una clase llamada, por ejemplo, `BasePanel` dentro de los componentes del dashboard no colisione en RAM con una clase `BasePanel` que pudiera existir en el inspector BIM 3D (`ui/inspector/`). Mantiene la base de código *Enterprise* saneada, organizada y lista para escalar sin efectos secundarios destructivos.

9 Ruta y Archivo: EcoCircuit/analitica_avanzada/componentes_dashboard/motor_grafico_multimodal.py

En la capa de visualización analítica del Gemelo Digital, este archivo funciona como la **GPU Matemática del Dashboard**. Sustituye deliberadamente a librerías tradicionales como **Matplotlib** en favor de motores basados en QtGraphics para garantizar que el renderizado de millones de puntos de telemetría (series temporales, armónicos, dispersión) se ejecute sin congelar el Hilo Principal de la aplicación.

9.1 1. Importaciones y Dependencias Estratégicas

El archivo importa herramientas de aceleración gráfica y cálculo vectorial:

- **pyqtgraph (pg)**: La dependencia más crítica. Utilizada por su capacidad de vectorización nativa en C++. Proporciona objetos como `PlotWidget`, `PlotCurveItem` y `ScatterPlotItem`.
- **numpy (np)**: Ingerida para el manejo de arreglos de coordenadas (X, Y) , transformaciones logarítmicas de ejes y filtrado rápido mediante máscaras booleanas.
- **PySide6.QtGui y QtWidgets**: Empleadas para inyectar los *Widgets* de pyqtgraph dentro de la jerarquía visual de EcoCircuit y aplicar `QPen` o `QBrush` a los trazos.

9.2 2. Variables Globales y Metadatos Gráficos

Aplica configuraciones paramétricas a nivel de módulo antes de instanciar clases:

- `pg.setConfigOptions(antialias=True)`: Fuerza el suavizado espacial de las curvas geométricas para evitar bordes dentados (aliasing) en pantallas de alta resolución.
- `pg.setConfigOption('background', 'k')` y `('foreground', 'd')`: Mapea el entorno gráfico al estándar *Dark CAD* del sistema, utilizando matrices RGB coherentes con `config.py`.

9.3 3. Análisis de Clases y Funciones (Desglose Lógico)

La clase principal, `MotorGraficoMultimodal`, orquesta la topología de la vista.

9.3.1 A. Renderizado de Series Temporales (`graficar_curva`)

Recibe arreglos unidimensionales t (tiempo) e y (magnitud). Utiliza `PlotDataItem` para dibujar la línea base. Implementa decimación inteligente (*downsampling*): si el número de puntos excede la resolución en píxeles del monitor, el motor colapsa los datos (mostrando solo el min/max por píxel) para mantener los cuadros por segundo (FPS) en 60.

9.3.2 B. Renderizado Multimodal (Áreas e Histogramas)

- **Análisis Espectral**: Si detecta que los datos provienen de `procesamiento_senales.py`, invoca `FillBetweenItem` para rellenar el área bajo la curva del periodograma (FFT), resaltando la energía de las frecuencias.
- **Distribución Estadística**: Utiliza `BarGraphItem` invocando internamente a `np.histogram(datos, bins='auto')` para construir de manera dinámica la función de densidad de probabilidad de los sensores.

9.3.3 C. Inyección de Anomalías y Límites (`marcar_anomalias`)

Este método es el traductor de la estadística de `control_calidad.py`:

- **Umbrales Estáticos y Dinámicos**: Instancia `InfiniteLine` para trazar las fronteras asintóticas (UCL/LCL).
- **Marcadores de Evento**: Aplica un `ScatterPlotItem` de alta prioridad visual (Z-Index alto) que lee el arreglo booleano de anomalías, plotando círculos rojos (●) exclusivamente en las coordenadas (t, y) donde el algoritmo EWMA o CUSUM detectó un fallo termodinámico.

9.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es de **Dibujo Reactivo (Reactive Rendering)**. El motor no tiene un bucle de refresco propio (*polling*). Espera pasivamente. Cuando un panel padre le entrega una nueva tupla de tensores (X, Y) , el motor limpia el *buffer* de la gráfica (`plot.clear()`), redibuja las curvas en memoria de video (VRAM) y devuelve el control al orquestador en una fracción de milisegundo.

9.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Es el **Esclavo Visual** de los paneles especializados dentro de `componentes_dashboard/`:

- El `PanelSeriesTiempo`, `PanelFFT` y `PanelSPC` no saben cómo dibujar una línea en la pantalla. Ellos instancian este `MotorGraficoMultimodal` como una propiedad (`self.lienzo_grafico`) y le delegan el 100% del renderizado pesado, respetando el Principio de Responsabilidad Única.

9.6 6. Relación Exocrina (Con el resto del Ecosistema)

Su interacción con el exterior es indirecta pero vital:

- Muestra físicamente en la pantalla de la GUI el resultado de las matemáticas procesadas por `motor_inferencias.py` y `procesamiento_senales.py`.
- A diferencia del motor OpenGL (PyVista) en `utils/` que renderiza tuberías y muros 3D, este motor se especializa en **planos cartesianos 2D matemáticos**, asegurando que el Gemelo Digital separe la visualización espacial de la telemetría abstracta.

9.7 7. Aporte al Funcionamiento Global Total

El aporte fundamental de este módulo es la **Alta Disponibilidad y Fluidez de la Interfaz (Anti-Freezing)**. Si se utilizara Matplotlib para graficar 100,000 puntos de telemetría provenientes de los tableros eléctricos, el hilo principal de Python se bloquearía durante 2-5 segundos, congelando toda la aplicación EcoCircuit. Gracias a la implementación de `pyqtgraph` en este archivo, el sistema es capaz de hacer *pan*, *zoom* y *scroll* en tiempo real sobre matrices masivas de datos SCADA de manera fluida, proveyendo al usuario una experiencia verdaderamente *Enterprise*.

10 Ruta y Archivo: EcoCircuit/analitica_avanzada/componentes_dashboard/gestor_memoria_ds.py

Dentro del Ecosistema *EcoCircuit Enterprise*, este archivo actúa como el **Pipeline de Ingestión y Administrador de Memoria RAM**. Es el puente vital entre el almacenamiento en disco (telemetría cruda) y los motores matemáticos en tiempo real. Su existencia es crucial para evitar que la carga de bases de datos históricas masivas provoque un error de saturación de memoria (*Out-Of-Memory / OOM*) en el Hilo Principal.

10.1 1. Importaciones y Dependencias Estratégicas

Su arsenal está diseñado para la manipulación ultra-rápida de tensores y bases de datos estructuradas:

- **pandas (pd):** La dependencia maestra de este archivo. Utilizada para construir y gestionar *DataFrames*, permitiendo índices temporales (**DatetimeIndex**) y alineación automática de datos faltantes (*NaNs*).
- **numpy (np):** Utilizado para preasignar bloques de memoria continua en RAM y realizar operaciones de álgebra vectorial en $O(1)$.
- **copy:** Importado para implementar la política estricta de *Zero-Leak* (cero fugas de punteros) mediante `copy.deepcopy()`, asegurando el aislamiento entre la base maestra y las vistas de usuario.

10.2 2. Variables Globales y Metadatos de Caché

Define políticas de retención y limpieza de memoria:

- **MAX_CACHE_SIZE:** Límite superior de registros mantenidos en RAM activa antes de aplicar políticas de desalojo (*Eviction*) como LRU (*Least Recently Used*).
- **CHUNK_SIZE:** Parámetro crítico para la lectura de archivos CSV históricos pesados. En lugar de cargar un archivo de 5GB de un golpe, lo lee en fragmentos (chunks) de 100,000 filas, procesándolos de manera incremental.

10.3 3. Análisis de Clases y Funciones (Desglose Lógico)

La clase principal, **GestorMemoriaDS**, opera como un *Singleton* o instancia centralizada de base de datos volátil:

10.3.1 A. Ingestión y Parseo (cargar telemetria)

Lee series temporales. Utiliza la función `pd.to_datetime()` con el flag `infer_datetime_format=True` (o un formato predefinido) para acelerar drásticamente la conversión de *strings* a *timestamps*. Inmediatamente, establece esta columna de tiempo como el Índice del DataFrame, transformándolo en una matriz de búsqueda rápida por rangos de fechas.

10.3.2 B. Vectorización e Ingeniería de Características (calcular variables derivadas)

En lugar de iterar con un bucle `for` (lo cual en Python es asintóticamente lento), ejecuta operaciones matriciales.

$$P_{activa} = V_{rms} \times I_{rms} \times \cos(\phi) \quad (8)$$

El método aplica esta ecuación multiplicando directamente las columnas enteras del DataFrame (`df['V'] * df['I'] * df['fp']`), delegando el cálculo matemático al procesador en lenguaje C subyacente de Pandas/NumPy.

10.3.3 C. Slicing y Recuperación Rápida (obtener ventana temporal)

Para inyectar datos a las gráficas, el gestor utiliza el buscador binario de Pandas (`df.loc[t_inicio : t_fin]`). Como el índice está ordenado cronológicamente desde la ingesta, el tiempo de búsqueda pasa de $O(N)$ a $O(\log N)$, recuperando semanas de telemetría en microsegundos. Para respetar el aislamiento de memoria, el método devuelve un `df.copy()`, protegiendo el conjunto maestro.

10.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Residente en Memoria (*Long-Lived/Resident*)**. Es instanciado al inicio por el orquestador principal del Dashboard (`dashboards.ui.py`) y se mantiene vivo en *background*. A lo largo de la ejecución del programa, va acumulando nuevos arreglos de datos provenientes del SCADA y los consolida (`pd.concat`) con los datos históricos existentes.

10.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Es el **Proveedor Maestro** dentro de `componentes_dashboard/`:

- Los paneles de visualización (**PanelSeriesTiempo**, **PanelFFT**) no tienen permiso para leer archivos físicos de la computadora. Están obligados a solicitarle la información al **GestorMemoriaDS**.
- Garantiza que si dos paneles distintos están evaluando el mismo motor HVAC, ambos consuman exactamente el mismo bloque de RAM, optimizando el uso de los recursos computacionales.

10.6 6. Relación Exocrina (Con el resto del Ecosistema)

Actúa como la aduana de entrada de datos hacia la sub-arquitectura analítica:

- **Con el Controlador SCADA (controladores/gestor_scada.py):** Recibe los paquetes crudos provenientes de los sensores virtuales o reales del Gemelo Digital y los normaliza.
- **Con el Núcleo Matemático (motor_inferencias.py):** Le envía los vectores puros y estructurados para que este último aplique las leyes termodinámicas y le regrese los sensores virtuales procesados.

10.7 7. Aporte al Funcionamiento Global Total

El aporte de `gestor_memoria_ds.py` es la **Integridad de Datos y Prevención de Fugas de Memoria**. Sin este gestor, el software tendría que abrir y leer archivos de disco en cada interacción del usuario con las gráficas, lo que resultaría en una latencia de entrada/salida (I/O) paralizante. Gracias a su orquestación y vectorización mediante Pandas, el EcoCircuit adquiere una robustez equiparable a servidores Big Data, procesando millones de puntos en tiempo real sin degradar el rendimiento gráfico del modelo BIM 3D.

11 Ruta y Archivo: EcoCircuit/analitica_avanzada/componentes_dashboard/panel_series_tiempo.py

En la capa de visualización de *EcoCircuit Enterprise*, este archivo actúa como el **Monitor de Signos Vitales (Dominio del Tiempo)**. Es el componente de interfaz gráfica principal que permite al ingeniero interactuar físicamente, mediante *zoom* y *pan*, con el flujo cronológico de la telemetría electromecánica del Gemelo Digital.

11.1 1. Importaciones y Dependencias Estratégicas

La construcción de este panel se apoya en librerías de interfaz y delegación gráfica:

- **PySide6.QtWidgets, QtCore:** Dependencias absolutas para heredar de **QWidget**, construir el *layout* vertical (**QVBoxLayout**) y habilitar el sistema de comunicación asíncrona nativo de Qt (**Signal** y **Slot**).
- **pyqtgraph (pg):** Importado para integrar las herramientas de control de cámara (ejes de coordenadas interactivos) y las señales de región lineal (**LinearRegionItem**).
- **motor_grafico_multimodal.py:** Inyectado para delegar la responsabilidad matemática del trazado en lugar de intentar dibujar píxeles manualmente.

11.2 2. Variables Globales y Metadatos de Estado

El panel maneja el estado de la vista mediante variables internas (*State Machine* visual):

- **variable_activa:** Almacena un *string* (ej. "Potencia_Activa" o Corriente"), dictando qué columna del *DataFrame* maestro se está renderizando actualmente.
- **rango_temporal_actual:** Tupla (t_0, t_1) que guarda los límites en formato de marca de tiempo (UNIX o *Datetime*) visibles en el eje X, vital para la sincronización inter-paneles.

11.3 3. Análisis de Clases y Funciones (Desglose Lógico)

La clase **PanelSeriesTiempo(QWidget)** consolida las interacciones del usuario y la topología visual:

11.3.1 A. Topología de UI (`_init_ui`)

Construye un contenedor que incluye una barra de herramientas superior (**QToolBar** o **QHBoxLayout**) con un **QComboBox** para seleccionar la métrica a visualizar, y reserva el 90 % del área restante para instanciar el **MotorGraficoMultimodal**.

11.3.2 B. Emisión de Eventos de Región (`_on_region_changed`)

Integra un componente avanzado llamado **LinearRegionItem** (un selector de ventana arrastrable sobre la gráfica).

- **Matemática de Selección:** Cuando el usuario arrastra los límites de esta región, el método lee los nuevos valores X_{min} y X_{max} .
- **Señalización:** Dispara `self.rango_modificado_sig.emit(x_min, x_max)`. Esta señal es la que notifica al orquestador padre que la ventana temporal de análisis ha cambiado.

11.3.3 C. Actualización de Datos (`actualizar_grafica`)

Recibe la matriz de datos (t, y) cortada. Este método no calcula derivadas ni estadísticas; simplemente invoca a `self.motor_grafico.graficar_curva(t, y)`, aplicando un formato legible para el eje de tiempo (*TimeAxisItem*) para que el usuario lea "14:30:00.^{en}" lugar de un flotante crudo como "1627409400.0".

11.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Perpetuo-Interactivo (*Event-Driven Persistent*)**. Es instanciado por `dashboards_ui.py` al abrir la ventana analítica. A partir de ese momento, entra en un estado de escucha pasiva (*Event Loop*). Solo consume CPU cuando el usuario hace *scroll* con la rueda del ratón (escalando matrices) o cuando el menú desplegable exige cargar una nueva métrica.

11.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Actúa como el **Controlador Maestro de Interacción** dentro del sub-espacio `componentes_dashboard/`:

- **Con `motor_grafico_multimodal.py`:** Le transfiere la matriz de datos cruda y las directrices estéticas (colores, grosores), actuando como su jefe directo.
- **Con `PanelFFT` y `PanelSPC` (Vía el Padre):** Al emitir `rango_modificado_sig`, obliga indirectamente a los demás paneles a recalcular la Transformada de Fourier o los Límites de Control **solamente** para la franja de tiempo que el usuario seleccionó en la gráfica de este panel. Es el dictador del contexto temporal.

11.6 6. Relación Exocrina (Con el resto del Ecosistema)

Su relación exocrina es bidireccional, enfocada en la Experiencia de Usuario (UX):

- Muestra la realidad física (consumos, caídas de tensión) modelada originalmente en la malla espacial por el usuario en el **Canvas 2D**.
- Convierte los datos que el `gestor_scada.py` recolectó en el *backend* en algo tangible, respondiendo a la selección de elementos BIM específicos mediante sus **UUID**.

11.7 7. Aporte al Funcionamiento Global Total

El aporte absoluto de `panel_series_tiempo.py` es la **Exploración Forense e Interactividad Humana**. Mientras que los motores de Data Science procesan números en frío, este archivo permite a un ingeniero humano detectar visualmente el instante exacto (hora, minuto y segundo) en que un sistema eléctrico colapsó, correlacionando visualmente un pico de corriente masivo con un evento de la simulación SCADA. Sin este panel, el Gemelo Digital sería una base de datos opaca e ininterpretable.

12 Ruta y Archivo: EcoCircuit/analitica_avanzada/componentes_dashboard/panel_fft.py

Dentro de la suite analítica de *EcoCircuit Enterprise*, este archivo opera como el **Espectrómetro del Gemelo Digital (Dominio de la Frecuencia)**. Su función es revelar los patrones cíclicos, armónicos eléctricos y vibraciones termodinámicas que se encuentran ocultos en el ruido blanco de las series temporales estándar.

12.1 1. Importaciones y Dependencias Estratégicas

Este panel requiere una amalgama de librerías de interfaz y delegación matemática:

- `PySide6.QtWidgets, QtCore`: Empleadas para heredar de `QWidget`, construir su *layout* (como los controles de escala logarítmica) y recibir las señales del orquestador padre.
- `analitica_avanzada.procesamiento_senales`: Importa `ProcesadorSenales`. A diferencia del panel de series de tiempo que grafica datos crudos, este panel necesita preprocesar las matrices pasando la información por la Transformada Rápida de Fourier.
- `motor_grafico_multimodal.py`: Invocado para instanciar su propio lienzo de renderizado de alto rendimiento utilizando internamente `pyqtgraph`.

12.2 2. Variables Globales y Metadatos de Estado

Gestiona el estado visual del dominio frecuencial:

- `escala_logaritmica_y`: Un indicador booleano (*flag*). Las amplitudes frecuenciales (*Power Spectral Density*) a menudo varían en varios órdenes de magnitud, por lo que el estado logarítmico $\log_{10}(A)$ es vital para visualizar tanto los armónicos masivos como los micro-ciclos simultáneamente.
- Unidades del eje X: Alterna entre *Hertz (Hz)* para eventos puramente eléctricos (como ruido en inversores a 60Hz) y *Ciclos Por Día (CPD)* para eventos mecánicos (como ciclos de encendido de un compresor HVAC).

12.3 3. Análisis de Clases y Funciones (Desglose Lógico)

La clase `PanelFFT(QWidget)` actúa como un puente entre la matemática pura y la representación visual:

12.3.1 A. Topología de UI y Ejes (`_init_ui`)

Construye el lienzo a través del `MotorGraficoMultimodal`. Configura los ejes cartesianos de manera estricta: restringe el eje X para que solo muestre valores positivos (ya que el espectro FFT recortado no tiene frecuencias negativas), previniendo que el usuario haga un *pan* hacia un espacio matemático inexistente.

12.3.2 B. Traducción Frecuencial (`actualizar_fft`)

Este es el núcleo de ejecución. Recibe los vectores del tiempo (*t*) y la magnitud (*y*) cortados por el panel temporal.

1. **Invocación Matemática**: Llama a `ProcesadorSenales.calcular_espectro_fft(t, y)`.
2. **Recepción de Tensores**: Recibe de vuelta la tupla (*f, A*) (frecuencias y amplitudes normalizadas).
3. **Inyección Visual**: Envía (*f, A*) a `self.motor_grafico.graficar_curva()`. Activa parámetros específicos del motor multimodal para rellenar el área bajo la curva, transformando una simple línea en un *Periodograma* sólido que resalta la densidad energética de los armónicos.

12.3.3 C. Detección Visual de Armónicos Dominantes (Picos)

Emplea la matemática de filtrado para encontrar las frecuencias con mayor amplitud ($\max(A)$). A través del `MotorGraficoMultimodal`, inyecta etiquetas de texto (`TextItem`) directamente sobre los picos dominantes, permitiendo que el usuario lea "24 CPD" instantáneamente, sin tener que adivinar el valor en el eje.

12.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Estrictamente Reactivo (*Strictly Reactive*)**. A diferencia del panel temporal que escucha directamente los clics del mouse, el `PanelFFT` es ciego a la selección del usuario. Se actualiza **únicamente** cuando el `dashboards_ui.py` (su padre) le dispara una señal de actualización con nuevos datos en respuesta a un cambio de rango temporal ocurrido en la ventana superior.

12.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Es el componente más sofisticado intelectualmente del `componentes_dashboard/`:

- Es el cliente principal de `procesamiento_senales.py`.
- Actúa como el **hermano simbiótico** del `PanelSeriesTiempo`. Mientras el panel temporal recorta el "dónde y cuándo", el `PanelFFT` traduce ese recorte al "por qué y cómo".

12.6 6. Relación Exocrina (Con el resto del Ecosistema)

No tiene contacto directo con la geometría 3D o el SCADA. Sin embargo, su impacto exocrino es diagnóstico: justifica las alarmas emitidas por otros sistemas. Si un componente electromecánico en el Gemelo Digital comienza a vibrar y a perder eficiencia, este panel es la herramienta final que demuestra la existencia de una oscilación destructiva en la red, validando el comportamiento simulado en `motor.inferencias.py`.

12.7 7. Aporte al Funcionamiento Global Total

El aporte absoluto de `panel_fft.py` es la capacidad de **Mantenimiento Predictivo Forense (*Root Cause Analysis*)**.

Sin este panel, un equipo de aire acondicionado encendiéndose y apagándose cada 5 minutos (ciclos cortos) parecería simplemente un bloque denso de líneas erráticas en una gráfica de tiempo. Gracias a este módulo, ese mismo comportamiento se revela como un pico masivo claro en la coordenada matemática de 288 CPD (Ciclos Por Día), permitiendo al ingeniero identificar inmediatamente que el equipo está sobredimensionado o el termostato está fallando, cumpliendo con el propósito máximo del software de grado *Enterprise*.

13 Ruta y Archivo: EcoCircuit/analitica_avanzada/componentes_dashboard/panel_spc.py

En la jerarquía visual del Gemelo Digital, este archivo funciona como el **Monitor de Control Estadístico de Procesos (SPC - Statistical Process Control)**. Es el componente terminal que cristaliza la complejidad matemática de la estadística bayesiana y los modelos predictivos en alertas visuales inequívocas, permitiendo la toma de decisiones críticas sobre la infraestructura electromecánica.

13.1 1. Importaciones y Dependencias Estratégicas

El panel se construye sobre una triada de librerías para vincular interfaz, cálculo y renderizado:

- **PySide6.QtWidgets, QtCore:** Requeridas para estructurar el panel de control (**QRadioButton, QGroupBox**) que permite al ingeniero conmutar entre los distintos algoritmos de control de calidad.
- **analitica_avanzada.control_calidad:** Importa el motor **ControlEstadistico**. A diferencia del panel de series de tiempo simple, este módulo no dibuja nada hasta que los datos hayan sido auditados matemáticamente.
- **motor_grafico_multimodal.py:** Inyectado para delegar la carga de renderizado vectorial de las nubes de dispersión y las líneas asintóticas de control.

13.2 2. Variables Globales y Metadatos de Estado

El panel maneja máquinas de estado para la configuración de umbrales:

- **modo_algoritmo:** Variable de control que almacena un enumerador o cadena (ej. `.EWMA`", `ÇUSUM`", o `"Z-SCORE"`), dictando qué vía matemática se aplicará a la matriz de datos actual.
- **nivel_confianza:** Parámetro flotante (por defecto 3,0 correspondiente a 3σ o 99.7%), que define la severidad visual con la que se marcan las anomalías.

13.3 3. Análisis de Clases y Funciones (Desglose Lógico)

La clase principal **PanelSPC(QWidget)** implementa la orquestación visual de las anomalías:

13.3.1 A. Topología de Interfaz y Controles (`_init_ui`)

Genera un *layout* dividido. En la franja superior o lateral, despliega controles interactivos que permiten ajustar el factor de suavizado λ (para EWMA) o el factor de holgura k (para CUSUM). El área restante es dominada por el **MotorGraficoMultimodal**.

13.3.2 B. Evaluación Estadística y Ruteo (`actualizar_spc`)

Es el método núcleo del panel. Recibe la matriz base de telemetría (t, y) y opera de la siguiente manera:

1. **Selección de Ruta:** Evalúa el `modo_algoritmo` seleccionado por el usuario.
2. **Invocación Matemática:** Pasa el tensor unidimensional y al motor de `control_calidad.py`.
3. **Recepción de Máscaras:** Recibe de vuelta tres elementos críticos: los Límites de Control Superiores (UCL), los Límites de Control Inferiores (LCL), y un **arreglo booleano (Máscara de Anomalías)** donde cada índice marcado con `True` representa una falla matemática en ese instante de tiempo t_i .

13.3.3 C. Inyección Visual Multicapa (`renderizar_anomalias`)

Una vez procesados los datos, transfiere la carga al motor gráfico invocando `self.motor_grafico.marcar_anomalias(t, y, mascara_booleanos, UCL, LCL)`. El resultado visual es la gráfica temporal base, flanqueada por líneas de umbral dinámicas, con puntos rojos de advertencia superpuestos (*Scatter Plot*) exactamente en las coordenadas donde el componente rompió el régimen de operabilidad estable.

13.4 4. El Orquestador y Ciclo de Vida

Al igual que sus paneles hermanos, su ciclo de vida es **Reactivo por Eventos (*Event-Driven Reactive*)**. Actualiza su estado en memoria visual bajo dos condiciones absolutas: 1) Cuando el `dashboards_ui.py` le notifica un desplazamiento temporal. 2) Cuando el usuario manipula los parámetros internos del panel (como cambiar de CUSUM a EWMA), forzando un recálculo aislado (estado ortogonal) sin solicitar nuevamente los datos al servidor.

13.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Completa la "Trinidad Analítica" dentro de la carpeta `componentes_dashboard/`:

- Mientras **PanelSeriesTiempo** muestra **Qué** está pasando, y **PanelFFT** muestra **Por qué** está vibrando, **PanelSPC** responde a la pregunta forense: **¿Cuándo falló y rompió la norma?**.
- Su existencia justifica los motores abstractos ubicados en el núcleo de Data Science.

13.6 6. Relación Exocrina (Con el resto del Ecosistema)

Opera como el "Semáforo Humano" del Gemelo Digital. En el *backend*, el Controlador SCADA y el Modelo Topológico 3D ya saben que el equipo está fallando, pero es a través de las gráficas renderizadas por este archivo que el arquitecto o ingeniero de mantenimiento puede auditar la falla, exportar el reporte probabilístico y tomar medidas correctivas.

13.7 7. Aporte al Funcionamiento Global Total

El aporte indiscutible de `panel_spc.py` es la **Visualización Forense y Justificación de Fallas**.

Una caída de tensión puede parecer sutil e inofensiva en una gráfica normal, pero al pasar por el modelo matemático y ser ploteada por este panel, la deriva térmica o eléctrica se vuelve incuestionable. Convierte datos abstractos y tensores masivos en evidencia ingenieril procesable (*Actionable Insights*), logrando que *EcoCircuit* trascienda de la simple simulación y alcance la categoría de Plataforma de Mantenimiento Ciberfísico Predictivo.

14 Ruta y Archivo: EcoCircuit/analitica_avanzada/nucleo_matematico/motor_inferencias.py

En la arquitectura lógica del Gemelo Digital, este archivo funciona como el **Motor de Inteligencia Físico-Matemática**. Su responsabilidad excluye por completo el renderizado o la interfaz gráfica; su único propósito es aplicar las Leyes de la Termodinámica y el Electromagnetismo para deducir variables abstractas (no medidas directamente por los sensores) a partir de la telemetría cruda, generando "Sensores Virtuales".

14.1 1. Importaciones y Dependencias Estratégicas

La arquitectura de este módulo se fundamenta en un aislamiento estricto de las capas gráficas, importando exclusivamente librerías de alto rendimiento numérico:

- **numpy (np) y pandas (pd):** Herramientas fundamentales para la vectorización de ecuaciones físicas. Permiten procesar tensores multidimensionales de telemetría sin emplear iteradores iterativos nativos de Python.
- **scipy.constants:** Utilizada para acceder a constantes físicas universales predefinidas con alta precisión algorítmica.

14.2 2. Variables Globales y Metadatos Físicos

El módulo define un glosario de constantes paramétricas para la simulación de materiales y termodinámica:

- **Resistividad Base:** Valores como $\rho_{cu} = 1,68 \times 10^{-8} \Omega \cdot m$ (Cobre) a 20°C.
- **Coefficientes de Convección:** h estimado para tableros eléctricos y ductos HVAC, utilizado para evaluar la disipación térmica.
- **Umbral de Eficiencia Mecánica:** Constantes de referencia para estimar las pérdidas en el eje de motores de inducción acoplados a bombas.

14.3 3. Análisis de Clases y Funciones (Desglose Matemático)

La clase principal, `MotorInferencias`, opera como una tubería (*pipeline*) de funciones puras, garantizando que para una misma entrada, la salida termodinámica sea idéntica determinísticamente:

14.3.1 A. Inferencia de Potencia y Factor de Potencia

A partir del voltaje RMS, corriente RMS y potencia real medida, el motor deduce el triángulo de potencias completo. Vectoriza las siguientes operaciones sobre los arreglos de datos:

$$S = V_{rms} \times I_{rms} \quad (PotenciaAparente) \tag{9}$$

$$Q = \sqrt{S^2 - P_{activa}^2} \quad (PotenciaReactiva) \tag{10}$$

$$FP = \cos(\phi) = \frac{P_{activa}}{S} \tag{11}$$

14.3.2 B. Cálculo Vectorizado del Efecto Joule

Utiliza la telemetría de corriente cruzada con los metadatos geométricos del modelo BIM (longitud L y sección transversal A de los cables) para deducir las pérdidas en forma de calor:

$$R_T = \rho \frac{L}{A} [1 + \alpha(T_{op} - 20)] \tag{12}$$

$$P_{pérdidas} = I^2 \cdot R_T \cdot Fases \tag{13}$$

Este cálculo devuelve un nuevo vector virtual que representa la disipación en Watts de la infraestructura a lo largo del tiempo.

14.3.3 C. Inferencia de Deriva Térmica (*Thermal Drift*)

Evalúa si la temperatura detectada por los sensores excede la temperatura teórica calculada por la ley de enfriamiento de Newton, inyectando una nueva métrica llamada `Riesgo_Termico_Delta`.

14.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Operacional Sin Estado (*Stateless Pipeline*)**. El motor de inferencias no almacena memoria ni estado previo de la simulación. Recibe un *DataFrame* crudo, lo atraviesa por sus matrices de ecuaciones y devuelve un *DataFrame* enriquecido. Este diseño de programación funcional previene bloqueos por condiciones de carrera (*Race Conditions*) cuando múltiples hilos solicitan cálculos simultáneos.

14.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Es el puente analítico central del sub-espacio `nucleo_matematico/`:

- Recibe el *DataFrame* base desde `gestor_memoria.ds.py`.
- Devuelve los arreglos enriquecidos (ej. la columna virtual de "Pérdidas Joule") a `control_calidad.py`. Sin el motor de inferencias, el control de calidad solo podría buscar derivas en el Voltaje, pero no podría auditar el Riesgo Termodinámico integral.

14.6 6. Relación Exocrina (Con el resto del Ecosistema)

Su función más crítica es **Conectar la Topología 3D con la Telemetría**.

- Mientras el SCADA solo conoce los datos de V e I , y el Canvas 2D/3D solo conoce la longitud de los cables y el volumen de los muros, este motor fusiona ambos mundos.
- Extrae la longitud topológica del Grafo Eléctrico (`utils/grafico_electrico.py`) para calcular las caídas de tensión exactas en la simulación temporal dinámica.

14.7 7. Aporte al Funcionamiento Global Total

El aporte fundamental de `motor_inferencias.py` es la **Generación de Sensores Virtuales y Síntesis Ciberfísica**.

En un edificio real, es inviable financieramente colocar sensores físicos de temperatura y pérdida de potencia en cada metro de cable. Este motor resuelve dicha limitación matemáticamente. Al deducir el comportamiento termodinámico oculto a partir de las pocas variables eléctricas medidas, proporciona al Gemelo Digital un nivel de omnisciencia analítica, permitiendo a los ingenieros evaluar el desgaste de la infraestructura completa sin necesidad de sobrecargar el modelo físico con hardware innecesario.

15 Ruta y Archivo: EcoCircuit/controladores/gestor_3d.py

Dentro de la arquitectura jerárquica de *EcoCircuit Enterprise*, este archivo actúa como el **Motor de Traducción Espacial y Renderizado (Controlador 3D)**. Es el puente algorítmico que transforma la planimetría bidimensional interactiva y la topología de red (elásticos, cables, nodos) en una representación volumétrica inmersiva mediante el paradigma *Boundary Representation (B-Rep)* y extrusiones paramétricas.

15.1 1. Importaciones y Dependencias Estratégicas

Este controlador requiere el manejo concurrente de memoria gráfica (VRAM) y matrices algebraicas:

- **pyvista / vtk:** Dependencias centrales. PyVista actúa como una interfaz pitónica sobre VTK (*Visualization Toolkit* en C++). Permite la instanciación de mallas poliédricas (*PolyData*) y el manejo directo del *Pipeline* de renderizado OpenGL.
- **numpy (np):** Fundamental para la manipulación de vértices espaciales, aplicación de transformaciones afines y cálculo de normales de superficie en tiempo constante $O(1)$.
- **PySide6.QtCore:** Utilizado para integrar el lazo de eventos (*Event Loop*) de la interfaz gráfica principal con el hilo de actualización de la cámara tridimensional.

15.2 2. Variables Globales y Metadatos Espaciales

El archivo maneja parámetros críticos para la optimización del *Z-Buffer* y el *Level of Detail* (LOD):

- **Umbral de Resolución (RESOLUCION_CILINDROS):** Define la cantidad de caras para las mallas curvas (ej. tuberías y cables). Un valor optimizado (ej. 16 a 32 segmentos) previene la sobrecarga de la GPU con vértices microscópicos invisibles al ojo humano.
- **Cámaras y Frustum:** Vectores por defecto de iluminación direccional y coordenadas de la cámara isométrica base.

15.3 3. Análisis de Clases y Funciones (Desglose Lógico)

La clase principal **Gestor3D** encapsula la escena virtual. Sus operaciones se dividen en tres áreas matemáticas:

15.3.1 A. Algoritmo de Extrusión Bidimensional (generar_volumen)

Convierte vectores del plano de planta (X, Y) a tensores volumétricos (X, Y, Z). Para un muro definido por los puntos bidimensionales $\vec{p}_1$ y $\vec{p}_2$, un grosor w y una altura h :

$$V_{malla} = f_{extrusión}(\vec{p}_1, \vec{p}_2, w, h) \quad (14)$$

Calcula el vector director perpendicular en el plano XY , construye los 8 vértices del paralelepípedo, e invoca internamente rutinas de Delaunay o indexación directa de caras cuadriláteras (Quads) en VTK.

15.3.2 B. Mapeo Topológico Inverso (Ray-Casting y Picking)

Implementa un diccionario de estado `self.actores_bim` que correlaciona de manera biunívoca el UUID del elemento arquitectónico con la dirección de memoria en C++ (el actor VTK). Cuando el usuario hace clic en el modelo 3D, el motor dispara un rayo matemático (*Ray-Casting*) desde el plano de la cámara hacia la escena:

$$\vec{R}(t) = \vec{C}_{origen} + t \cdot \vec{D}_{direccion} \quad (15)$$

Detecta la intersección de malla más cercana, recupera el puntero VTK y resuelve en tiempo $O(1)$ a qué UUID del proyecto corresponde, permitiendo abrir paneles de información del equipo seleccionado.

15.3.3 C. Actualización de Materiales (PBR Shaders)

Expone el método `actualizar_estado_visual(uuid, estado)`. Si un motor eléctrico pasa a estado de falla térmica (dictado por `control_calidad.py`), este método recupera la malla y altera dinámicamente sus propiedades ópticas:

$$Color(r, g, b)_{t=1} = Color_Alerta \quad | \quad Opacidad(\alpha) = 0,8 \quad (16)$$

Logrando un renderizado de alerta o parpadeo visual directamente a través de las primitivas de OpenGL.

15.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Demanda Diferida (*Lazy Rendering*)**. Para no congelar la UI mientras el ingeniero dibuja muros en 2D, el **Gestor3D** no recalcula las mallas instantáneamente con cada movimiento del ratón. Escucha las señales estructurales del lienzo (ej. `geometria_modificada.sig`), agrupa las modificaciones y lanza un recálculo por lotes (*Batch Update*) únicamente cuando el panel 3D recupera el foco visual.

15.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como pilar de la carpeta `controladores/`:

- Funciona de forma paralela y agnóstica al Controlador Principal y al `gestor_scada.py`.
- Se encarga del ‘‘Cuerpo’’ físico de los elementos, mientras que el gestor SCADA se encarga del ‘‘Alma’’ termodinámica, consolidando ambas en este entorno 3D.

15.6 6. Relación Exocrina (Con el resto del Ecosistema)

- **Capa Base de UI (componentes_canvas/):** Recibe la geometría pura (líneas y polígonos) y los metadatos de los elementos BIM.
- **Capa Analítica (analitica_avanzada/):** Reacciona a las salidas matemáticas para modificar estéticamente la infraestructura (ej. un cable que sobrepasa su límite de Ampacidad se pinta gradualmente de rojo para simular disipación térmica por efecto Joule).

15.7 7. Aporte al Funcionamiento Global Total

El aporte de `gestor_3d.py` es la **Materialización Espacial e Inmersión Sensorial**. Transmuta un diagrama esquemático árido en un entorno navegable, comprensible espacialmente para el ojo humano. Sin este archivo, la detección de colisiones (*Clash Detection*) interdisciplinarias entre tuberías mecánicas y bandejas eléctricas sería matemáticamente invisible para el usuario. Es el motor que consolida a *EcoCircuit* como una auténtica plataforma *Building Information Modeling (BIM)*.

16 Ruta y Archivo: EcoCircuit/controladores/gestor_io.py

En la columna vertebral de *EcoCircuit Enterprise*, este archivo opera como el **Motor de Persistencia, Serialización y Transaccionalidad (Gestor I/O)**. Es el responsable absoluto de traducir el estado vivo y volátil del Gemelo Digital (alojado en la memoria RAM) a un formato estático, inmutable y recuperable en el disco duro, asegurando la interoperabilidad y previniendo la corrupción de datos.

16.1 1. Importaciones y Dependencias Estratégicas

La arquitectura de este módulo está diseñada para manejar operaciones a nivel de sistema operativo y compresión de bytes:

- **json y pickle:** Para la serialización de diccionarios de metadatos y objetos complejos de Python, transformando jerarquías de clases en cadenas de texto o flujos binarios.
- **zipfile / shutil:** Empleados para empaquetar múltiples capas de información (geometría plana, grafos eléctricos, bases de datos CSV del SCADA) en un único contenedor comprimido (ej. un archivo de extensión propietaria `.eco`).
- **os y pathlib.Path:** Dependencias fundamentales para el enrutamiento independiente del sistema operativo y la validación de permisos de escritura.
- **hashlib:** Utilizado para calcular firmas criptográficas (*Checksums* SHA-256) que validan la integridad del archivo y previenen la inyección de código malicioso al cargar proyectos externos.

16.2 2. Variables Globales y Metadatos de Archivo

Este archivo define el protocolo de firmas del formato de guardado:

- **MAGIC HEADER:** Una cadena de bytes invariable (ej. `b'ECOBIM.V4'`) ubicada al inicio de cada archivo guardado, permitiendo al sistema abortar cargas si el usuario intenta abrir un archivo dañado o de otro programa.
- **AUTO_SAVE_INTERVAL_MS:** Constante de temporización que dicta la frecuencia del guardado en segundo plano (típicamente cada 5 o 10 minutos).

16.3 3. Análisis de Clases y Funciones (Desglose Lógico)

La clase principal `GestorIO` implementa un riguroso patrón de repositorio mediante las siguientes fases:

16.3.1 A. Proceso de Serialización y Aplanamiento (`guardar_proyecto`)

La memoria RAM utiliza punteros (direcciones de memoria) para relacionar objetos. Al guardar en disco duro, estos punteros son inútiles. El gestor ejecuta un algoritmo de aplanamiento transversal ($O(V + E)$ para el grafo BIM) que:

1. Recorre todos los muros, equipos y cables.
2. Extrae exclusivamente sus atributos crudos (coordenadas espaciales, propiedades físicas) y los agrupa utilizando sus **UUID** inmutables como llaves relacionales primarias.
3. Empaqueta esta estructura plana en un archivo JSON o binario.

16.3.2 B. Reconstrucción Topológica (`cargar_proyecto`)

Es el proceso inverso e intelectualmente más complejo.

$$Estado_Memoria = f_{instanciar}(Parseo(Disco)) \tag{17}$$

Lee el archivo, verifica la firma criptográfica SHA-256 para asegurar que ningún bit fue corrompido, instancia de nuevo los objetos visuales (Canvas 2D) e invoca a los controladores para rearmar los grafos eléctricos, enlazando nuevamente las conexiones físicas basándose en los **UUID** recuperados.

16.3.3 C. Escritura Atómica y Prevención de Corrupción (`_escritura_segura`)

Implementa tolerancia a fallos de hardware (ej. apagones durante el guardado):

- Nunca sobrescribe el archivo original directamente.
- Escribe toda la información en un archivo temporal oculto (`proyecto.tmp`).
- Solo cuando los bytes se han escrito al 100 % de manera exitosa, invoca una instrucción a nivel OS (`os.replace`) para intercambiar el archivo original por el nuevo. Esto garantiza que la transacción es atómica: o se guarda perfectamente, o no se modifica nada.

16.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Impulsado por Comandos y Tareas en Segundo Plano**. Entra en acción de manera sincrónica cuando el usuario presiona "Guardar/Abrir" (`Ctrl+S / Ctrl+O`) o invoca una exportación (a CSV, PDF o IFC). Alternativamente, ejecuta un hilo en segundo plano (*Daemon Thread*) que crea instantáneas (*Snapshots*) silenciosas para la función de Autorrecuperación.

16.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Opera como el registrador maestro dentro de `controladores/`:

- Pausa temporalmente el `gestor_scada.py` y bloquea las modificaciones en `gestor_3d.py` durante los milisegundos que dura el proceso de guardado para evitar condiciones de carrera" (donde un usuario borra un elemento mientras el sistema intenta serializarlo).

16.6 6. Relación Exocrina (Con el resto del Ecosistema)

Actúa como el principal puerto de **Interoperabilidad Exterior**.

- **Hacia el motor Analítico:** Puede empaquetar los *DataFrames* de `gestor_memoria_ds.py` y exportarlos como `.csv` puro para que un equipo de científicos de datos externo evalúe el edificio en Python o R.
- **Hacia el SO:** Gestiona el árbol de directorios creado originalmente por `main.py` en el perfil del usuario, vaciando cachés antiguos y gestionando la carpeta de `backups/`.

16.7 7. Aporte al Funcionamiento Global Total

El aporte de `gestor_io.py` es la **Continuidad, Seguridad y Portabilidad de la Ingeniería**. El renderizado más hermoso y el análisis de Data Science más preciso son irrelevantes si el trabajo no puede ser preservado, compartido o protegido contra caídas del sistema. Al implementar escrituras atómicas y firmas de integridad, este archivo certifica que EcoCircuit no es solo un entorno de experimentación volátil, sino una herramienta comercial sólida en la que firmas de arquitectura pueden confiar el resguardo de años de propiedad intelectual.

17 Ruta y Archivo: EcoCircuit/controladores/gestor_scada.py

En la anatomía de *EcoCircuit Enterprise*, este archivo constituye el **Corazón Temporal** y el **Motor de Adquisición de Datos (SCADA - Supervisory Control and Data Acquisition)**. Su responsabilidad es arrancar la dimensión del tiempo, transformando un modelo CAD estático y topológico en un Gemelo Digital dinámico, vivo y reactivo a las leyes de la física a lo largo del tiempo.

17.1 1. Importaciones y Dependencias Estratégicas

Este módulo requiere librerías enfocadas en concurrencia asíncrona y telemetría:

- **PySide6.QtCore (QTimer, QThread, QMutex):** Dependencias críticas para la ejecución de bucles de tiempo en segundo plano sin congelar la Interfaz de Usuario. Garantiza el paralelismo arquitectónico.
- **numpy (np):** Utilizado para inyectar ruido estocástico a las lecturas de los sensores, simulando el comportamiento eléctrico imperfecto del mundo real.
- **analitica.avanzada.gestor_memoria_ds:** Importado para despachar los paquetes de telemetría recolectados directamente hacia el almacenamiento masivo en RAM.
- **utils.grafo_electrico:** Requerido para leer el estado del árbol de nodos físicos y resolver los flujos de carga en cada iteración temporal.

17.2 2. Variables Globales y Metadatos Temporales

Establece las reglas del espacio-tiempo de la simulación:

- **TICK_RATE_MS:** Frecuencia de actualización en milisegundos (usualmente 1000 ms). Define la resolución máxima temporal del sistema.
- **TIME_MULTIPLIER:** Factor de aceleración algorítmica ($1x$, $10x$, $100x$). Permite al usuario simular el comportamiento de un mes de operación electromecánica en cuestión de minutos.
- **SIMULATION_STATE:** Bandera atómica (Booleana o Enum) que dictamina si el motor está **RUNNING**, **PAUSED** o **STOPPED**.

17.3 3. Análisis de Clases y Funciones (Desglose Lógico)

La clase principal **GestorSCADA** orquesta el lazo de integración y muestreo:

17.3.1 A. Bucle de Simulación e Integración (.physics_tick)

Constituye el iterador principal. En cada *tick* del reloj, integra el avance del tiempo $t_{i+1} = t_i + \Delta t$. Recorre el grafo topológico en complejidad $O(V + E)$ (Vértices y Aristas) para actualizar perfiles de carga dinámicos (ej. simulando que a las 14:00 hrs la carga de los aires acondicionados aumenta por la inercia térmica solar).

17.3.2 B. Adquisición de Telemetría Sintética (recolectar_lecturas)

Extrae los tensores ideales de voltaje y corriente calculados por las leyes de Kirchhoff en el grafo, y les inyecta incertidumbre gaussiana para crear una señal realista:

$$V_{sensor} = V_{ideal} + \mathcal{N}(0, \sigma_{ruido}^2) \tag{18}$$

Esto previene que Data Science analice líneas rectas planas irreales, forzando a los motores de *Control de Calidad* a trabajar con varianza estadística genuina.

17.3.3 C. Canalización y Despacho (emitir_paquetes)

Agrupar todas las lecturas de los N nodos del sistema en un único diccionario serializado o bloque de datos matricial. Utiliza señales asíncronas (**self.nuevos_datos_sig.emit(paquete)**) para expulsar la información hacia las capas superiores, evitando cuellos de botella”por acoplamiento duro.

17.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es el de un **Demonio en Segundo Plano (*Background Daemon*)**. Permanece latente al abrir el programa. Solo despierta cuando el usuario hace clic en el botón Iniciar Simulación.^{em} la interfaz principal. A partir de ahí, se independiza del Hilo Principal (*Main Thread*) delegando su iterador a un **QTimer** interno o un hilo de trabajo (**QThread**) protegido por **QMutex** (exclusión mutua) para evitar sobreescritura de memoria.

17.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como pilar en **controladores/**:

- **Con gestor_io.py:** Recibe comandos de pausa. Si el sistema necesita guardar el proyecto, el SCADA congela su *tick* temporal para evitar que el estado cambie mientras **gestor_io.py** lee la memoria, previniendo archivos corruptos.
- **Con gestor_3d.py:** Informa de estados críticos eléctricos inmediatos (ej. un interruptor que saltó por cortocircuito) para que el motor 3D renderice el componente apagado o en alerta.

17.6 6. Relación Exocrina (Con el resto del Ecosistema)

Es el **Proveedor Maestro** del sub-ecosistema de Data Science. El **gestor_memoria_ds.py** (que se analizó previamente) no existiría sin la tubería constante de vectores que este SCADA produce. Adicionalmente, alimenta los paneles de herramientas en vivo (Tooltips) del Canvas 2D, permitiendo que al pasar el ratón sobre un tablero, se desplieguen sus Amperes actuales oscilando en tiempo real.

17.7 7. Aporte al Funcionamiento Global Total

El aporte indiscutible de **gestor_scada.py** es la **Cuarta Dimensión (El Tiempo)**.

Transforma una base de datos estática (geometría y topología) en un organismo cibernético. Al introducir la progresión temporal y la simulación de varianzas, convierte a *EcoCircuit* de una simple herramienta de dibujo arquitectónico en un Simulador de Estrés Operacional. Es el motor que posibilita predecir cómo se comportará la infraestructura eléctrica de un hospital a lo largo de un año de operación antes de que se coloque el primer ladrillo físico.

18 Ruta y Archivo: EcoCircuit/core/catalogo_electrico.py

En la base epistemológica de *EcoCircuit Enterprise*, este archivo funge como el **Cerebro Normativo y Repositorio de Componentes** (*Master Data Management*). Es la única fuente de verdad (SSOT) sobre las propiedades físicas, geométricas y eléctricas de los componentes del mundo real (calibres AWG, interruptores termomagnéticos, transformadores), blindando la simulación contra violaciones a las leyes de la física y normativas internacionales (NEC, IEC, NOM).

18.1 1. Importaciones y Dependencias Estratégicas

Su arquitectura se basa en el acceso ultrarrápido y la indexación estructurada, manteniéndose libre de dependencias gráficas o motores de tiempo:

- **json y pathlib:** Utilizados para cargar de manera determinista los archivos de metadatos (bases de datos estáticas en disco) que contienen las tablas de resistividad y catálogo de fabricantes sin saturar la RAM con código estático.
- **dataclasses / typing (Dict, List):** Dependencias fundamentales para forzar un tipado estricto. Cada cable o motor no es un diccionario genérico, sino un objeto inmutable de datos (ej. `@dataclass class CableAWG`), previniendo errores de tipo en las capas de cálculo.

18.2 2. Variables Globales y Metadatos Normativos

Este archivo encapsula las tablas asintóticas de la ingeniería eléctrica:

- **Tablas de Ampacidad:** Mapeos de valores de diccionario (Hash Maps) que relacionan el calibre del conductor (ej. "10 AWG") con su corriente máxima soportada a 75°C (ej. 35A) y 90°C (ej. 40A).
- **Constantes Físicas de Materiales:** Parámetros fijos de resistividad en corriente alterna y reactancia inductiva (Ω/km), indispensables para el cálculo de caída de tensión.
- **Calibres de Protección:** Arreglos inmutables de los tamaños estándar comerciales de interruptores termomagnéticos ([15, 20, 30, 40, 50, ...]).

18.3 3. Análisis de Clases y Funciones (Desglose Lógico)

La clase principal `CatalogoElectrico` implementa el patrón *Singleton* y de *Factory*, garantizando complejidad $O(1)$ en todas sus consultas:

18.3.1 A. Resolución de Cables y Caída de Tensión (`obtener_parametros_cable`)

Recibe una cadena *string* del calibre y retorna su estructura de datos. Permite al motor matemático ejecutar cálculos de factores de ajuste térmico y de agrupamiento:

$$I_{corregida} = I_{tabla} \times F_{temp} \times F_{agrupamiento} \tag{19}$$

Devuelve el área transversal A (en mm^2) o el diámetro total para que el motor 3D sepa de qué grosor debe renderizar el cilindro geométrico.

18.3.2 B. Motor de Dimensionamiento Automático (`dimensionar_proteccion`)

Implementa lógica de ingeniería inversa para sugerir automáticamente el tamaño del *breaker*.

$$I_{protección} \geq 1,25 \times I_{cargacontinua} + 1,00 \times I_{carganocontinua} \tag{20}$$

Aplica este algoritmo y luego busca mediante Bisección el valor comercial inmediatamente superior en su arreglo de capacidades estándar.

18.3.3 C. Validación Normativa (`validar_cumplimiento`)

Actúa como un juez. Recibe una corriente de diseño $I_{diseño}$ y las propiedades de un conductor elegido por el usuario. Si $I_{diseño} > I_{corregida}$, el método devuelve una excepción o una advertencia (*Warning*) de "Sobrecarga Normativa", bloqueando diseños inseguros.

18.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida ocurre en **Tiempo de Carga** (*Load-Time Initialization*). Es instanciado e hidratado una única vez durante el *bootstrapping* de la aplicación (orquestado por `main.py`). Una vez que los diccionarios de datos JSON son cargados a la memoria RAM como diccionarios nativos de Python, el catálogo entra en estado *Read-Only* (Sólo Lectura), garantizando seguridad frente a condiciones de carrera (*Thread-Safety*) si múltiples motores lo consultan en paralelo.

18.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como pilar central de la carpeta `core/`:

- Establece la taxonomía universal que utilizarán todos los demás módulos. Dictamina cómo debe llamarse un material o un dispositivo en todo el sistema.
- Nutre indirectamente al sistema de *logs* y reportes, asegurando que los PDF de memoria de cálculo se generen utilizando la nomenclatura técnica correcta.

18.6 6. Relación Exocrina (Con el resto del Ecosistema)

Su red de impacto abarca todas las capas del Gemelo Digital:

- **Capa de Interfaz (UI):** Popula todos los `QComboBox` (menús desplegables) en los paneles de propiedades (ej. `ui/inspector/`), asegurando que el ingeniero no pueda escribir "Cable Mágico", sino que deba seleccionar obligatoriamente un estándar del catálogo.
- **Capa Matemática (`motor_inferencias.py`):** Provee las constantes de resistividad (ρ) y reactancia (X) necesarias para calcular el Efecto Joule o la impedancia de cortocircuito.
- **Capa 3D (`gestor_3d.py`):** Transfiere los diámetros físicos reales de las tuberías (Conduit) para escalar las mallas B-Rep paramétricas de manera fidedigna.

18.7 7. Aporte al Funcionamiento Global Total

El aporte irremplazable de `catalogo_electrico.py` es el **Determinismo Físico y el Cumplimiento Legal (Compliance)**.

Sin este archivo, el Gemelo Digital sería una simulación abstracta (un simple grafo de nodos). Al incorporar las restricciones matemáticas del cobre, el aluminio y los fabricantes reales, asegura que cualquier cálculo arrojado por *EcoCircuit* pueda ser directamente utilizado para solicitar permisos de construcción e instalaciones eléctricas certificadas en la vida real. Es el anclaje del software a las leyes físicas y regulatorias del planeta Tierra.

19 Ruta y Archivo: EcoCircuit/core/exportador_mep.py

Dentro de la arquitectura de *EcoCircuit Enterprise*, este archivo constituye el **Puente de Interoperabilidad Universal (BIM Exporter)**. Mientras que el gestor interno de I/O preserva el estado del programa, el `exportador_mep.py` tiene la misión de traducir la geometría propietaria, la topología electromecánica (MEP) y los metadatos generados por Data Science hacia estándares abiertos de la industria (ISO 16739), permitiendo la comunicación directa con plataformas como Autodesk Revit, ArchiCAD o Navisworks.

19.1 1. Importaciones y Dependencias Estratégicas

Este módulo integra librerías especializadas en traducción de mallas y parseo de estándares abiertos:

- **ifcopenshell**: La dependencia maestra para la interoperabilidad. Permite la instanciación de clases del estándar *Industry Foundation Classes (IFC)*, traduciendo los tensores de PyVista en entidades paramétricas `IfcProduct`.
- **ezdxf**: Motor de escritura vectorial utilizado para proyectar y exportar la planimetría bidimensional hacia el estándar cerrado pero universal `.dxf` o `.dwg` (AutoCAD).
- **pandas (pd)**: Requerido para la consolidación de inventarios y cuantificaciones masivas en forma de *Bill of Materials (BOM)*, listos para exportarse a Excel (`.xlsx`) o CSV.
- **numpy (np)**: Utilizado para calcular las matrices de transformación afín (Rotación, Traslación y Escala) necesarias para alinear los sistemas de coordenadas locales al Norte Verdadero del proyecto.

19.2 2. Variables Globales y Metadatos de Estándar

Establece los vectores de conformidad para los archivos resultantes:

- **IFC_SCHEMA_VERSION = 'IFC4x3'**: Fuerza al motor a utilizar el esquema más moderno de IFC, el cual incluye soporte nativo y extendido para infraestructuras lineales y redes eléctricas (*IfcDistributionSystem*).
- **COORDENADAS_ORIGEN_GLOBAL**: Tupla (X_0, Y_0, Z_0) que correlaciona el origen (0,0,0) del *Canvas* local con las coordenadas UTM del mundo real.
- **PSET_ECOCIRCUIT_ANALYTICS**: Identificador del conjunto de propiedades (*Property Set*) personalizado, donde se inyectarán las conclusiones matemáticas del Gemelo Digital (ej. Riesgo Térmico.º "Pérdidas Joule").

19.3 3. Análisis de Clases y Funciones (Desglose Matemático)

La clase `ExportadorMEP` despliega algoritmos de traducción geométrica y semántica:

19.3.1 A. Mapeo Semántico Volumétrico (exportar_a_ifc)

Traduce la representación visual interna (*Boundary Representation*) a entidades semánticas inteligibles por Revit. Para cada nodo eléctrico, aplica una Matriz de Transformación Afín homogénea T de 4×4 :

$$T = R_{11}R_{12}R_{13}t_xR_{21}R_{22}R_{23}t_yR_{31}R_{32}R_{33}t_z0001 \tag{21}$$

Multiplica esta matriz por los vértices locales del objeto para posicionarlo en el espacio global del IFC. Clasifica automáticamente los cables como `IfcCableSegment` y los tableros como `IfcFlowController`.

19.3.2 B. Proyección Planimétrica Ortogonal (generar_planos_dxf)

Si el ingeniero requiere planos tradicionales 2D, este método ejecuta un aplanamiento topológico. Proyecta matemáticamente todos los vértices tridimensionales (x_i, y_i, z_i) sobre el plano $Z = 0$, conservando las jerarquías de capas (*Layers*):

$$P_{2D} = 100010xyz = xy \tag{22}$$

Asigna grosores de línea (Lineweights) y tipos de trazo normativos en AutoCAD (ej. líneas punteadas para tuberías ocultas bajo losa).

19.3.3 C. Agregación de Inventario - BOM (cuantificar_proyecto)

Ejecuta un recorrido iterativo $O(N)$ sobre el árbol del proyecto, aglomerando longitudes de cables (con una constante de desperdicio empírica del 5%), conteo de interruptores por calibre, y volumen de concreto. Utiliza las operaciones de agrupamiento de Pandas (`groupby('Material').sum()`) para generar el presupuesto maestro.

19.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Bajo Demanda Terminal (*On-Demand Export*)**. Este archivo no está activo durante el modelado, ni durante la simulación temporal del SCADA. Su código se ejecuta como el clímax del flujo de trabajo: es llamado por `main_window.py` únicamente cuando el usuario navega a **Archivo >Exportar >Modelo IFC / Planos DXF**. Abre un *Thread* paralelo con una barra de progreso, dado que la instanciación de un archivo IFC de 200MB puede tomar varios segundos o minutos de computación intensiva en el disco (*I/O Bound*).

19.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como remate de la carpeta `core/`:

- **Con `catalogo_electrico.py`**: Utiliza los nombres de fabricantes y descripciones formales de este catálogo para llenar los atributos `IfcMaterial` y `IfcManufacturer`, asegurando que el modelo exportado tenga especificaciones comerciales reales.

19.6 6. Relación Exocrina (Con el resto del Ecosistema)

Actúa como la **Embajada del Software**:

- Recibe la malla tridimensional pura desde `controladores/gestor_3d.py`.
- Recibe el grafo eléctrico desde `utils/grafico_electrico.py` para mantener la relación de conectividad (`IfcRelConnectsPorts`) en el archivo exportado, asegurando que Navisworks entienda qué cable alimenta a qué motor.
- Recibe los hallazgos matemáticos desde `analitica_avanzada/` para inyectarlos en las propiedades (*Property Sets*) de los objetos.

19.7 7. Aporte al Funcionamiento Global Total

El aporte de `exportador_mep.py` es la **Ruptura del Cerco Propietario (Vendor Lock-in Prevention)**.

Por muy avanzado que sea el Gemelo Digital y su análisis termodinámico, si la información queda atrapada dentro de *EcoCircuit*, el software sería inútil en un flujo de construcción colaborativo (*OpenBIM*). Este módulo convierte la herramienta de un mero "software de cálculo." a una **Plataforma de Autoría BIM de Grado Profesional**, permitiendo a firmas de ingeniería enviar sus modelos certificados, presupuestos y planos directamente a despachos de arquitectura, contratistas generales y dependencias gubernamentales utilizando el idioma universal de la industria.

20 Ruta y Archivo: EcoCircuit/core/migrador.py

En la evolución a largo plazo del Gemelo Digital, este archivo funciona como el **Motor de Retrocompatibilidad y Transmutación de Esquemas (*Schema Migration Engine*)**. Su misión es salvaguardar la propiedad intelectual del usuario, garantizando que un proyecto electromecánico diseñado en una versión antigua del software (ej. v2.1) pueda ser interpretado, corregido algorítmicamente y cargado sin corrupción de memoria en la arquitectura actual (v4.6.0).

20.1 1. Importaciones y Dependencias Estratégicas

La arquitectura del migrador exige un manejo profundo de estructuras de datos crudas y versionamiento:

- **packaging.version o semver:** Utilizado para evaluar el versionamiento semántico (*Semantic Versioning*) mediante comparaciones algorítmicas estrictas, permitiendo al sistema saber si "3.9.5« "4.0.0" de manera matemática, sin depender de comparaciones de cadenas de texto falibles.
- **copy (deepcopy):** Crucial para crear mutaciones del árbol JSON o del diccionario de estado en la RAM sin alterar el archivo original, manteniendo una política estricta de *Zero-Data-Loss* en caso de falla durante la migración.
- **core.catalogo_electrico:** Importado para mapear identificadores de componentes (**Part Numbers**) obsoletos o descontinuados hacia sus equivalentes modernos.

20.2 2. Variables Globales y Metadatos de Versionamiento

Define el mapa de topología evolutiva del software:

- **CURRENT_SCHEMA_VERSION = "4.6.0":** La meta termodinámica y estructural a la que todos los archivos antiguos deben llegar.
- **MIGRATION_PATHWAYS:** Un diccionario de punteros a funciones (*Function Pointers*) que define el enrutamiento algorítmico, por ejemplo: {"3.x>transformar_v3.a_v4", "4.0->parche_v4.2"}.

20.3 3. Análisis de Clases y Funciones (Desglose Lógico)

La clase principal **MigradorProyectos** implementa un *Pipeline* iterativo de transformaciones topológicas:

20.3.1 A. Bucle de Salto Discreto (ejecutar_migracion)

Si se carga un archivo v2.0, el sistema no tiene una función para saltar directamente a v4.6. El método aplica una integración paso a paso, pasando el árbol de datos por cada subversión mayor, asegurando la consistencia:

$$D_{v_n} = f_{v_{n-1} \rightarrow v_n} \left(f_{v_{n-2} \rightarrow v_{n-1}} \left(\dots (D_{v_0}) \right) \right) \tag{23}$$

Donde D es el diccionario maestro del proyecto y f es la función de transformación de esquema.

20.3.2 B. Transformaciones Geométricas y Escala (_migrar_escal_a_espacial)

Históricamente, versiones previas pudieron utilizar milímetros o coordenadas absolutas distintas. Este submétodo rastrea todos los vértices del modelo y aplica una matriz escalar de conversión unitaria (ej. cm a metros) para evitar que un hospital diseñado en v2.0 se renderice 100 veces más pequeño en el motor 3D actual:

$$X_{nuevo} Y_{nuevo} Z_{nuevo} = S_x 000 S_y 000 S_z X_{viejo} Y_{viejo} Z_{viejo} \tag{24}$$

20.3.3 C. Reparación de Grafos Huérfanos (_reparar_topologia_mep)

Versiones antiguas del código podían permitir errores topológicos (ej. un cable conectado a un tablero que fue borrado). El migrador ejecuta un barrido topológico en complejidad $O(V + E)$ verificando la existencia de los UUID. Si encuentra una arista (*Edge*) huérfana, la desconecta de manera segura o la asigna a un "Nodo Neutro" virtual para evitar que el **gestor_scada.py** colapse por un **KeyError**.

20.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Interceptor Dinámico de Tiempo de Carga (*Load-Time Interceptor*)**. El archivo permanece dormido la mayor parte del tiempo. Solo se activa instantáneamente después de que **gestor_io.py** lee un archivo del disco duro, pero **estrictamente antes** de que cualquier controlador (**gestor_3d.py**, **ui/inspector/**) intente instanciar la geometría. Trabaja en el nivel más bajo de los datos crudos en la RAM. Si la versión del archivo coincide con **CURRENT_SCHEMA_VERSION**, el migrador se apaga inmediatamente en tiempo $O(1)$.

20.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como escudo de la carpeta **core/**:

- Trabaja en simbiosis absoluta con **gestor_io.py**. Mientras I/O se encarga de los "bytes y la compresión", el **migrador.py** se encarga de la "lógica semántica".
- Se retroalimenta del **catalogo_electrico.py** para actualizar propiedades de materiales que pudieron cambiar por nuevas regulaciones internacionales entre los años de una versión a otra.

20.6 6. Relación Exocrina (Con el resto del Ecosistema)

Actúa como un **Muro de Fuego (*Firewall*) Semántico**:

- Protege a las capas superiores (SCADA, Data Science y Motor Gráfico) de la necesidad de mantener código sucio (*Spaghetti Code*) para lidiar con formatos antiguos.
- Gracias a este archivo, el **MotorInferencias** asume que el arreglo de datos siempre tiene la estructura perfecta de v4.6, simplificando drásticamente el desarrollo en el resto del ecosistema.

20.7 7. Aporte al Funcionamiento Global Total

El aporte irremplazable de **migrador.py** es la **Preservación de la Inversión Intelectual y Ciclo de Vida del Producto (PLM)**.

En el ámbito *Enterprise*, una firma de ingeniería puede tener proyectos con ciclos de aprobación y construcción que duran de 3 a 5 años. Si el software se actualiza y pierde la capacidad de leer archivos de hace dos años (o los lee con errores de cálculo), la herramienta es comercialmente inviable. Este módulo garantiza la inmortalidad de los datos, asegurando que un diseño de 2021 pueda ser analizado con los motores termodinámicos de 2026 sin perder un solo atributo topológico.

21 Ruta y Archivo: EcoCircuit/core/model.py

Dentro del paradigma Modelo-Vista-Controlador (MVC) y el Diseño Orientado al Dominio (*Domain-Driven Design*), este archivo es la **Fuente de Verdad en Memoria Viva (El Modelo)**. Constituye la abstracción pura del edificio y su red electromecánica. Está estrictamente purgado de cualquier lógica de renderizado gráfico o gestión de ventanas; su única misión es representar matemáticamente las entidades del mundo físico (transformadores, ductos, muros) y la jerarquía relacional entre ellas.

21.1 1. Importaciones y Dependencias Estratégicas

Su arquitectura se basa en la integridad estructural y la notificación asíncrona:

- **uuid**: Dependencia crítica para instanciar `uuid4()`. Garantiza que cada objeto creado en el ecosistema posea una firma criptográfica única de 128 bits, previniendo colisiones de memoria a nivel mundial incluso si se fusionan dos proyectos distintos.
- **dataclasses y typing (Optional, List, Dict)**: Utilizados para construir estructuras de datos rígidas, eficientes en RAM, y penalizar cualquier intento de inyectar variables no tipadas (*Duck Typing*) en el núcleo de la ingeniería.
- **PySide6.QtCore (QObject, Signal)**: Aunque el modelo no dibuja nada, hereda de `QObject` para implementar el patrón *Observer* pasivo. Permite emitir .ºndas de choque.º señales (ej. `modelo_cambiado_sig`) hacia los controladores cuando un atributo muta.

21.2 2. Variables Globales y Metadatos del Dominio

Establece las enumeraciones y los límites del universo lógico:

- **Enumeradores Topológicos (Enum)**: Define clases como `TipoNodo` (TABLERO, CARGA, GENERADOR) y `EstadoFisico` (ACTIVO, FALLA, MANTENIMIENTO).
- **Umbrales del Árbol**: Parámetros protectores contra desbordamientos de pila (*Stack Overflow*), limitando la profundidad de la jerarquía (ej. Proyecto → Edificio → Nivel → Habitación → Tablero → Breaker).

21.3 3. Análisis de Clases y Funciones (Desglose Lógico y Estructural)

El archivo consolida la estructura en forma de un Grafo Acíclico Dirigido (DAG) para la jerarquía espacial, y un Multigrafo Ponderado para las conexiones eléctricas:

21.3.1 A. Abstracción Base (EntidadBIM)

Clase abstracta (*Abstract Base Class*) de la que heredan todos los elementos físicos. Contiene la genética compartida:

$$Entidad = \{UUID_{128}, Nombre, Coordenadas\vec{p}(x, y, z), Padre_UUID\} \tag{25}$$

Obliga a que ningún elemento flote .^{en} el vacío"; todo equipo eléctrico debe pertenecer topológicamente a un espacio arquitectónico.

21.3.2 B. Definición de Vértices y Aristas (NodoElectrico y ConectorMEP)

- **NodoElectrico**: Representa cajas de conexión, motores o tableros. Posee propiedades termodinámicas latentes (potencia base, factor de potencia nominal).
- **ConectorMEP (El Cable/Tubería)**: Representa las aristas del grafo. A diferencia de un nodo, requiere estrictamente dos firmas UUID: `origen_uuid` y `destino_uuid`. Su longitud no se dibuja, se calcula matemáticamente por la norma euclidiana:

$$L = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2} \tag{26}$$

21.3.3 C. El Contenedor Supremo (ProyectoBIM)

Es la clase orquestadora del estado (El Árbol Raíz). Maneja repositorios indexados (Diccionarios *Hash Map*) con complejidad $O(1)$ para la recuperación de entidades: `self.entidades[uuid]` = `Entidad`. Expone métodos de agregación atómica como `agregar_nodo()` y `eliminar_conexion()`, asegurando que si un tablero se borra, todos los cables conectados a él se destrullan automáticamente (Cascada de Integridad Referencial).

21.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Residente en Memoria y Conducido por Transacciones (*State-Container*)**. Es instanciado al crear un nuevo proyecto o al terminar la carga de uno existente mediante `gestor_io.py`. Permanece vivo y en constante mutación durante toda la sesión de trabajo. No tiene hilos paralelos ni temporizadores; reacciona pasiva e instantáneamente a las invocaciones de los controladores de interfaz.

21.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como epicentro lógico de la carpeta `core/`:

- Valida sus instanciaciones con `catalogo_electrico.py` (un nodo no puede crearse si su material base no existe en el catálogo normativo).
- Se deja manipular por `migrador.py`, el cual ajusta su diccionario de atributos a las versiones más modernas del esquema al cargar archivos heredados.

21.6 6. Relación Exocrina (Con el resto del Ecosistema)

Es el **Sujeto de Observación Universal**:

- **Hacia el Canvas 2D y Visor 3D**: Estas interfaces son ciegas". Para saber dónde dibujar una línea o instanciar un prisma de PyVista, deben consultar las coordenadas X, Y, Z almacenadas exclusivamente en este modelo.
- **Hacia el SCADA (gestor_scada.py)**: El simulador termodinámico no inventa las potencias; lee iterativamente los atributos de `NodoElectrico` en este modelo para ejecutar los flujos de carga temporal.

21.7 7. Aporte al Funcionamiento Global Total

El aporte de `model.py` es la **Desacoplación Arquitectónica y la Centralización de la Verdad**.

Sin este archivo, las coordenadas espaciales vivirían dentro de los botones de la interfaz gráfica, y las propiedades eléctricas dentro del visor 3D (un antipatrón conocido como *Spaghetti State*). Al agrupar todo en una estructura de datos abstracta y matemáticamente pura, asegura que si el ecosistema decide reemplazar completamente PySide6 por una interfaz web, o PyVista por otro motor de render, toda la ingeniería electromecánica, los cálculos y la topología permanezcan intactos. Es el *Alma* inmutable del Gemelo Digital.

22 Ruta y Archivo: EcoCircuit/core/motor_3d.py

Dentro del ecosistema de *EcoCircuit Enterprise*, este archivo constituye el **Núcleo Matemático Geométrico (Geometric Kernel)**. A diferencia de `controladores/gestor_3d.py` (que se encarga de la cámara y el renderizado en la interfaz gráfica mediante OpenGL/PyVista), `motor_3d.py` vive puramente en la carpeta **core/**. Su responsabilidad exclusiva es el cálculo tensorial masivo, la generación paramétrica de mallas volumétricas (Vértices y Caras) y la resolución de álgebra espacial para la detección de colisiones físicas interdisciplinarias (*Clash Detection*).

22.1 1. Importaciones y Dependencias Estratégicas

La arquitectura de este módulo prescinde de cualquier librería de Interfaz Gráfica de Usuario (GUI), enfocándose en el cómputo de alto rendimiento:

- **numpy (np):** Dependencia absoluta. Utilizada para procesar nubes de puntos, rotaciones espaciales y cálculo de vectores normales a través de matrices densas, garantizando velocidad vectorizada en C.
- **scipy.spatial (KDTree, ConvexHull):** Importado para implementar particionamiento espacial recursivo. Permite buscar vecinos cercanos y superposiciones de objetos en tiempo logarítmico $O(N \log N)$ en lugar de cuadrático $O(N^2)$.
- **math / quaternions:** Para resolver transformaciones rotacionales tridimensionales sin sufrir el colapso de grados de libertad métricos conocido como *Gimbal Lock*.

22.2 2. Variables Globales y Metadatos Espaciales

Define los límites de resolución computacional para las matemáticas de mallas:

- **TOLERANCIA_INTERSECCION:** Un escalar ínfimo (ej. 1×10^{-5} metros) para mitigar los errores de punto flotante al evaluar operaciones booleanas entre sólidos (ej. un tubo atravesando un muro).
- **TESELACION_DEFECTO:** Determina la cantidad de subdivisiones poligonales para objetos curvos, equilibrando la fidelidad métrica y la memoria RAM disponible.

22.3 3. Análisis de Clases y Funciones (Desglose Matemático)

La clase principal `MotorGeometrico` opera como una factoría y auditor de formas 3D puras:

22.3.1 A. Generación Paramétrica (*Sweeping & Extrusion*)

Calcula la piel volumétrica (*Boundary Representation*) de una trayectoria. Para generar un ducto eléctrico, el motor toma un perfil circular bidimensional en un plano local y lo barre (*Sweep*) a lo largo de una trayectoria *Spline* tridimensional definida por los nodos del sistema:

$$V_{superficie} = \oint_{trayectoria} \mathbf{T}(t) \cdot Perfil_{2D} dt \quad (27)$$

Donde $\mathbf{T}(t)$ es la matriz de transformación (Frenet-Serret) que alinea la normal del perfil bidimensional con el vector tangente de la curva en cada punto.

22.3.2 B. Álgebra de Transformación y Cuaterniones

Para rotar un tablero eléctrico en el espacio 3D, el motor elude los ángulos de Euler, empleando Cuaterniones unitarios para representar la orientación:

$$q = \cos\left(\frac{\theta}{2}\right) + (u_x \mathbf{i} + u_y \mathbf{j} + u_z \mathbf{k}) \sin\left(\frac{\theta}{2}\right) \quad (28)$$

Multiplicando este cuaternión por los vértices de la malla, el motor actualiza la posición del equipo eléctrico de forma segura, invertible y computacionalmente eficiente.

22.3.3 C. Detección de Colisiones (Clash Detection Engine)

Implementa un algoritmo de Jerarquía de Volúmenes Envolventes (*Bounding Volume Hierarchy - BVH*). Primero, aproxima cada elemento arquitectónico y electromecánico con cajas alineadas a los ejes (AABB - *Axis Aligned Bounding Boxes*). Si existe una intersección entre dos AABB (ej. Tubería Eléctrica vs Ducto HVAC):

$$AABB_A \cap AABB_B \neq \emptyset \iff \{ A_{x,min} \leq B_{x,max} \wedge A_{x,max} \geq B_{x,min} \wedge A_{y,min} \leq B_{y,max} \wedge A_{y,max} \geq B_{y,min} \wedge A_{z,min} \leq B_{z,max} \wedge A_{z,max} \geq B_{z,min} \} \quad (29)$$

Solo si esta prueba barata en $O(1)$ es verdadera, el motor desciende al nivel de los triángulos (GJK Algorithm) para confirmar una penetración física exacta.

22.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es un **Servicio de Cómputo Bajo Demanda (*Stateless Compute Engine*)**. Carece de memoria interna persistente. El orquestador o los controladores le inyectan un problema geométrico (ej. "Calcula la intersección entre este cilindro y este cubo"), el motor destina todos los hilos de CPU disponibles a resolver la ecuación matricial, retorna la malla resultante (los vértices y triángulos) y libera la memoria.

22.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como pilar puramente abstracto de la carpeta **core/**:

- **Con `model.py`:** Lee la materia prima. Toma el vector abstracto de coordenadas $\vec{p}(x, y, z)$ definido en el modelo y lo transforma en matrices tangibles de geometría (V, F) .

22.6 6. Relación Exocrina (Con el resto del Ecosistema)

Funciona como la ****Unidad Lógica Aritmética Espacial (ALU)**** para el resto del software:

- **Hacia `controladores/gestor_3d.py`:** Le entrega las mallas calculadas para que este último simplemente las envíe a la tarjeta de video (GPU) y les aplique colores o texturas PBR.
- **Hacia `exportador_mep.py`:** Le provee el volumen paramétrico tridimensional exacto necesario para compilar el archivo IFC bajo el estándar ISO 16739.

22.7 7. Aporte al Funcionamiento Global Total

El aporte irremplazable de `motor_3d.py` es la **Conciencia Espacial Ciberfísica**.

Una tubería y una viga estructural no pueden ocupar el mismo espacio en el universo físico al mismo tiempo. En un programa de dibujo básico 2D, este error humano pasaría inadvertido hasta el día de la construcción, costando millones en retrabajos. Al integrar este núcleo geométrico, *EcoCircuit* procesa instantáneamente estas violaciones físicas, alertando al ingeniero de las colisiones en milisegundos. Es el módulo que consagra al software como una plataforma de Ingeniería BIM genuina, matemáticamente infalible y espacialmente rigurosa.

23 Ruta y Archivo: EcoCircuit/core/project_manager.py

En la cúspide de la lógica de dominio de *EcoCircuit Enterprise*, este archivo opera como el **Administrador de Sesión y Patrón Fachada (*Facade Pattern Master*)**. Mientras que `model.py` contiene los datos en bruto de la simulación, el `project_manager.py` es la entidad soberana que orquesta su creación, destrucción, hidratación y contexto a nivel de sistema, blindando la complejidad del núcleo (Core) ante las peticiones de la Interfaz de Usuario.

23.1 1. Importaciones y Dependencias Estratégicas

Su diseño exige librerías enfocadas en el control del estado y la temporalidad del sistema operativo:

- **core.model**: Importación cardinal para instanciar la clase `ProyectoBIM` raíz, que albergará todo el grafo de nodos electromecánicos.
- **datetime y pytz**: Utilizados para sellar cronológicamente (*Timestamping*) la creación, última modificación y tiempos de simulación del proyecto con precisión de milisegundos y rigor de zonas horarias.
- **gc (Garbage Collector)**: Importado desde la librería estándar de CPython para forzar la recolección de basura (liberación de memoria RAM) de manera explícita cuando un ingeniero cierra un modelo masivo de 5GB para abrir uno nuevo.

23.2 2. Variables Globales y Metadatos de Sesión

Establece las reglas de estado y permisos del entorno vivo:

- **SESSION_ACTIVIA**: Puntero global (generalmente instanciado como Singleton) que mantiene la referencia estricta al proyecto actualmente cargado en memoria.
- **Máquina de Estados de Modificación**: Variables booleanas como `is_dirty = True/False`. Si el usuario añade un solo cable, el Gestor de Proyecto marca la sesión como "Sucia" (*Dirty*), lo que desencadenará advertencias algorítmicas si intenta cerrar la aplicación sin invocar al `gestor_io.py`.

23.3 3. Análisis de Clases y Funciones (Desglose Lógico)

La clase principal `ProjectManager` opera como el conmutador central del estado:

23.3.1 A. Instanciación y Destrucción (`crear_nuevo_proyecto` y `cerrar_proyecto`)

Al invocar la creación, destruye cualquier puntero residual de simulaciones previas y genera un "Universo Vacío". La transición de estados se puede definir rigurosamente como:

$$S_{t+1} = \Phi(\text{Limpiar_RAM}) \rightarrow \text{Instanciar}(\text{ProyectoBIM}(id \leftarrow \text{UUID4}, t_0 \leftarrow \text{UTC})) \quad (30)$$

Al cerrar, invoca `gc.collect()` destruyendo los objetos de dominio para prevenir *Memory Leaks*.

23.3.2 B. Enrutador de Interacciones (`solicitar_guardado`)

No realiza la compresión binaria él mismo, sino que ejerce delegación arquitectónica. Actúa como mediador: recibe la orden de la GUI, pausa el *Event Loop* del modelo, e invoca al `gestor_io.py`, pasándole el puntero de memoria del proyecto vivo.

23.3.3 C. Auditoría de Metadatos (`obtener_resumen_contextual`)

Extrae la estadística vital del proyecto en tiempo $O(1)$ (ej. "Nombre del Autor", "Normativa Aplicada: NEC 2023", Cantidad de Nodos"). Este resumen es consumido por las pantallas de bienvenida y los encabezados de los reportes PDF.

23.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida abarca el **Tiempo de Ejecución Total de la Aplicación (*Application Lifetime*)**. Es instanciado en el milisegundo en que `main.py` arranca el programa, incluso antes de que la ventana gráfica (PySide6) se haya dibujado. Sobrevive a la apertura y cierre de múltiples archivos `.eco` durante una misma sesión de usuario. Es el supervisor inmortal del proceso de memoria hasta que el sistema operativo envía la señal `SIGTERM` (Cerrar Programa).

23.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como Gobernador de la carpeta `core/`:

- **Patrón Envoltorio (*Wrapper*)**: Abraza y protege a `model.py`. Ningún controlador o vista externa puede modificar una propiedad del modelo directamente; deben pedir permiso y canalizar sus acciones a través de `project_manager.py`.
- Coordina a `gestor_io.py` y `exportador_mep.py` indicándoles exactamente **qué** objeto de memoria deben procesar.

23.6 6. Relación Exocrina (Con el resto del Ecosistema)

Actúa como el **Único Punto de Contacto (*Single Point of Truth*)** para la Interfaz de Usuario:

- El archivo `ui/main_window.py` tiene un acoplamiento exclusivo con este Gestor. Cuando el arquitecto hace clic en "Nuevo Proyecto", la GUI solo lanza la instrucción ciega `manager.crear_nuevo()`, sin necesidad de entender cómo funcionan los árboles de grafos en el núcleo.

23.7 7. Aporte al Funcionamiento Global Total

El aporte de `project_manager.py` es el **Principio de Inversión de Dependencias y Encapsulamiento Absoluto**.

Garantiza que la lógica de negocios y la termodinámica del proyecto existan en un vacío esterilizado, intocables por los clics desordenados del ratón en la interfaz. Al implementar el patrón *Facade*, permite que un equipo de desarrolladores cambie por completo el diseño de las ventanas o el motor 3D, sabiendo que el ciclo de vida, el control de la memoria RAM y la integridad de la sesión en el núcleo del *EcoCircuit* permanecerán arquitectónicamente indestructibles.

24 Ruta y Archivo: EcoCircuit/core/utilidades.py

En la topología del ecosistema *EcoCircuit Enterprise*, este archivo funge como el **Motor de Servicios Transversales y Trazabilidad Forense**. A diferencia de los modelos y controladores que manejan la lógica de negocio, `utilidades.py` es una colección de funciones puras, decoradores de rendimiento y envoltorios de registro (*Logging*), implementando el principio *DRY (Don't Repeat Yourself)* para garantizar homogeneidad criptográfica y física en toda la aplicación.

24.1 1. Importaciones y Dependencias Estratégicas

Como módulo fundacional, evita importar librerías pesadas (como NumPy o PySide6) para no ralentizar el tiempo de arranque. Se apoya en la biblioteca estándar de Python:

- **logging y sys:** Dependencias absolutas para la instanciación del `EcoLogger`, capturando flujos de error (`stderr`) y salidas estándar (`stdout`).
- **functools (wraps):** Utilizado para la construcción de decoradores algorítmicos (funciones de orden superior) que perfilan el rendimiento de otros métodos.
- **time y math:** Requeridos para el cronometraje de rutinas de alto costo computacional y cálculos trigonométricos o exponenciales puros.

24.2 2. Variables Globales y Metadatos de Sistema

Define constantes universales que no son susceptibles de modificación en tiempo de ejecución:

- **LOG_FORMAT:** Un formato de cadena estricto (ej. "`%(asctime)s] [%(levelname)s] [%(module)s] ->%(message)s`") que estandariza cómo se escriben los eventos en los archivos `.log` para facilitar su posterior parseo con herramientas como Elasticsearch.
- **Constantes de Conversión Unitarias:** Escalares precisos para transformar unidades imperiales a métricas, asegurando que un pulgada (`inch`) siempre sea matemáticamente $25,4 \times 10^{-3}$ metros.

24.3 3. Análisis de Clases y Funciones (Desglose Matemático)

El módulo se divide en tres dominios lógicos independientes (sin estado):

24.3.1 A. Trazabilidad Asíncrona (EcoLogger)

Clase Singleton que gestiona la escritura concurrente en archivos de registro. Si el `gestor_scada.py` detecta un colapso térmico o el `migrador.py` corrige un archivo corrupto, invocan al `EcoLogger`. Este categoriza el evento (*INFO*, *WARNING*, *CRITICAL*) y lo sella con una marca de tiempo inmutable.

24.3.2 B. Transformación Dimensional Matemática (awg_a_mm2)

Implementa funciones puras de ingeniería física. Por ejemplo, el sistema no almacena en bases de datos el grosor de todos los cables AWG; en su lugar, utiliza la formulación logarítmica estándar de la ASTM para calcular el diámetro d (en mm) de un conductor sólido calibre n :

$$d_n = 0,127 \times 92^{\frac{36-n}{39}} \tag{31}$$

A partir del diámetro, devuelve el área de sección transversal $A = \frac{\pi \cdot d_n^2}{4}$, entregando resultados deterministas en tiempo $O(1)$ a la clase `CatalogoElectrico`.

24.3.3 C. Perfilamiento Computacional (@medir_rendimiento)

Un decorador que inyecta código de cronometraje alrededor de las funciones pesadas del sistema (como la Transformada de Fourier o la instanciación IFC).

$$\Delta t_{ejecucion} = t_{final} - t_{inicial} \tag{32}$$

Si $\Delta t >$ umbral crítico, el decorador emite una advertencia de *Performance Hit* al archivo de log, permitiendo la optimización asintótica por parte del equipo de desarrollo.

24.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Operacional Sin Estado (*Stateless Utility Belt*)**. Las funciones definidas en este archivo se compilan en *Bytecode* al iniciar el programa y viven de forma omnipresente en el espacio de nombres (Namespace) global. No almacenan instancias ni guardan memoria de sus invocaciones pasadas. Su costo computacional latente es cero.

24.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Es el **Cimiento de core/**:

- `model.py` utiliza sus funciones de parseo y validación de UUID.
- `project_manager.py` utiliza su `EcoLogger` para auditar la apertura y cierre de sesiones, previniendo la corrupción silenciosa de datos.

24.6 6. Relación Exocrina (Con el resto del Ecosistema)

Su impacto es de **Alcance Global (Cross-Cuttingting)**:

- **Data Science:** Los módulos de `analitica_avanzada/` importan el convertidor de *Timestamps* unix a cadenas legibles ISO 8601.
- **Capa Visual:** Los controladores UI (`dashboards_ui.py` o `main_window.py`) delegan a `utilidades.py` el redondeo y formateo de números de coma flotante para no sobrecargar el código de la ventana con manipulaciones de *strings* (ej. formatear `14.238491 A` como `"14.24 A"`).

24.7 7. Aporte al Funcionamiento Global Total

El aporte definitivo de `utilidades.py` es la **Higiene Arquitectónica y Auditoría Pericial**.

Sin este archivo centralizado, cada programador del sistema escribiría su propia función para convertir unidades, resultando en inconsistencias de punto flotante que desencadenarían fallos catastróficos en el simulador SCADA. Al unificar las constantes físicas, la matemática de áreas y el sistema de bitácoras, este módulo certifica que *EcoCircuit* sea un software trazable, depurable e implacablemente exacto en sus cimientos matemáticos, digno de certificación *Enterprise*.

25 Ruta y Archivo: EcoCircuit/core/window_manager.py

En la jerarquía estructural de *EcoCircuit Enterprise*, este archivo funciona como el **Controlador de Topología Visual y Estado Multi-Monitor**. Su propósito es garantizar la integridad del entorno de trabajo (UX/UI) abstrayendo la creación, destrucción, acoplamiento (*Docking*) y posicionamiento espacial de las ventanas. Previene fugas de memoria gráfica aislando las instancias y evitando que componentes críticos (como los paneles analíticos) sean duplicados o se pierdan fuera de los límites de la pantalla.

25.1 1. Importaciones y Dependencias Estratégicas

Como gestor del espacio visual en la capa núcleo, requiere dependencias específicas de control de aplicaciones, sin acoplarse al contenido de las ventanas:

- **PySide6.QtCore (QSettings, QByteArray):** Para la serialización y lectura ultrarrápida del estado binario de las geometrías de ventana en el registro del Sistema Operativo.
- **PySide6.QtGui (QScreen, QGuiApplication):** Utilizados para auditar la matriz de monitores físicos conectados al equipo, su resolución, densidad de píxeles (DPI) y coordenadas relativas.
- **typing (Dict, Type):** Empleado para mantener un registro indexado e inmutable de los punteros a memoria de las ventanas activas.

25.2 2. Variables Globales y Metadatos de Estado

El módulo define el mapa de persistencia del espacio de trabajo:

- **REGISTRY_ORGANIZATION y REGISTRY_APPLICATION:** Constantes que enrutan el almacenamiento de preferencias en el Regedit de Windows o en los archivos **.plist** de macOS.
- **diccionario_ventanas:** Un *Hash Map* (`{}`) privado que actúa como contenedor de instancias unívocas, donde la llave es el ID del componente y el valor es el puntero de memoria de su interfaz gráfica.

25.3 3. Análisis de Clases y Funciones (Desglose Lógico)

La clase principal **WindowManager** implementa el patrón *Registry* y gestor espacial:

25.3.1 A. Prevención de Duplicidad e Instanciación (`obtener_o_crear_ventana`)

Asegura que herramientas costosas computacionalmente (como el **DashboardAnalitica**) sigan un patrón *Singleton* modificado por llave.

$$V_{instancia} = \{ Mem[id] | siid \in Mem \rightarrow Inicializar(Clase, id) \rightarrow Mem[id] | siid \notin Mem \} \quad (33)$$

Si el ingeniero solicita abrir el Panel de Fourier cuando ya está abierto detrás de otras ventanas, el gestor no consume RAM creando uno nuevo; simplemente invoca **raise_()** y **activateWindow()** sobre el puntero existente.

25.3.2 B. Persistencia Topológica (`guardar_layout` y `restaurar_layout`)

Extrae el tensor de estado visual $\vec{V} = [x, y, w, h, z, d]$ de cada panel acoplable (*QDockWidget*) y barra de herramientas de la ventana principal, serializándolo en un arreglo de bytes (**saveState** y **saveGeometry** de Qt). Esto permite que el sistema recuerde matemáticamente si un usuario prefiere tener el inspector BIM en su monitor izquierdo y el lienzo 3D en el monitor derecho.

25.3.3 C. Detección y Corrección de Fronteras (`_validar_limites_pantalla`)

Previene el clásico error CAD de "ventanas perdidas" tras desconectar un monitor externo. El algoritmo audita las coordenadas de la ventana (X_w, Y_w) contra el volumen envolvente global (*Bounding Box*) de todos los monitores activos:

$$X_{seguro} = \max(\min(X_w, Monitor_{max_X} - W_{ancho}), Monitor_{min_X}) \quad (34)$$

Si la ecuación determina que la ventana cae en un área de "vacío geométrico", el gestor reubica sus coordenadas de origen hacia el centro del monitor principal de forma automática.

25.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida abarca el **Tiempo de Ejecución Gráfico** (*GUI Lifetime*). Se instancia inmediatamente después de construir la aplicación principal (**QApplication**) y permanece residente en memoria. Entra en acción como orquestador pasivo cuando el usuario abre sub-paneles y ejecuta sus guardados de estado al momento de invocar el evento de cierre global (**closeEvent**).

25.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como servicio paralelo dentro de **core/**:

- Convive de manera independiente con el **project_manager.py**. Mientras este último se encarga de que los **datos termodinámicos** sobrevivan a la sesión, el **window_manager.py** garantiza que el **espacio visual humano** sobreviva.

25.6 6. Relación Exocrina (Con el resto del Ecosistema)

Actúa como el **Titiritero Global de Interfaces**:

- **Hacia ui/main_window.py:** Inyecta los arreglos de bytes guardados para reconstruir instantáneamente la distribución de paneles de la última sesión.
- **Hacia analitica_avanzada/ y controladores/:** Brinda la infraestructura lógica para que un análisis termodinámico o el visor 3D se "desprendan" de la ventana principal (*Floating Window*) de manera segura, sin colapsar el hilo de comunicación subyacente.

25.7 7. Aporte al Funcionamiento Global Total

El aporte de **window_manager.py** es la **Continuidad Ergónomica y el Flujo de Trabajo Enterprise**.

En simuladores de ingeniería ciberfísica, el usuario típico utiliza configuraciones de dos o tres monitores (ej. esquemático 2D en el centro, analítica de datos a la derecha, tablas normativas a la izquierda). Sin este gestor de ventanas centralizado, la aplicación sufriría de instanciaciones múltiples destructivas, bloqueos de hilos (Deadlocks UI) y pérdida de paneles al cambiar resoluciones, reduciendo la eficiencia del ingeniero. Este archivo consagra la confiabilidad comercial (UX/UI de grado industrial) de *EcoCircuit*.

26 Ruta y Archivo: EcoCircuit/simulador/motor_datos.py

Dentro de la arquitectura de *EcoCircuit Enterprise*, este archivo opera como el **Sintetizador Estocástico de la Realidad (Fábrica de Telemetría)**. Cuando el Gemelo Digital no está conectado a sensores físicos IoT (Internet of Things) de un edificio real, `motor_datos.py` asume el rol del universo físico. Genera las firmas eléctricas, los ciclos termodinámicos y las anomalías de desgaste utilizando modelos de probabilidad avanzados, engañando efectivamente a la capa de Data Science para que analice los datos como si provinieran de hardware auténtico.

26.1 1. Importaciones y Dependencias Estratégicas

La arquitectura de este módulo se fundamenta en la generación de números pseudoaleatorios y la superposición de ondas matemáticas:

- **numpy (np.random, np.sin):** El motor principal. Se utiliza para generar distribuciones probabilísticas densas (Gaussiana, Poisson) y evaluar funciones trigonométricas vectorizadas a altísima velocidad.
- **scipy.stats (markov):** Implementado para modelar Cadenas de Markov, permitiendo simular transiciones de estado complejas (ej. la probabilidad de que un motor pase de un estado *Normal* a *Falla Incipiente* basándose en su estrés térmico previo).
- **math y time:** Dependencias base para cálculos de fase angular (ω) y sincronización de las ondas sintéticas con el reloj del mundo real.

26.2 2. Variables Globales y Metadatos Sintéticos

Establece los axiomas del laboratorio virtual:

- **FRECUENCIA_RED:** $60Hz$ o $50Hz$, dictando la velocidad angular $\omega = 2\pi f$ para todas las ondas base de voltaje y corriente.
- **SEMILLA_ESTOCASTICA (Random Seed):** Un valor numérico opcional. Si se define, obliga al motor a generar exactamente la misma secuencia de perturbaciones eléctricas cada vez que se corre la simulación, crucial para depurar algoritmos de *Machine Learning* en la capa analítica.
- **Amplitudes Armónicas Base:** Constantes de inyección (ej. $k_3 = 0,15$ para el 3^{er} armónico), simulando la presencia de cargas no lineales como inversores o luces LED.

26.3 3. Análisis de Clases y Funciones (Desglose Matemático)

La clase principal `GeneradorTelemetria` actúa como un orquestador de funciones de onda perturbadas:

26.3.1 A. Síntesis de Onda con Distorsión Armónica Total (THD)

En lugar de generar una onda senoidal perfecta e irreal, el método `generar_firma_electrica` superpone armónicos impares y añade ruido blanco gaussiano para simular la degradación de la red eléctrica en tiempo real:

$$I(t) = I_{rms}\sqrt{2}\left(\sin(\omega t) + \sum_{h=3,5,7} k_h \sin(h\omega t + \phi_h)\right) + \mathcal{N}(0, \sigma_{ruido}^2) \quad (35)$$

Donde $\mathcal{N}$ representa la distribución normal que ensucia la lectura, simulando la interferencia electromagnética (EMI) de los sensores físicos.

26.3.2 B. Simulación Termodinámica (Inercia HVAC)

Para cargas como aires acondicionados, el consumo no es constante, sino que obedece a ciclos de trabajo dictados por la temperatura ambiente. El método `simular_ciclo_termodinamico` aplica un modelo aproximado de la Ley de Enfriamiento de Newton acoplado a un termostato de histéresis:

$$T_{espacio}(t + \Delta t) = T_{ext} - (T_{ext} - T_{espacio}(t))e^{-\Delta t/\tau_{muro}} - \dot{Q}_{HVAC} \cdot C_{estado} \quad (36)$$

Donde $C_{estado} \in \{0, 1\}$. Esto produce encendidos periódicos (arranques de compresor) que se traducen en picos masivos de corriente (Corriente de Inrush) en el tablero eléctrico virtual.

26.3.3 C. Inyección de Deriva de Sensores (Sensor Drift)

Implementa un algoritmo de caminata aleatoria (*Random Walk*) para simular cómo un sensor de temperatura o voltaje pierde calibración a lo largo de meses virtuales:

$$Error_{calibración}(t) = Error_{calibración}(t - 1) + \mathcal{N}(0, \sigma_{deriva}^2) \quad (37)$$

Esto fuerza a los algoritmos de Control Estadístico de Procesos (SPC) en `panel_spc.py` a detectar y discriminar entre fallas del equipo real y fallas del sensor.

26.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es de **Ejecución por Instancias Matemáticas (Stateless Math Engine)**. El generador no almacena matrices masivas. Es invocado en cada iteración por el `gestor_scada.py`. Recibe el tiempo absoluto t , los parámetros nominales del equipo (ej. $480V$, $100A$), ejecuta las ecuaciones diferenciales o trigonométricas en microsegundos, devuelve el flotante perturbado, y limpia su memoria caché interna.

26.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como pilar del submódulo `simulador/`:

- Centraliza toda la lógica de "Falsa Realidad". Si un programador quiere simular un cortocircuito, no altera los controladores BIM, sino que inyecta una función escalón (Función de Heaviside) transitoria directamente en las matemáticas de este archivo.

26.6 6. Relación Exocrina (Con el resto del Ecosistema)

Actúa como el **Proveedor de Materia Prima para el Gemelo Digital**:

- **Hacia controladores/gestor_scada.py:** Le entrega los vectores numéricos en bruto en cada *Tick* de reloj para que el SCADA los distribuya por el grafo eléctrico.
- **Impacto en analitica_avanzada/:** Gracias a la asimetría matemática de este archivo, los tableros analíticos como el `PanelFFT` tienen algo interesante que mostrar (los picos armónicos que este motor inyecta deliberadamente), validando la robustez de las herramientas de Data Science.

26.7 7. Aporte al Funcionamiento Global Total

El aporte de `motor_datos.py` es la **Viabilidad Ciberfísica y Validación Analítica (Proof of Concept)**.

Sin este motor, *EcoCircuit* requeriría estar obligatoriamente conectado a un PLC y a un edificio físico para demostrar sus capacidades analíticas, lo cual limitaría drásticamente su venta, pruebas y demostraciones. Al incluir un motor capaz de generar "Datos Reales Falsos" con precisión termodinámica y eléctrica, el software se convierte en una herramienta completa (Caja Blanca) que permite realizar Análisis de Sensibilidad y Entrenamiento de IA preventivo antes de que el edificio sea construido.

27 Ruta y Archivo: EcoCircuit/simulador/motor_tiempo.py

En la arquitectura fundamental de *EcoCircuit Enterprise*, este archivo actúa como el **Manejador del Espacio-Tiempo Relativista (*Chronos Engine*)**. Su propósito es independizar el reloj biológico de la computadora (Tiempo Real del CPU) del reloj termodinámico del Gemelo Digital (Tiempo Simulado), permitiendo compresión cronológica (para predicción de desgaste a largo plazo) o dilatación temporal (para análisis microscópico de transitorios eléctricos).

27.1 1. Importaciones y Dependencias Estratégicas

La arquitectura de este módulo exige un manejo riguroso de la concurrencia y los relojes de hardware de alta resolución:

- **time (perf_counter, monotonic):** Utilizado para acceder al reloj de hardware del procesador más preciso disponible, inmune a las actualizaciones del servidor NTP (Network Time Protocol) o cambios de horario de verano, evitando saltos temporales corruptos.
- **datetime y timedelta:** Requeridos para convertir los flotantes de alta precisión Unix en sellos de tiempo de calendario legibles (ej. "Lunes 08:00 AM") cruciales para la simulación de cargas solares o térmicas.
- **threading (Lock):** Dependencia crítica para implementar exclusión mutua (*Mutex*). Dado que la interfaz gráfica y los motores matemáticos intentarán leer el tiempo simultáneamente, el cerrojo previene condiciones de carrera durante la actualización del *Tick*.

27.2 2. Variables Globales y Metadatos de Escala

Define los tensores estáticos del flujo temporal:

- **ESTADO_CRONOS:** Enumerador estricto que controla la máquina de estados lógicos: `PLAY`, `PAUSE`, `FAST_FORWARD`.
- **FACTOR_ACELERACION (α):** Escalar multiplicador por defecto. Un $\alpha = 1,0$ equivale a tiempo real; un $\alpha = 3600,0$ hace que un segundo de cómputo equivalga a una hora de desgaste termodinámico en el modelo.

27.3 3. Análisis de Clases y Funciones (Desglose Matemático)

La clase principal `MotorTiempo` implementa una integración continua de variables discretas:

27.3.1 A. Integración del Avance Temporal (actualizar_tick)

A diferencia de un temporizador simple, el motor calcula el paso del tiempo simulado a través de una integración de Riemann, adaptándose a los cambios dinámicos del multiplicador de velocidad:

$$t_{sim}(t) = t_{sim}(t_0) + \int_{t_0}^t \alpha(\tau) d\tau \tag{38}$$

En el código discreto, esto se traduce en computar el delta de tiempo real en cada *Tick*, multiplicarlo por α , y sumarlo al reloj virtual, todo bloqueado tras un `threading.Lock()` para asegurar atomicidad.

27.3.2 B. Interpolación Sub-Milisegundo (tiempo_actual_absoluto)

Para eventos transitorios (ej. un cortocircuito que ocurre en un ciclo de $16,6ms$ a $60Hz$), el motor permite consultas de tiempo con precisión de microsegundos mediante el uso de `time.perf_counter()`, sirviendo de base matemática de fase (ωt) para las ondas sinusoidales en `motor_datos.py`.

27.3.3 C. Máquina de Estados y Pausa Estricta (congelar_tiempo)

Cuando el usuario pausa la simulación, el motor no detiene los hilos de la computadora (lo que congelaría la UI). Simplemente hace $\alpha = 0$, guardando la marca de tiempo de la interrupción. Al reanudar, recalcula el *offset* para que la simulación retome exactamente en el nanosegundo en que se detuvo.

27.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es el de un **Cronometrador Residente (Stateful Timekeeper)**. Instanciado al arrancar una nueva simulación, vive en la memoria de manera pasiva. No ejecuta bucles `while True` por sí mismo (para no acaparar núcleos de CPU). Es un ".^{Es}clavo Invocable"; el Controlador SCADA lo interroga en cada ciclo para preguntarle: "*¿Cuánto tiempo virtual ha pasado desde la última vez que te pregunté?*".

27.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como pilar del sub-espacio `simulador/`:

- Dicta el ritmo de `motor_datos.py`. Sin el `MotorTiempo`, la generación estocástica de telemetría no tendría un eje *X* (Tiempo) sobre el cual proyectar las Cadenas de Markov y la degradación de los sensores.

27.6 6. Relación Exocrina (Con el resto del Ecosistema)

Actúa como el **Sincronizador Universal** del Gemelo Digital:

- **Hacia `gestor_scada.py`:** Provee el Δt que el SCADA inyecta en el `motor_inferencias.py` para calcular la acumulación de calor (Joules = Watts $\times$ Segundos) en los cables a través del tiempo.
- **Hacia la Interfaz de Usuario (ui/):** Alimenta el reloj digital en la barra de estado de la aplicación, permitiendo al usuario ver el "Día, Mes y Hora" virtuales a medida que avanzan rápidamente.

27.7 7. Aporte al Funcionamiento Global Total

El aporte definitivo de `motor_tiempo.py` es la capacidad de **Predicción de Ciclo de Vida (Life-Cycle Forecasting) y Análisis Transitorio**.

Para que un software BIM y un Gemelo Digital sean útiles preventivamente, no pueden limitarse a simular el presente. Este motor permite a un ingeniero acelerar el tiempo ($\alpha = 31,536,000$) para observar cómo el aislamiento térmico de un cableado se degrada a lo largo de un año entero en tan solo 24 horas de computación. Brinda a *EcoCircuit* su Cuarta Dimensión, posibilitando el mantenimiento predictivo ciberfísico antes de invertir capital en construcción.

28 Ruta y Archivo: EcoCircuit/simulador/panel_simulacion.py

En la capa de interacción de *EcoCircuit Enterprise*, este archivo funciona como el **Puesto de Mando y Control Táctico (*Simulation Cockpit*)**. Es el componente de Interfaz Gráfica de Usuario (GUI) que expone las complejas variables de estado del motor de tiempo y del SCADA hacia el ingeniero humano, permitiendo la intervención manual (Human-In-The-Loop) sobre el flujo cronológico y termodinámico del Gemelo Digital.

28.1 1. Importaciones y Dependencias Estratégicas

La arquitectura de este panel exige librerías orientadas a eventos y renderizado de controles interactivos:

- **PySide6.QtWidgets:** Importación masiva de *Widgets* de control (QPushButton, QSlider, QLCDNumber, QProgressBar). Permite la construcción física de los botones de *Play*, *Pause*, *Fast-Forward*.
- **PySide6.QtCore (Signal, Slot, QTimer):** Fundamentales para establecer la comunicación asíncrona. El panel no invoca funciones del núcleo directamente bloqueando el hilo de ejecución; emite señales que son capturadas por los orquestadores subyacentes.
- **math y numpy:** Utilizados para realizar interpolaciones logarítmicas de los valores del deslizador (*Slider*) de aceleración temporal, suavizando la experiencia de usuario (UX).

28.2 2. Variables Globales y Metadatos de Interfaz

Define las tolerancias de actualización visual para no saturar la GPU:

- **UI_REFRESH_RATE_MS:** Frecuencia de refresco (típicamente $50ms$ a $100ms$). Aunque el motor SCADA pueda estar calculando 10,000 *ticks* por segundo, la pantalla LCD solo se actualiza visualmente 10 veces por segundo para preservar ciclos de CPU.
- **Estilos QSS (*Qt Style Sheets*):** Variables de colores semánticos (ej. Verde para simulación activa, Rojo para emergencias o paros del motor, Ámbar para pausa térmica).

28.3 3. Análisis de Clases y Funciones (Desglose Lógico)

La clase principal `PanelSimulacion(QWidget)` es el arquetipo de Controlador de Vistas. Se subdivide en tres dominios operacionales:

28.3.1 A. Topología del Panel y Controles (`_init_ui`)

Instancia la botonería y el *Display* cronológico. Configura un `QSlider` horizontal que permite al usuario modificar el factor de compresión temporal. Para evitar que el usuario pase de tiempo real (1x) a velocidad de simulación anual extrema (100,000x) de manera lineal abrupta, aplica una función de mapeo logarítmico:

$$\alpha(x) = 10^{\left(\frac{x}{x_{max}} \cdot C\right)} \quad (39)$$

Donde x es la posición lineal del *Slider*, convirtiéndola en un crecimiento exponencial paramétrico para el factor de aceleración α .

28.3.2 B. Máquina de Estados de Interfaz (`conmutar_estado`)

Gestiona la lógica booleana y la semántica visual de los botones. Si se presiona "Play", el método desactiva (`setEnabled(False)`) los botones de configuración de proyecto, cambia el ícono del botón a "Pause", y emite la señal `self.comando.simulacion.sig.emit("PLAY")`.

28.3.3 C. Bucle de Retroalimentación Sensorial (`actualizar_reloj_visual`)

Mediante un `QTimer` exclusivo de la UI, este método lee periódicamente el tiempo virtual desde `motor_tiempo.py`. Si la simulación está a 3600x, el usuario verá cómo los minutos y horas avanzan como un cronómetro de milisegundos. Calcula adicionalmente los *Frames Per Second* (FPS) lógicos y los muestra en la esquina inferior.

28.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es el de un **Componente Persistente Empotrado (*Embedded Dock Widget*)**. Es instanciado una única vez por el `WindowManager` o `main_window.py` al iniciar el programa. Suele ubicarse en la parte superior (como una cinta de opciones) o inferior de la pantalla principal. Subsiste en un estado de letargo hasta que el usuario hace clic o hasta que recibe confirmaciones de estado (*callbacks*) desde el SCADA.

28.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como la Çara Humana"de la carpeta `simulador/`:

- Es el envoltorio protector de `motor_tiempo.py` y `motor_datos.py`. Oculta por completo la complejidad de las integrales de Riemann y las Cadenas de Markov, exponiendo únicamente controles intuitivos de manipulación temporal al usuario.

28.6 6. Relación Exocrina (Con el resto del Ecosistema)

Opera como un **Comandante Asíncrono**:

- **Hacia controladores/gestor_scada.py:** Le ordena encender sus hilos paralelos (*Threads*). Es este panel el que tiene la autoridad de presionar el "Gatillo"que arranca la simulación de toda la infraestructura.
- **Hacia analitica.avanzada/:** Indirectamente, al detener la simulación desde este panel, se pausan las actualizaciones de las gráficas espectrales y estadísticas de Data Science, congelando la evidencia visual para su inspección pericial.

28.7 7. Aporte al Funcionamiento Global Total

El aporte de `panel_simulacion.py` es la **Ergonomía, Control Ciberfísico y Empoderamiento del Usuario**.

Por más potentes que sean los motores matemáticos subyacentes, sin una interfaz de control que permita acelerar el tiempo, pausar en un milisegundo exacto para evaluar un cortocircuito, o rebobinar una falla térmica, el Gemelo Digital sería inmanejable. Este panel transforma a *EcoCircuit* de una çaja negra"que ejecuta *scripts* en segundo plano, en un **Laboratorio Interactivo de Pruebas Electromecánicas**, consolidando la Experiencia de Usuario (UX) de grado *Enterprise*.

29 Ruta y Archivo: EcoCircuit/ui/canvas.py

Dentro del paradigma Modelo-Vista-Controlador de *EcoCircuit Enterprise*, este archivo constituye la **Vista Topológica Primaria y el Motor de Captura Geométrica (Plano 2D)**. Es el restirador digital donde el ingeniero interactúa físicamente con la plataforma, traduciendo gestos del ratón y comandos humanos en tensores matemáticos, grafos de red y objetos paramétricos que posteriormente conformarán el Gemelo Digital tridimensional.

29.1 1. Importaciones y Dependencias Estratégicas

La arquitectura de este módulo se fundamenta en la delegación del renderizado bidimensional a la GPU y el manejo riguroso de eventos de usuario:

- **PySide6.QtWidgets (QGraphicsView, QGraphicsScene):** Dependencias absolutas. El archivo no dibuja píxel por píxel; emplea el framework de la escena gráfica de Qt para manejar miles de elementos vectoriales de forma eficiente mediante árboles de partición espacial (BSP Trees).
- **PySide6.QtGui (QPainter, QTransform):** Requeridos para aplicar suavizado (*Antialiasing*) a las líneas de tensión y manipular las matrices matemáticas de cámara (Zoom y Pan).
- **core.model:** Importado para instanciar directamente los nodos lógicos cuando el usuario finaliza una operación de dibujo geométrico.

29.2 2. Variables Globales y Metadatos de Interacción

Establece las tolerancias físicas del cursor y los límites del universo dibujable:

- **GRID_SIZE:** Constante métrica (ej. $0.5m$) que define la rejilla imantada.
- **SNAP_TOLERANCE:** Radio euclidiano en píxeles. Si el cursor se acerca a un nodo a una distancia menor a este valor, el motor de gravedad geométrica fuerza la conexión.
- **Máquina de Estados (ModoCanvas):** Enumerador que rige la lógica de los clics: SELECCION, DIBUJO_MURO, ENRUTAMIENTO_CABLE, INSERCIÓN_EQUIPO.

29.3 3. Análisis de Clases y Funciones (Desglose Matemático)

La clase principal `CanvasBIM(QGraphicsView)` sobrescribe los eventos nativos del sistema operativo para aplicar su propia álgebra espacial:

29.3.1 A. Mapeo del Espacio Afín (Zoom y Paneo)

El canvas mantiene una separación estricta entre las coordenadas de la pantalla (píxeles) y las coordenadas de la escena (metros reales). Cuando el usuario hace *scroll* para aplicar Zoom, el motor recalcula la Matriz de Transformación Afín de 3×3 :

$$X_{escena} Y_{escena} 1 = S_x 0 T_x 0 S_y T_y 0 0 1^{-1} X_{pantalla} Y_{pantalla} 1 \quad (40)$$

Esto garantiza que la geometría subyacente (un cable de 10 metros) mantenga su validez termodinámica sin importar cuán grande se vea en el monitor.

29.3.2 B. Algoritmo de Gravedad Magnética (*Grid Snapping*)

Para evitar imprecisiones micrométricas humanas (ej. dibujar un muro en $X = 1.00004$), el método `_aplicar_snap(x, y)` redondea matemáticamente las coordenadas antes de cualquier inserción lógica:

$$X_{snap} = round\left(\frac{X}{G}\right) \times G \quad (41)$$

Donde G es el `GRID_SIZE`. Este proceso previene fugas de luz o "grietas" topológicas que destruirían la simulación térmica o la detección de colisiones 3D posteriormente.

29.3.3 C. Bucle de Captura Geométrica (Eventos de Ratón)

El método `mousePressEvent` actúa como una compuerta lógica. Si el estado es `ENRUTAMIENTO_CABLE`, un clic captura el $\vec{P}_1$ (origen). El `mouseMoveEvent` traza una línea virtual (*Ghost Line*). El segundo clic captura el $\vec{P}_2$ (destino). En ese instante, se calcula la Norma Euclidiana $L = \|\vec{P}_2 - \vec{P}_1\|$ y se inyecta la arista en el modelo.

29.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es de **Concurrencia Guiada por Eventos (*Event-Driven GUI Loop*)**. El Canvas no consume ciclos de CPU cuando el usuario no mueve el ratón. Reposa en la memoria esperando interrupciones de hardware (clics, teclado). Solo invoca la costosa función `paintEvent()` del `QPainter` cuando se altera la topología o cuando el `window_manager.py` redimensiona la ventana, garantizando una eficiencia energética óptima en el hilo principal.

29.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como el núcleo de la carpeta `ui/`:

- Trabaja en simbiosis con las barras de herramientas (`toolbars`) y el panel de propiedades (`inspector.py`). Cuando un elemento en el Canvas es seleccionado (disparando un evento de intersección *Raycast 2D*), emite una señal con el UUID del objeto para que el Inspector popule sus campos paramétricos.

29.6 6. Relación Exocrina (Con el resto del Ecosistema)

Es el **Gran Traductor de Intención Humana a Realidad Ciberfísica**:

- **Hacia `core/model.py`:** Transfiere las entidades geométricas validadas para que el Gestor de Proyecto las almacene en el Grafo de Memoria.
- **Hacia `controladores/gestor_3d.py`:** Actúa como la "Planta Arquitectónica". Cuando el usuario finaliza de dibujar una serie de líneas 2D, el Canvas notifica al motor 3D, el cual aplica las ecuaciones de extrusión para levantar los muros volumétricos instantáneamente.

29.7 7. Aporte al Funcionamiento Global Total

El aporte irremplazable de `canvas.py` es la **Ingesta de Topología y el Diseño Generativo Sensorial**.

El motor de Data Science y el simulador SCADA son entes puramente numéricos; no comprenden el concepto de "dibujar". Este archivo provee el medio físico (el pergamino digital) para que el ingeniero conciba espacialmente el edificio. Es la interfaz que cierra la brecha entre el diseño humano y el cálculo algorítmico masivo, convirtiendo a *EcoCircuit* en una herramienta intuitiva CAD sin sacrificar el rigor de una plataforma matemática de clase mundial.

30 Ruta y Archivo: EcoCircuit/ui/dialogos_electricos.py

Dentro del ecosistema de interacción de *EcoCircuit Enterprise*, este archivo funciona como el **Filtro de Ingesta Paramétrica y Controlador de Ventanas Modales**. Su propósito es interceptar la intención del usuario al configurar propiedades complejas de ingeniería (como dimensionar un transformador o ajustar curvas de disparo de interruptores) a través de cuadros de diálogo interactivos (*Pop-ups*), garantizando que ninguna variable físicamente imposible o normativamente inválida alcance el núcleo de la simulación.

30.1 1. Importaciones y Dependencias Estratégicas

La arquitectura de este módulo se apoya fuertemente en formularios estructurados y validadores de datos:

- **PySide6.QtWidgets** (`QDialog`, `QFormLayout`, `QDoubleSpinBox`, `QComboBox`): Dependencias fundamentales para construir interfaces de captura de datos bloqueantes (*Modals*) que suspenden temporalmente el flujo de la ventana principal hasta que el usuario confirme o cancele la operación.
- **core.catalogo_electrico**: Importado para hidratar dinámicamente las listas desplegables. Asegura que el ingeniero solo pueda seleccionar calibres de cable (ej. AWG 10, 1/0) o voltajes nominales que existan en el mercado real.
- **PySide6.QtGui** (`QDoubleValidator`, `QRegularExpressionValidator`): Empleados para aplicar restricciones criptográficas a nivel de teclado, impidiendo que el usuario ingrese texto en campos de amperaje o valores negativos en campos de impedancia.

30.2 2. Variables Globales y Metadatos de Validación

Define las reglas de negocio en la frontera visual:

- **Umbral Operativo Absoluto**: Diccionarios que establecen los límites lógicos de entrada. Por ejemplo, $V_{min} = 12V$, $V_{max} = 115000V$; o Factores de Potencia restringidos al intervalo estricto $FP \in (0,0, 1,0]$.
- **Caché Transaccional**: Variables temporales que almacenan el estado del diálogo antes de ser confirmado, permitiendo un *Rollback* perfecto si el usuario presiona “Cancelar”.

30.3 3. Análisis de Clases y Funciones (Desglose Lógico)

El archivo alberga múltiples clases que heredan de `QDialog`, cada una especializada en un subdominio termodinámico o eléctrico:

30.3.1 A. Topología de Formularios (`DialogoConfiguracionCarga`)

Construye la matriz visual de captura de datos. Al instanciarse para un motor de inducción, genera un `QFormLayout` que solicita Potencia Nominal (P), Factor de Potencia ($\cos \phi$) y Eficiencia Mecánica (η). Internamente, el diálogo pre-calcula métricas de solo lectura para el usuario utilizando derivadas de la Ley de Watt:

$$I_{estimada} = \frac{P}{\sqrt{3} \cdot V_{red} \cdot \cos(\phi) \cdot \eta} \tag{42}$$

Mostrando este valor en tiempo real a medida que el usuario ajusta los parámetros (Retroalimentación Inmediata).

30.3.2 B. Compuertas Lógicas y Validación de Estado (`accept()`)

Sobrescribe el método nativo de confirmación. Antes de cerrar el cuadro de diálogo y enviar los datos al motor, ejecuta una auditoría matemática cruzada:

$$Valido = \{ \textit{True}, \textit{si} I_{protección} \geq 1,25 \times I_{carga} \textit{False}, \textit{encasocontrario}(\textit{DisparaunQMessageAlert}) \} \tag{43}$$

Si la lógica normativa falla, el diálogo se niega a cerrarse, forzando al ingeniero a corregir la discrepancia térmica o de cortocircuito.

30.3.3 C. Inyección de Catálogos (`popular_menus`)

Realiza consultas en tiempo $O(1)$ al Singleton de `catalogo_electrico.py` para llenar los `QComboBox`. Transforma la experiencia de usuario de .entrada libre.^a ”selección discreta”, reduciendo la entropía del sistema.

30.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Effímero y Bloqueante** (*Ephemeral Modal Lifecycle*). A diferencia del Canvas o el Gestor 3D que viven perpetuamente en la memoria RAM, los diálogos nacen y mueren bajo demanda. Cuando un ingeniero hace doble clic en un Transformador en el `canvas.py`, el diálogo se instancia, captura el hilo principal de la UI (*event loop blocking*), transfiere la información estructurada mediante un diccionario, y luego es destruido por el *Garbage Collector* al cerrarse.

30.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como módulo táctico en `ui/`:

- Es el hermano detallista del `inspector.py`. Mientras que el inspector permite ediciones rápidas y superficiales (ej. cambiar un nombre), los `dialogos_electricos.py` se invocan para operaciones profundas que requieren asistentes visuales (*Wizards*) y matemáticas en tiempo real.

30.6 6. Relación Exocrina (Con el resto del Ecosistema)

Actúa como la **Aduana Ciberfísica**:

- **Hacia `core/model.py`**: Es el principal emisor de mutaciones de propiedades eléctricas complejas. Empaqueta las decisiones del usuario y las envía al controlador de proyecto para actualizar el nodo en el grafo.
- **Hacia `controladores/gestor_scada.py`**: Al blindar la entrada de datos, garantiza que el simulador SCADA jamás reciba, por error tipográfico, un cable con una resistencia de -5Ω , lo cual causaría una violación matemática (y un colapso del sistema de ecuaciones) durante la simulación de Flujos de Carga.

30.7 7. Aporte al Funcionamiento Global Total

El aporte irremplazable de `dialogos_electricos.py` es la **Prevención de Corrupción Semántica** (*Garbage-In, Garbage-Out Prevention*).

El motor matemático más avanzado del mundo es inútil si el humano que lo opera introduce parámetros imposibles. Este archivo traslada la carga de validación desde los profundos motores matemáticos hacia la frontera visual. Al obligar al usuario a diseñar dentro de los confines de la física y las normativas reales antes de aceptar el *input*, consagra la robustez del Gemelo Digital, asegurando que las simulaciones resultantes sean siempre deterministas, creíbles y certificables.

31 Ruta y Archivo: EcoCircuit/ui/inspector_handler.py

Dentro del paradigma visual de *EcoCircuit Enterprise*, este archivo actúa como el **Motor de Vinculación Bidireccional y Reflexión de Propiedades (*Property Inspector* / *Two-Way Data Binding Engine*)**. Es el microscopio analítico de la interfaz de usuario: captura los objetos abstractos seleccionados en el Canvas 2D o el Visor 3D, descompone sus atributos matemáticos y los expone en un formulario dinámico que permite la manipulación granular por parte del ingeniero.

31.1 1. Importaciones y Dependencias Estratégicas

La arquitectura de este módulo requiere librerías para la generación procedimental de interfaces y la interceptación de señales:

- **PySide6.QtWidgets** (QDockWidget, QFormLayout, QDoubleSpinBox, QLineEdit): Dependencias para la construcción del panel acoplable (*Dock*) y los campos de entrada de datos (*Widgets*).
- **PySide6.QtCore** (Signal, Slot, QObject): Fundamentales para establecer el patrón *Observer*. Permiten que el Inspector reaccione en microsegundos a un evento de clic en el lienzo o viceversa.
- **core.project_manager**: Importado para solicitar la lectura y escritura segura de las propiedades en el Grafo Electromecánico en Memoria (**model.py**), respetando el encapsulamiento.

31.2 2. Variables Globales y Metadatos de Estado

Gestiona la memoria volátil del contexto de selección:

- **entidad_actual_uuid**: Almacena la firma criptográfica (UUID) del elemento actualmente enfocado. Previene que el motor redibuje costosos formularios si el usuario hace clic múltiples veces sobre el mismo transformador.
- **Diccionario de Mapeo de Tipos (*Type Mapping*)**: Una matriz de rutas lógicas que asocia un tipo de dato subyacente con un elemento gráfico. Ej: *Booleano* $\rightarrow$ *QCheckBox*, *Flotante* $\rightarrow$ *QDoubleSpinBox*.

31.3 3. Análisis de Clases y Funciones (Desglose Matemático)

La clase **InspectorHandler** implementa una interfaz mutante (*Dynamic Form Generation*) que adapta su estructura física al objeto seleccionado:

31.3.1 A. Reflexión Dinámica y Generación de Vistas (_poblar_formulario)

Cuando se selecciona una entidad, este método destruye el *layout* anterior y emplea un algoritmo de reflexión (introspección) para leer los atributos del objeto (ej. Potencia, Material, Fases). Mapea el tipo de atributo hacia un control gráfico:

$$f_{UI_map}(P_i) = \{ \textit{QComboBox}, si P_i \in Enum(ej.AWG, Material) \textit{QDoubleSpinBox}, si P_i \in R(ej.Amperaje, Longitud) \textit{QLineEdit}, si P_i \in String(ej.Nombre del Tablero) \} \quad (44)$$

Generando el formulario con complejidad $O(A)$, donde A es la cantidad de atributos expuestos de la entidad.

31.3.2 B. Rompimiento de Bucles Recursivos (*Signal Blocking*)

Un riesgo matemático en la vinculación bidireccional (*Two-Way Binding*) es la resonancia infinita: El Modelo actualiza la UI $\rightarrow$ La UI detecta un cambio y notifica al Modelo $\rightarrow$ El Modelo actualiza la UI... hasta desbordar la pila (*Stack Overflow*). El método `_actualizar_desde_modelo()` implementa la instrucción `widget.blockSignals(True)` antes de inyectar el valor numérico, asegurando la siguiente lógica de exclusión:

$$\textit{Actualizar}(M \rightarrow UI) \textit{Bloquear_Señal}(UI \rightarrow M) \quad (45)$$

Liberando las señales inmediatamente después de la escritura.

31.3.3 C. Validación y Escritura Atómica (_commit_cambio)

Cuando el usuario modifica un valor en el Inspector, este método captura la señal `editingFinished` o `valueChanged`. Empaqueta la nueva variable y, en lugar de alterar el modelo directamente, emite un comando formal al **project_manager.py** asegurando que la acción se registre en el sistema de Deshacer/Rehacer (*Undo/Redo Stack*).

31.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Persistente, pero de Estado Efímero (*Persistent Shell, Ephemeral State*)**. El contenedor (el panel físico a la derecha de la pantalla) nace con la aplicación y sobrevive hasta que se cierra. Sin embargo, su contenido interno es volátil; es destruido por el Recolector de Basura (Garbage Collector) y reconstruido desde cero cada vez que el foco de selección del usuario cambia de un elemento a otro.

31.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como pilar en `ui/`:

- **Con `canvas.py`**: Es el receptor pasivo de las señales de selección (ej. `nodo_seleccionado_sig`).
- **Con `dialogos_electricos.py`**: Mientras el Inspector maneja ediciones "superficiales" continuas de atributos base, aloja botones que invocan a los diálogos modales para configuraciones matemáticas mucho más profundas (ej. botón "Configurar Curva de Disparo").

31.6 6. Relación Exocrina (Con el resto del Ecosistema)

Actúa como el **Bisturí del Arquitecto**:

- **Hacia `controladores/gestor_3d.py`**: Cuando un usuario cambia el calibre de un cable en el Inspector, la señal de cambio fluye hasta el motor 3D obligándolo a recalcular el radio (*Radius*) del cilindro y actualizar el renderizado del tubo espacial en milisegundos.
- **Hacia el `gestor_scada.py`**: Si el usuario modifica la potencia nominal de un motor en plena simulación en vivo, el Inspector canaliza este nuevo escalar termodinámico, obligando al SCADA a recalcular el flujo de carga al instante.

31.7 7. Aporte al Funcionamiento Global Total

El aporte de `inspector_handler.py` es la **Manipulación Granular y la Transparencia Paramétrica**.

Un plano CAD o un visor 3D solo muestran la "Piel" la "Posición" de la infraestructura. El Inspector es la herramienta que revela la "Genética" matemática de cada componente. Al proveer una interfaz dinámica, bidireccional y blindada contra ciclos infinitos, permite a los ingenieros ajustar parámetros termodinámicos, eléctricos y de metadatos de manera precisa, convirtiendo un simple dibujo vectorial en una verdadera Base de Datos Espacial (BIM).

32 Ruta y Archivo: EcoCircuit/ui/inspector_window.py

En la arquitectura de Interfaz de Usuario (UI) de *EcoCircuit Enterprise*, este archivo constituye la **Envolvente Visual** y el **Contenedor Físico Topológico** (***Dockable Window Shell***). Mientras que `inspector_handler.py` (analizado previamente) funge como el “cerebro” matemático que extrae y vincula los datos, `inspector_window.py` actúa como el “cuerpo” físico. Su responsabilidad es alojar los formularios dinámicos, gestionar el desbordamiento visual (*Overflow*) mediante áreas de desplazamiento, y manejar las físicas de acoplamiento (*Docking*) dentro del espacio de trabajo principal.

32.1 1. Importaciones y Dependencias Estratégicas

Su diseño exige librerías enfocadas estrictamente en la manipulación geométrica de ventanas y jerarquías de *Widgets*:

- `PySide6.QtWidgets` (`QDockWidget`, `QScrollArea`, `QVBoxLayout`, `QWidget`): Componentes estructurales. El `QDockWidget` permite que el panel sea arrastrado, desprendido (*Torn-off*) o anclado a los bordes de la pantalla. El `QScrollArea` es crítico para prevenir la inaccesibilidad a parámetros inferiores.
- `PySide6.QtCore` (`Qt`): Requerido para invocar banderas de estado a nivel bit, definiendo qué áreas de anclaje están permitidas (ej. `Qt.LeftDockWidgetArea | Qt.RightDockWidgetArea`).

32.2 2. Variables Globales y Metadatos de Disposición

Define las tolerancias de renderizado de la envoltura gráfica:

- `MIN_WIDTH_INSPECTOR`: Constante (ej. $320px$) que impide al motor de ventanas aplastar los campos numéricos más allá de la legibilidad humana, imponiendo una restricción matemática al dimensionamiento del *layout*.
- **Políticas de Desplazamiento** (***Scroll Policies***): Banderas inmutables como `ScrollBarAsNeeded` en el eje Y , y `ScrollBarAlwaysOff` en el eje X , forzando a los elementos internos a encogerse horizontalmente en lugar de desbordarse.

32.3 3. Análisis de Clases y Funciones (Desglose Estructural)

La clase principal `InspectorWindow` hereda de `QDockWidget` e implementa una topología de muñecas rusas (*Matryoshka Topology*):

32.3.1 A. Topología de Contención (`_construir_esqueleto`)

El archivo construye un árbol de jerarquía visual estricto para evitar el colapso de renderizado cuando se inyectan formularios de 50 o más parámetros termodinámicos. La estructura obedece la siguiente jerarquía de punteros:

$$QDockWidget \rightarrow QScrollArea \rightarrow Contenedor_Maestro(QWidget) \rightarrow QVBoxLayout \tag{46}$$

Al aislar el contenedor maestro dentro del `QScrollArea`, el algoritmo asegura que:

$$Si H_{contenido} > H_{pantalla}.Activar_Barra(Y, \Delta h) \tag{47}$$

Garantizando el acceso ergonómico integral.

32.3.2 B. Físicas de Desprendimiento (`_manejar_flotabilidad`)

Gestiona la transición de estado booleana `isFloating()`. Cuando el ingeniero arrastra el inspector fuera de la ventana principal, el objeto recalcula sus sombras, sus bordes y solicita al gestor de ventanas del Sistema Operativo (Windows DWM o macOS Quartz) un marco independiente (*OS Window Frame*) con su propio botón de cierre.

32.3.3 C. Inyección Dinámica (`montar_formulario`)

Es la compuerta de entrada para el `inspector_handler.py`. Recibe el bloque *Layout* generado procedimentalmente y lo inserta en su `Contenedor_Maestro`. Destruye (invocando `deleteLater()`) de forma segura cualquier *widget* residual del nodo eléctrico seleccionado anteriormente para prevenir fugas de memoria gráfica (*VRAM Leaks*).

32.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Persistente a lo largo de la Sesión** (***Session-Persistent Shell***). Es instanciado en el milisegundo en que la `MainWindow` es construida. Puede ser ocultado (cambiando su estado a `setVisible(False)`), pero no se destruye de la memoria RAM. Sobrevive a la apertura y cierre de múltiples proyectos, actuando como un receptáculo inmutable a la espera de nuevos datos.

32.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como componente de la carpeta `ui/`:

- **Con `inspector_handler.py`**: Mantienen una relación simbiótica estricta de Contenedor/Contenido. El `window` no sabe de ingeniería eléctrica ni de grafos (desacoplamiento total); solo sabe “mostrar botones”. El `handler` no sabe de barras de desplazamiento; solo sabe “vincular matemáticas”.

32.6 6. Relación Exocrina (Con el resto del Ecosistema)

Actúa como el **Módulo de Enganche Arquitectónico**:

- **Hacia `ui/main_window.py`**: Se acopla a la ventana principal a través del método `addDockWidget(Qt.RightDockWidgetArea, self)`.
- **Hacia `core/window_manager.py`**: Expone su nombre de objeto único (`setObjectName("InspectorBIM")`) para que el gestor de ventanas pueda serializar su ancho en píxeles y su posición de anclaje al cerrar el programa, restaurándolo milimétricamente en la siguiente sesión.

32.7 7. Aporte al Funcionamiento Global Total

El aporte de `inspector_window.py` es la **Estabilidad de la Interfaz de Usuario y la Flexibilidad Ergonómica (UX)**.

Un Gemelo Digital paramétrico puede contener subestaciones eléctricas con docenas de atributos simultáneos (materiales, eficiencias, factores de potencia, coordenadas 3D). Sin una ventana dotada de físicas de desplazamiento (*Scrolling*) y capacidades de desprendimiento (*Floating Multi-Monitor Support*), el usuario no tendría espacio físico en pantalla para auditar el equipo. Este módulo dota a *EcoCircuit* de la usabilidad elástica que exigen los entornos de ingeniería industrial moderna.

33 Ruta y Archivo: EcoCircuit/ui/level_manager.py

Dentro de la arquitectura de *EcoCircuit Enterprise*, este archivo funge como el **Gestor Topológico de Elevación y Segmentación Espacial (*Floor/Level Manager*)**. Su propósito fundamental es domar la complejidad tridimensional (el Eje Z) presentándola en una interfaz jerárquica bidimensional comprensible para el ingeniero. Controla los planos de corte visuales, aislando matemáticamente pisos específicos de un edificio para permitir el diseño electromecánico sin la superposición caótica de redes multicapa.

33.1 1. Importaciones y Dependencias Estratégicas

La arquitectura de este módulo requiere librerías para el manejo de listas interactivas y emisión de eventos de cámara:

- **PySide6.QtWidgets (QListWidget, QDockWidget, QPushButton):** Utilizados para renderizar el panel acoplable que enlista los niveles (ej. "Sótano", "Planta Baja", "Nivel 1") y los botones de operaciones espaciales (Añadir/Eliminar Nivel).
- **PySide6.QtCore (Signal, Slot):** Dependencia crítica. Almacena la señal `nivel_cambiado.sig`, la cual actúa como un pulso electromagnético (*Broadcast*) a través de todo el software para redibujar el Canvas y el motor 3D.
- **core.project_manager:** Inyectado como dependencia para consultar la estructura de árbol del edificio y obtener las elevaciones relativas de cada losa estructural.

33.2 2. Variables Globales y Metadatos de Segmentación

Define las reglas paramétricas para la creación de nuevos planos de trabajo:

- **ALTURA_NIVEL_DEFECTO:** Una constante métrica estandarizada (ej. $3,20m$). Si el usuario añade un "Nivel 2" sin especificar altura, el sistema asume esta distancia ortogonal desde el nivel subyacente.
- **Caché de Puntero Activo (nivel_activo_uuid):** Variable volátil que almacena la firma criptográfica del piso en el que el ingeniero tiene enfocada su atención actualmente.

33.3 3. Análisis de Clases y Funciones (Desglose Matemático)

La clase principal `LevelManager(QDockWidget)` actúa como un filtro discriminador de coordenadas Z :

33.3.1 A. Lógica de Apilamiento Espacial (`crear_nuevo_nivel`)

Cuando el usuario presiona el botón "Añadir Nivel", la interfaz no simplemente agrega un texto a una lista. Ejecuta un algoritmo de apilamiento integrativo para calcular la elevación global Z_n del nuevo piso respecto al origen $(0, 0, 0)$:

$$Z_n = Z_0 + \sum_{i=1}^{n-1} h_i \quad (48)$$

Donde h_i es la altura individual (entrepiso) de cada nivel inferior. Esta elevación absoluta se encapsula y se envía al `model.py` para instanciar la entidad `NivelBIM`.

33.3.2 B. Conmutación de Contexto (Context Switching)

El método `_al_seleccionar_nivel(item)` se dispara al hacer clic en la lista. Evalúa el nuevo UUID seleccionado y emite un tensor de corte $[Z_{min}, Z_{max}]$. Define un filtro booleano espacial de visibilidad para cualquier nodo electromecánico k :

$$V(k) = \{ \textit{True}, si Z_k \in [Z_{piso}, Z_{piso} + h_{piso}) \textit{False}, encasocontrario \} \quad (49)$$

Garantizando que un contacto eléctrico en el Nivel 2 no se dibuje por error cuando el usuario está diseñando la Planta Baja.

33.3.3 C. Destrucción en Cascada (`eliminar_nivel`)

Invoca una advertencia crítica (*Warning Dialog*). Si el usuario elimina un piso, el Gestor de Niveles exige al Modelo Central una destrucción topológica profunda (Cascading Delete) de complejidad $O(V_{piso} + E_{piso})$. Todo tablero, cable y muro asociado genéticamente a ese nivel es borrado de la memoria RAM para prevenir Nodos Huérfanos (*Orphan Nodes*).

33.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Permanente e Interactivo (*Persistent Session Widget*)**. Se instancia como un panel acoplable junto a `inspector_window.py`. Su estado (la lista de niveles) es destruido y reconstruido íntegramente cada vez que el `project_manager.py` carga un archivo `.eco` nuevo desde el disco duro, pero su estructura de control permanece viva mientras la aplicación esté abierta.

33.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como supervisor de visibilidad dentro de `ui/`:

- **Control absoluto sobre canvas.py:** El lienzo 2D carece de voluntad propia en el eje Z . Dibuja exclusivamente los elementos cuyo UUID pertenece al 'nivel_{activo}_{uuid}' dictado por `este_level_manager.py`.

33.6 6. Relación Exocrina (Con el resto del Ecosistema)

Actúa como el **Plano Secante (Clipping Plane Commander)** del renderizador volumétrico:

- **Hacia controladores/gestor_3d.py:** Cuando se cambia de nivel, este gestor le envía una señal al motor 3D para que aplique un *Clipping Box* algorítmico (mediante `vtkClipPolyData`). Esto corta visualmente el edificio por la mitad (como en un plano arquitectónico), ocultando el techo y los pisos superiores para permitir la inspección cenital sin obstrucciones.
- **Hacia core/exportador_mep.py:** Define la estructura `IfcBuildingStorey`. Sin este gestor de niveles, el archivo IFC exportado sería un solo bloque de geometría caótica, inutilizable para herramientas como Autodesk Revit.

33.7 7. Aporte al Funcionamiento Global Total

El aporte de `level_manager.py` es la **Jerarquización Estructural y el Orden Cognitivo**.

En un hospital de 15 pisos, pueden existir más de 50,000 cables y 10,000 dispositivos. Intentar visualizar y calcular topologías eléctricas en un solo espacio continuo provocaría una sobrecarga cognitiva insostenible para el usuario y un solapamiento inmanejable en la pantalla 2D. Este módulo divide un problema masivo en subconjuntos planimétricos (*Divide and Conquer*), permitiendo que *EcoCircuit* sea viable no solo para viviendas pequeñas, sino para infraestructuras verticales masivas de grado *Enterprise*.

34 Ruta y Archivo: EcoCircuit/ui/main_window.py

En la cúspide de la jerarquía de presentación de *EcoCircuit Enterprise*, este archivo funge como el **Marco de Ensamblaje Topológico y Orquestador Supremo de Vistas (*Root GUI Container & Mediator*)**. Heredando de la clase `QMainWindow`, es la frontera límite física del programa; actúa como la "Placa Madre" (Motherboard) visual donde se conectan, dimensionan y comunican todos los demás paneles, lienzo y barras de herramientas analizados en los capítulos anteriores.

34.1 1. Importaciones y Dependencias Estratégicas

La arquitectura de este módulo requiere la consolidación masiva de casi todo el ecosistema de la interfaz gráfica y los gestores núcleo:

- **PySide6.QtWidgets** (`QMainWindow`, `QMenuBar`, `QStatusBar`, `QDockWidget`): Dependencias cardinales para instanciar la envoltura del Sistema Operativo, las barras de menús nativas y el área central de trabajo (`Central Widget`).
- **PySide6.QtGui** (`QAction`, `QKeySequence`, `QCloseEvent`): Para el mapeo de atajos de teclado globales (ej. *Ctrl+S*, *Ctrl+Z*) y la intercepción de señales críticas del hardware.
- **Módulos Internos** (`ui.canvas`, `ui.inspector_window`, `ui.level_manager`): Importados como subcomponentes para ser instanciados y anclados matricialmente en las diferentes zonas de la ventana.
- `core.project_manager`, `core.window_manager`: Dependencias del dominio inyectadas para enlazar las acciones del usuario con los motores lógicos y guardar el estado espacial.

34.2 2. Variables Globales y Metadatos de Aplicación

Establece los tensores y vectores primarios del entorno de escritorio:

- **Constantes de Geometría Base**: Dimensiones de arranque predeterminadas (ej. 1280×720 píxeles) para asegurar que la aplicación sea utilizable en monitores de menor resolución antes de que el `window_manager.py` restaure el perfil de un usuario avanzado.
- **Nombre y Firma del Software**: Cadenas de texto estáticas que alimentan la barra de título, concatenando dinámicamente la versión actual y la ruta del archivo activo (ej. *EcoCircuit Enterprise v4.6 - [Hospital.Central.eco]*).

34.3 3. Análisis de Clases y Funciones (Desglose Estructural)

La clase principal `MainWindow(QMainWindow)` implementa el patrón de diseño *Mediator*, evitando que los subcomponentes se acoplen directamente entre sí:

34.3.1 A. Topología de Pantalla y Partición Espacial (`_ensamblar_layout`)

El método de inicialización no emplea un *Layout* convencional, sino que divide el área de la pantalla en un Espacio Euclidiano central y 4 cuadrantes perimetrales de anclaje (*Dock Areas*). Matemáticamente, asegura que el área total A_T se distribuya como:

$$A_T = A_{Canvas_Central} + \sum_{i=1}^n A_{Dock_i} \quad (50)$$

Fuerza al `canvas.py` a ser el `CentralWidget` (ocupando siempre el máximo espacio remanente) y acopla el `inspector_window.py` en el límite vectorial Este (`Qt.RightDockWidgetArea`).

34.3.2 B. Enrutamiento de Acciones y Señales (`_conectar_senales`)

Actúa como un enrutador (Router) de grafos de eventos. Mapea la intención del usuario hacia el servicio subyacente. Define una matriz de adyacencia de interacciones:

$$M_{eventos} = Acci3n_GuardarinvocaGestorIO.guardar()Clic_Zoom_3DinvocaGestor3D.enfocar_camara()Se3al_ErrorinvocaMainWindow.mostrar_alerta() \quad (51)$$

Garantizando que un botón en la barra de herramientas no modifique la base de datos directamente, sino a través de la vía arquitectónica legal.

34.3.3 C. Intercepción del Protocolo de Cierre (`closeEvent`)

Sobrescribe el evento de destrucción dictado por el sistema operativo. Antes de permitir que Windows o macOS aniquilen el proceso de la RAM, evalúa la variable booleana *is_dirty* del `project_manager.py`:

$$f_{cierre} = \{ \textit{GuardarySalir}, \textit{siis_dirty} = \textit{TrueDestruir(inmediato)}, \textit{siis_dirty} = \textit{False} \quad (52)$$

También invoca al `window_manager.py` para serializar las coordenadas exactas de la ventana. Solo si estos procesos devuelven éxito, ejecuta `event.accept()`, de lo contrario, cancela el cierre de la aplicación con `event.ignore()`.

34.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida abarca la **Totalidad de la Existencia Interactiva (*Main Event Loop*)**. Es el primer objeto visual instanciado por `main.py` tras inicializar `QApplication`, y es el último en ser destruido. Mantiene con vida el *Event Loop* (bucle de eventos infinito) que permite a la aplicación escuchar el ratón, el teclado y los *ticks* del SCADA a 60 fotogramas por segundo.

34.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como Rey soberano de la carpeta `ui/`:

- Es el "Padre" (*Parent Object*) en el árbol de punteros de C++. Es dueño de la memoria del Inspector, el Nivelador, el Canvas y los diálogos modales. Si la `MainWindow` es destruida, el framework Qt garantiza la aniquilación en cascada de todos los *widgets* hijos, evitando *Memory Leaks*.

34.6 6. Relación Exocrina (Con el resto del Ecosistema)

Es el **Cuerpo Físico del Gemelo Digital**:

- **Hacia core/ y controladores/**: Aunque contiene toda la interfaz, la `MainWindow` está matemáticamente vacía de conocimientos de ingeniería. Su existencia se limita puramente a alojar los píxeles y delegar las matemáticas pesadas al `MotorInferencias` y al `GestorSCADA`.

34.7 7. Aporte al Funcionamiento Global Total

El aporte de `main_window.py` es la **Cohesión, Unificación y Puntos de Acceso (Entry-point UI)**.

Sin este archivo, todos los geniales sistemas matemáticos, renders en 3D, inspectores paramétricos y analítica espectral flotarían en el sistema operativo como docenas de ventanas desconectadas y caóticas. Este módulo provee el andamiaje (*Scaffolding*) estructural para unificar la complejidad ciberfísica en un único programa comprensible, armónico y profesional, consagrando la integridad visual del producto final que el ingeniero humano consumirá.

35 Ruta y Archivo: EcoCircuit/ui/menu_bar.py

Dentro de la estructura de interfaz de *EcoCircuit Enterprise*, este archivo funge como el **Enrutador Global de Comandos y Árbol de Navegación Jerárquica** (***Command Dispatcher & Navigation Tree***). Materializado como la barra superior clásica de aplicaciones de escritorio (Archivo, Editar, Ver, Simulación, Ayuda), su propósito arquitectónico es abstraer y categorizar todas las acciones posibles del Gemelo Digital, mapeando las interrupciones de hardware (atajos de teclado) a señales de software sin contener una sola línea de lógica de negocios.

35.1 1. Importaciones y Dependencias Estratégicas

La pureza de este módulo reside en su estricta separación de responsabilidades (MVC). Solo requiere librerías para la construcción de interfaces semánticas:

- **PySide6.QtWidgets (QMenuBar, QMenu):** Los contenedores estructurales base. Permiten organizar las opciones en un grafo acíclico dirigido de menús y submenús.
- **PySide6.QtGui (QAction, QKeySequence, QIcon):** Dependencias críticas para la instanciación de comandos atómicos. **QKeySequence** es especialmente vital, ya que estandariza las pulsaciones del teclado independientemente del sistema operativo (*Ctrl* en Windows vs *Cmd* en macOS).

35.2 2. Variables Globales y Metadatos de Estructura

Define la taxonomía inmutable de la plataforma:

- **Diccionarios de Atajos (*Shortcut Maps*):** Estructuras de datos que vinculan cadenas semánticas con combinaciones de teclas, garantizando homogeneidad (ej. `SHORTCUT_GUARDAR = QtCtrl+S"`, `SHORTCUT_NUEVO = QtCtrl+N"`).
- **Nomenclatura Internacionalizada:** Cadenas de texto preparadas para los identificadores de traducción (*i18n*), permitiendo que la interfaz pueda mutar de idioma (ej. `tr("File") → .Archivo`) sin alterar los punteros lógicos.

35.3 3. Análisis de Clases y Funciones (Desglose Matemático)

La clase `MenuBarBIM(QMenuBar)` se comporta como una Matriz de Transformación, convirtiendo la intención difusa del usuario en vectores de acción concretos:

35.3.1 A. Topología de Árbol de Acciones (`_construir_jerarquia`)

El menú no es una lista plana, sino un árbol topológico. El método construye los nodos padre (Archivo, Edición) y sus hijos (Guardar, Deshacer). Matemáticamente, define un mapeo de funciones:

$$M_{root} = \bigcup_{i=1}^n Menu_i \quad \text{donde } Menu_i = \{A_1, A_2, \dots, A_m\} \tag{53}$$

Donde cada acción A_k es un objeto `QAction` aislado, dotado de un puntero en memoria listo para disparar una interrupción asíncrona.

35.3.2 B. Mapeo de Interrupciones de Hardware (`_vincular_atajos`)

Implementa un observador global de teclado. El motor asocia una secuencia física matricial del teclado (**QKeySequence**) con una señal de software subyacente (**triggered**).

$$f_{evento_teclado}(Tecla_i \wedge Modificador_j) QAction_k.emit() \tag{54}$$

Garantizando que presionar *Ctrl+E* (Exportar) desde cualquier parte de la aplicación dispare la misma función que navegar con el ratón por el árbol visual.

35.3.3 C. Inyección de Dependencias y Despacho (`_conectar_mediator`)

El menú **no** realiza las acciones. Si el usuario hace clic en ".Exportar IFC ", el menú no invoca al `exportador_mep.py`. En su lugar, emite una señal que es atrapada por su clase padre (`main_window.py`), respetando el Principio de Responsabilidad Única (SRP):

$$MenuBarSeñal : "Exportar_IFC_Solicitado" MainWindowInvocaExportadorMEP \tag{55}$$

35.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Estático y Persistente (Application-Lifetime Object)**. Se instancia durante la construcción del constructor de la `MainWindow`. Una vez que el árbol lógico (`QMenu`) se adjunta a la barra superior del sistema operativo, su estructura no cambia, aunque sus acciones pueden ser deshabilitadas (`setEnabled(False)`) dinámicamente si el estado de la simulación así lo exige (ej. bloquear el botón "Guardar" mientras se está ejecutando un flujo de carga de corto circuito).

35.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como componente satélite dentro de `ui/`:

- **Con `main_window.py`:** Es un órgano vital empotrado físicamente en la cabeza de la ventana principal mediante `self.setMenuBar()`. La ventana actúa como el receptor (Listener) central de todas las señales disparadas por esta barra.

35.6 6. Relación Exocrina (Con el resto del Ecosistema)

Opera como el **Directorio de Servicios (*Service Discovery*)**:

- Para el ingeniero usuario, es el mapa que revela todas las capacidades del *EcoCircuit*. Oculta la complejidad de decenas de sub-módulos (`simulador/`, `core/`, `analitica_avanzada/`) agrupándolos en categorías semánticas lógicas (ej. agrupar `PanelFFT` y `PanelSPC` bajo el menú "Data Science").

35.7 7. Aporte al Funcionamiento Global Total

El aporte irremplazable de `menu_bar.py` es la **Descubribilidad (Discoverability)** y la **Eficiencia Operativa (Power User Workflow)**.

Una arquitectura de software puede tener los motores termodinámicos más avanzados del planeta, pero si el usuario no puede encontrar el botón para ejecutarlos, la herramienta fracasa. Este módulo garantiza que la totalidad del inmenso ecosistema de *EcoCircuit* sea navegable intuitivamente. A su vez, al centralizar el mapeo de atajos de teclado, multiplica la velocidad de operación del usuario experto, elevando la plataforma al estándar ergonómico que exigen las soluciones de grado *Enterprise*.

36 Ruta y Archivo: EcoCircuit/simulador/section_manager.py

Dentro del núcleo termodinámico y eléctrico de *EcoCircuit Enterprise*, este archivo opera como el **Gestor de Particionamiento Topológico y Aislamiento de Red (*Grid Islanding & Section Manager*)**. Su propósito ciberfísico es monitorear el estado de los seccionadores (interruptores, fusibles, *breakers*) e identificar dinámicamente cuándo el grafo global de la infraestructura se fractura en subredes independientes (Micro-redes o Islas), aislando las fallas para evitar el colapso matemático del simulador SCADA.

36.1 1. Importaciones y Dependencias Estratégicas

La arquitectura de este módulo requiere librerías especializadas en teoría de grafos y álgebra lineal dispersa:

- **networkx (nx)**: Dependencia absoluta para el análisis de topología de redes. Utilizado para encontrar Componentes Conexas (*Connected Components*) en tiempo $O(V + E)$ cuando un disyuntor cambia su estado a abierto.
- **scipy.sparse (csgraph)**: Requerido para manipular la Matriz Laplaciana de la red. Al tratar con hospitales o fábricas de decenas de miles de nodos, el uso de matrices dispersas reduce el consumo de RAM de gigabytes a megabytes.
- **concurrent.futures (ThreadPoolExecutor)**: Permite la asignación de hilos de CPU independientes para simular cada subred aislada en paralelo.

36.2 2. Variables Globales y Metadatos de Topología

Establece los axiomas de conductividad del grafo:

- **ESTADO_SECCIONADOR**: Enumerador booleano que define si la arista e_{ij} permite el flujo de electrones (**CLOSED** = 1) o presenta impedancia infinita (**OPEN** = 0).
- **Caché de Componentes Conexas (subgrafos_activos)**: Un arreglo dinámico en memoria que almacena los punteros a cada isla.^{el}éctrica viva en el ciclo de simulación actual.

36.3 3. Análisis de Clases y Funciones (Desglose Matemático)

La clase principal **SectionManager** aplica álgebra espectral de grafos para dominar la topología del Gemelo Digital:

36.3.1 A. Fractura Topológica y Matriz Laplaciana (evaluar_islas_electricas)

Cuando el **gestor_scada.py** reporta que un interruptor se ha disparado, el Gestor de Secciones recalcula la Matriz Laplaciana $L = D - A$ (donde D es la matriz de grados y A la matriz de adyacencia). El número de islas eléctricas (subredes aisladas) k se determina por la multiplicidad algebraica del valor propio $\lambda_0 = 0$ de la matriz Laplaciana:

$$L\mathbf{v} = \lambda\mathbf{v} \Rightarrow k = \dim(\ker(L)) \quad (56)$$

Si $k > 1$, la red se ha fracturado. El algoritmo extrae entonces los k subgrafos $G_1, G_2, \dots, G_k$.

36.3.2 B. Detección de Apagones (Nodos Huérfanos)

Para cada subgrafo G_n , el motor busca la existencia de al menos un "Nodo Fuente" (Transformador de acometida o Generador de Emergencia).

$$Estado(G_n) = \{ ENERGIZADO, si \exists nodo S \in G_n tal que S_{tipo} = FUENTE \vee APAGÓN(Blackout), encasocontrario \} \quad (57)$$

Si el estado es *Apagón*, el gestor anula todos los tensores de voltaje y corriente ($V = 0, I = 0$) para ese subgrafo sin desperdiciar ciclos de CPU intentando resolver las Ecuaciones de Kirchhoff.

36.3.3 C. Optimización por Computación Paralela (despachar_simulaciones)

La inversión de la matriz de admitancia nodal Y_{bus} para resolver el flujo de carga tiene una complejidad de $O(N^3)$. Al particionar la red, el Gestor de Secciones explota la propiedad matemática de que la suma de los cubos de las partes es menor que el cubo del todo:

$$O(|V_1|^3) + O(|V_2|^3) \ll O((|V_1| + |V_2|)^3) \quad (58)$$

Despachando cada subgrafo G_i a un núcleo distinto del procesador (*Multithreading*), reduciendo exponencialmente el tiempo de simulación.

36.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Dinámico y Conducido por Eventos de Falla (*Fault-Driven Lifecycle*)**. A diferencia del **motor_tiempo.py** que se ejecuta en cada *Tick*, el **SectionManager** es un árbitro pasivo. Solo interviene y consume ciclos de CPU cuando se detecta un cambio de estado topológico (ej. el usuario abre un tablero manualmente o una sobrecarga hace saltar un fusible termomagnético). Una vez que recalcula la topología, se vuelve a dormir.

36.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como autoridad topológica en **simulador/**:

- **Con gestor_scada.py**: Funciona como su asesor estratégico. Antes de que el SCADA intente calcular los voltajes de todo el edificio y colapse matemáticamente por una matriz singular (causada por cables desconectados), el **SectionManager** le entrega matrices pre-filtradas e invertibles.

36.6 6. Relación Exocrina (Con el resto del Ecosistema)

Opera como el **Señalizador de Desastres Ciberfísicos**:

- **Hacia ui/canvas.py y gestor_3d.py**: Cuando un subgrafo G_n es declarado en "Apagón", este gestor emite un evento de pérdida de energía (**blackout_sig**). El Canvas captura esta señal y cambia instantáneamente el color de todas las luces, motores y pantallas de esa zona arquitectónica a gris oscuro, proveyendo retroalimentación visual inmediata.

36.7 7. Aporte al Funcionamiento Global Total

El aporte de **section_manager.py** es la **Estabilidad Asintótica y la Simulación Realista de Fallas Cascada**.

En simuladores básicos, abrir un interruptor principal suele provocar un error de división por cero o la corrupción del modelo matemático completo. Este archivo dota a *EcoCircuit* de la resiliencia algorítmica necesaria para manejar apagones parciales, operaciones en isla de micro-redes solares y transferencias de carga de emergencia. Al convertir un problema NP-Difícil masivo en sub-problemas paralelizables, consagra la viabilidad y velocidad del Gemelo Digital a escala *Enterprise*.

37 Ruta y Archivo: EcoCircuit/ui/toolbar.py

En la capa de ergonomía de *EcoCircuit Enterprise*, este archivo funge como el **Cinturón de Herramientas Háptico y Panel de Acceso Heurístico** (*Quick-Access Toolbar & State Mutator*). Mientras que `menu_bar.py` contiene la topología exhaustiva de todos los comandos posibles, `toolbar.py` es la interfaz de alta frecuencia. Alberga las herramientas de interacción geométrica primaria (Dibujar Cable, Insertar Tablero, Mover, Borrar) y controla directamente la Máquina de Estados Finita (FSM) del lienzo de trabajo.

37.1 1. Importaciones y Dependencias Estratégicas

La arquitectura de este módulo exige un manejo rápido de iconos, acciones agrupadas y botones con estados mutuamente excluyentes:

- **PySide6.QtWidgets (QToolBar, QActionGroup, QToolButton):** Componentes estructurales. El `QActionGroup` es vital para implementar la exclusión lógica (cuando el usuario selecciona "Dibujar", la herramienta "Seleccionar" se desactiva automáticamente).
- **PySide6.QtGui (QIcon, QCursor):** Requeridos para cargar gráficos vectoriales escalables (.SVG) en la interfaz, manteniendo la nitidez en monitores 4K, y para alterar el puntero del ratón según la herramienta activa.
- **PySide6.QtCore (Signal, Slot):** Para la emisión asíncrona de los cambios de estado hacia los controladores geométricos.

37.2 2. Variables Globales y Metadatos de Estado

Define las tolerancias de renderizado y las constantes de operación:

- **TAMANO_ICONO_DEFAULT:** Constante (ej. 24×24 píxeles). Define el tamaño base de la matriz de botones, ajustable dinámicamente según la densidad de píxeles (DPI) del monitor.
- **Modos de Herramienta (TOOL_MODES):** Enumerador que clasifica las herramientas (ej. `SELECT_MODE`, `WIRE_MODE`, `EQUIPMENT_MODE`).

37.3 3. Análisis de Clases y Funciones (Desglose Matemático)

La clase `MainToolBar(QToolBar)` implementa una lógica de exclusión booleana estricta para garantizar que el sistema nunca caiga en un estado ambiguo:

37.3.1 A. Exclusión Mutua (*Mutually Exclusive States*)

Para el conjunto de herramientas de dibujo $T = \{t_1, t_2, \dots, t_n\}$, el `toolbar.py` garantiza matemáticamente que solo una herramienta pueda estar activa en cualquier instante de tiempo τ :

$$\sum_{i=1}^n Activo(t_i, \tau) = 1 \quad (59)$$

Esto se implementa mediante la vinculación de las acciones de la barra a un `QActionGroup` con la propiedad `setExclusive(True)`, previniendo que el usuario intente dibujar y borrar simultáneamente, lo cual corrompería la memoria del `canvas.py`.

37.3.2 B. Emisión del Cambio de Estado (`_al_cambiar_herramienta`)

Cuando la exclusión mutua se resuelve a favor de una nueva herramienta t_k , el método emite una señal codificada. Define una función de transición de estado δ para el motor espacial:

$$S_{t+1} = \delta(S_t, t_k) \quad (60)$$

Esta señal notifica al `canvas.py` para que cambie el comportamiento físico de su bucle de captura de ratón (por ejemplo, pasar de emitir Raycasts de selección a instanciar vectores de enrutamiento).

37.3.3 C. Inyección de Atajos Secundarios (*Toolbar Shortcuts*)

Aunque el `menu_bar.py` gestiona los atajos globales, la barra de herramientas vincula atajos de una sola tecla (ej. presionar 'W' para Wire/Cable, 'V' para Selección). Estas son capturadas en tiempo de $O(1)$ permitiendo un flujo de diseño ininterrumpido (CAD Workflow).

37.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Persistente y Acoplable** (*Persistent Dockable UI*). Se instancia junto con la `MainWindow`. Por defecto, se ancla en el eje superior (`Qt.TopToolBarArea`) o izquierdo (`Qt.LeftToolBarArea`), pero posee físicas que le permiten ser arrastrada a cualquier borde o flotar libremente sobre el monitor. Sobrevive a lo largo de toda la sesión del usuario.

37.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como comandante táctico dentro de `ui/`:

- **Con `canvas.py`:** Es su director directo. El Canvas es ciego respecto a "qué" debe hacer con el ratón hasta que el `toolbar.py` le inyecta la variable de estado activo.

37.6 6. Relación Exocrina (Con el resto del Ecosistema)

Actúa como un **Multiplexor de Comandos Rápidos**:

- Además de herramientas de dibujo, hospeda los "Botones de Pánico" "Lanzadores Rápido" (ej. Iniciar Simulación SCADA, Exportar IFC). Al hacer clic en ellos, dirige la señal hacia el `project_manager.py` o el `gestor_scada.py`, saltándose la tediosa navegación por menús desplegables.

37.7 7. Aporte al Funcionamiento Global Total

El aporte de `toolbar.py` es la **Velocidad Cinética y la Reducción de Fricción Operativa (UX)**.

En el diseño paramétrico y electromecánico, un ingeniero realiza miles de clics por hora. Obligar al usuario a navegar por submenús para cambiar entre la herramienta de "Mover" "Dibujar" niquilaría la productividad. Este módulo condensa las funciones de mayor entropía de la aplicación en una matriz visual inmediatamente accesible, dotando a *EcoCircuit* de la agilidad operativa que distingue a un software de dibujo industrial de primer nivel.

38 Ruta y Archivo: EcoCircuit/ui/view_3d_global.py

Dentro de la arquitectura de visualización de *EcoCircuit Enterprise*, este archivo funge como el **Motor de Renderizado Tridimensional y Visor Volumétrico Interactivo (3D Viewport Interface)**. Es el puente visual que integra las mallas matemáticas generadas por el *Core* con el hardware de gráficos por computadora (GPU). Su propósito es presentar al ingeniero la representación volumétrica real del edificio, superponiendo telemetría térmica, colisiones físicas y flujos de energía en un espacio tridimensional fidedigno.

38.1 1. Importaciones y Dependencias Estratégicas

La arquitectura de este módulo requiere librerías de renderizado científico y aceleración por hardware de alto rendimiento:

- **pyvistaqt (BackgroundPlotter):** Dependencia absoluta. Envuelve la biblioteca de visualización científica VTK (Visualization Toolkit) dentro de los componentes nativos de Qt, permitiendo la coexistencia fluida de interfaces de usuario tradicionales con un motor 3D acelerado por OpenGL/Vulkan.
- **numpy (np):** Requerido para manipular masivamente arreglos de coordenadas de vértices, vectores normales y matrices de transformación de color en tiempo de ejecución.
- **core.motor_3d:** Importado para consumir los algoritmos de barrido paramétrico y mallas volumétricas puras calculadas en el núcleo.

38.2 2. Variables Globales y Metadatos de Renderizado

Establece los parámetros estéticos y de rendimiento visual del entorno tridimensional:

- **FPS_TARGET:** Constante de tasa de refresco (ej. *60FPS*) para mantener una transición fluida en la rotación orbital de la cámara.
- **Mapas de Color Térmico (Colormaps):** Diccionarios estáticos que vinculan rangos de temperatura o sobrecarga de corriente con paletas cromáticas normalizadas (ej. escala *Cool-to-Warm* o *Jet*).

38.3 3. Análisis de Clases y Funciones (Desglose Matemático)

La clase `View3DGlobal(QWidget)` opera como la interfaz gráfica que gestiona el búfer de tramas de la GPU:

38.3.1 A. Inicialización del Contexto Gráfico y Canalización (`_inicializar_viewport`)

Al instanciarse, incrusta un lienzo VTK optimizado. Configura las matrices de proyección de la cámara utilizando coordenadas homogéneas para definir la perspectiva tridimensional:

$$\mathbf{v}_{clip} = \mathbf{P} \cdot \mathbf{V} \cdot \mathbf{M} \cdot \mathbf{v}_{modelo} \tag{61}$$

Donde **P** es la matriz de proyección, **V** la matriz de vista (cámara) y **M** la matriz de modelado del objeto electromecánico.

38.3.2 B. Actualización de Texturas y Mapeo Térmico (*Scalar Coloring*)

Durante la simulación en vivo, el SCADA emite métricas de temperatura para los conductores. El método `actualizar_telemetria_3d` inyecta estos escalares en las mallas poligonales, modificando el color de los elementos en tiempo real mediante una función de transferencia lineal:

$$Color(T) = C_{min} + \left(\frac{T - T_{min}}{T_{max} - T_{min}} \right) (C_{max} - C_{min}) \tag{62}$$

Esto permite visualizar de manera instantánea puntos de calentamiento excesivo (*Hotspots*) directamente sobre los modelos 3D de los tableros eléctricos.

38.3.3 C. Detección de Selección por Raycasting (`_al_hacer_clic`)

Cuando el usuario hace clic en un objeto dentro del espacio 3D, el visor lanza un rayo virtual desde la posición de la cámara del usuario hacia la profundidad de la escena (Algoritmo de Ray-Casting / Picking), determinando el primer triángulo intersectado para identificar el UUID de la entidad y notificar al Inspector.

38.4 4. El Orquestador y Ciclo de Vida

Su ciclo de vida es **Residente de Alta Demanda Gráfica (GPU-Bound Persistent View)**. Es instanciado por la `MainWindow` al arrancar. Permanece activo en una pestaña principal o ventana acoplada. Consume ciclos de GPU de forma constante para refrescar el búfer de pantalla ante cualquier movimiento de cámara, liberando recursos únicamente cuando la pestaña del visor es minimizada u ocultada.

38.5 5. Relación Endémica (Dentro de su Entorno Raíz)

Como cúspide visual dentro de `ui/`:

- **Con `canvas.py`:** Coexisten como las dos almas espaciales del programa. Mientras el Canvas domina la planimetría bidimensional ortogonal (plano arquitectónico), este visor domina la volumetría tridimensional y el comportamiento ciberfísico espacial.

38.6 6. Relación Exocrina (Con el resto del Ecosistema)

Actúa como el **Pantallazo de Realidad Ciberfísica**:

- **Hacia `core/motor_3d.py`:** Recibe los tensores geométricos crudos calculados por el núcleo aritmético y los convierte en objetos gráficos palpables (mallas de PyVista/VTK).
- **Hacia `controladores/gestor_scada.py`:** Se alimenta de los flujos de corriente simulados para renderizar efectos visuales (ej. partículas de falla o alertas volumétricas de cortocircuito).

38.7 7. Aporte al Funcionamiento Global Total

El aporte de `view_3d_global.py` es la ****Inmersión Espacial y la Validación Sensorial Definitiva****.

Un ingeniero puede calcular números perfectos en hojas de cálculo, pero la realidad física tridimensional de un edificio industrial frecuentemente revela choques espaciales, errores de enrutamiento o acumulación de calor imprevista que los planos 2D ocultan. Este visor dota a *EcoCircuit* de una ventana tridimensional interactiva de grado industrial, cerrando la brecha entre el diseño abstracto y la ejecución física real del proyecto.